



物联网虚拟仿真实验

MQTT 部分

(V1.1)

华清远见教育集团●研发中心



170-9108-5953



2306275952/2668462267



www.fsdev.com.cn



support@farsight.com.cn

目录

第 1 章 MQTT 简介	1
1.1 MQTT 基础知识	1
1.2 本系统 MQTT 架构.....	5
1.3 本系统 MQTT 协议格式.....	8
第 2 章 MQTT 代理服务器部署及客户端软件安装.....	10
2.1 实验目的.....	10
2.2 实验步骤.....	10
2.3 实验结果.....	12
第 3 章 物联网网关 MQTT 通讯协议配置实验.....	12
3.1 实验目的.....	12
3.2 实验步骤.....	13
第 4 章 MQTT 消息订阅实验.....	15
4.1 实验目的.....	15
4.2 实验步骤.....	15
4.3 实验结果.....	17
第 5 章 MQTT 消息发布实验.....	17
5.1 实验目的.....	17
5.2 实验步骤.....	17
5.3 实验结果.....	19
第 6 章 排风系统控制实验.....	19
6.1 实验目的.....	19
6.2 实验步骤.....	20
6.3 实验结果.....	25
第 7 章 智能报警系统控制实验.....	26
7.1 实验目的.....	26
7.2 实验步骤.....	26
7.3 实验结果.....	30
第 8 章 微型电控锁系统控制实验.....	31
8.1 实验目的.....	31
8.2 实验步骤.....	31
8.3 实验结果.....	35
第 9 章 电灯系统控制实验.....	36
9.1 实验目的.....	36
9.2 实验步骤.....	36

9.3 实验结果.....	40
第 10 章 电动窗帘控制实验.....	40
10.1 实验目的.....	40
10.2 实验步骤.....	40
10.3 实验结果.....	44
第 11 章 RFID 远程控制门禁实验.....	45
11.1 实验目的.....	45
11.2 实验步骤.....	45
11.3 实验结果.....	49
第 12 章 13.56MHz RFID 标签远程读取实验.....	49
12.1 实验目的.....	49
12.2 实验步骤.....	49
12.3 实验结果.....	53
第 13 章 13.56MHz RFID 电子钱包实验.....	53
13.1 实验目的.....	53
13.2 实验步骤.....	53
13.3 实验结果.....	56
第 14 章 915MHz RFID 库存盘点实验.....	56
14.1 实验目的.....	56
14.2 实验步骤.....	57
14.3 实验结果.....	60
第 15 章 2.4GHz RFID 停车场闸机控制系统实验.....	61
15.1 实验目的.....	61
15.2 实验步骤.....	61
15.3 实验结果.....	65
第 16 章 基于一键还原节点的多传感器控制实验.....	65
16.1 实验目的.....	65
16.2 实验步骤.....	65
16.3 实验结果.....	68

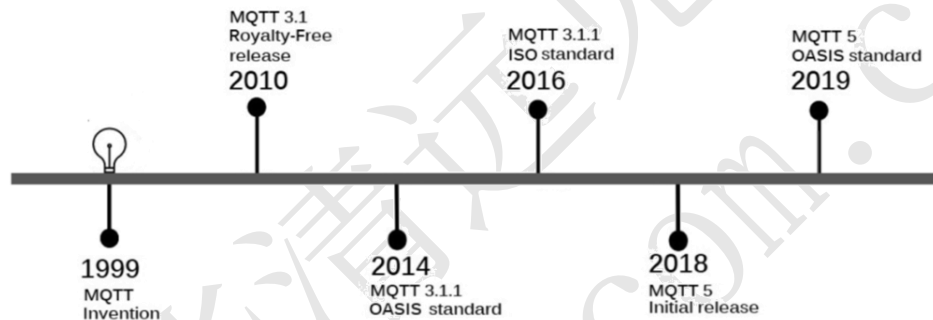
第 1 章 MQTT 简介

1.1 MQTT 基础知识

MQTT 协议由 Andy Stanford-Clark (IBM) 和 Arlen Nipper (Arcom, 现为 Cirrus Link) 于 1999 年发明。他们需要一种通过卫星连接石油管道的协议, 以最大限度地减少电池损耗和带宽。所以他们为这个协议规定了几种要求:

- 这个协议必须易于实现;
- 这个协议中的数据必须易于传输, 消耗成本小;
- 这个协议必须提供服务质量管理;
- 这个协议必须支持连续的会话控制
- 假设数据不可知, 不强求传输数据的类型与格式, 保持灵活性。

该规定也适合目前的物联网信息传输场景。因此, 经过多年的发展, MQTT 成为了物联网信息传输的主要技术之一。



MQTT 所具有的适用于物联网特定需求的特点和功能, 使其成为物联网领域最佳的协议之一。它的主要特点包括:

轻量级: 物联网设备通常在处理能力、内存和能耗方面受到限制。MQTT 开销低、报文小的特点使其非常适合这些设备, 因为它消耗更少的资源, 即使在有限的能力下也能实现高效的通信。

可靠: 物联网网络常常面临高延迟或连接不稳定的情况。MQTT 支持多种 QoS 等级、会话感知和持久连接, 即使在困难的条件下也能保证消息的可靠传递, 使其非常适合物联网应用。

安全通信: 安全对于物联网网络至关重要, 因为其经常涉及敏感数据的传输。为确保数据在传输过程中的机密性, MQTT 提供传输层安全 (TLS) 和安全套接层 (SSL) 加密功能。此外, MQTT 还通过用户名/密码凭证或客户端证书提供身份验证和授权机制, 以保护网络及其资源的访问。

双向通信: MQTT 的发布-订阅模式为设备之间提供了无缝的双向通信方式。客户端既可以向主题发布消息, 也可以订阅接收特定主题上的消息, 从而实现了物联网生态系统中的高效数据交换, 而无需直接将设备耦合在一起。这种模式也简化了新设备的集成, 同时保证了系统易于扩展。

连续、有状态的会话: MQTT 提供了客户端与 Broker 之间保持有状态会话的能力, 这使得

系统即使在断开连接后也能记住订阅和未传递的消息。此外，客户端还可以在建立连接时指定一个保活间隔，这会促使 Broker 定期检查连接状态。如果连接中断，Broker 会储存未传递的消息（根据 QoS 级别确定），并在客户端重新连接时尝试传递它们。这个特性保证了通信的可靠性，降低了因间断性连接而导致数据丢失的风险。

大规模物联网设备支持：物联网系统往往涉及大量设备，需要一种能够处理大规模部署的协议。MQTT 的轻量级特性、低带宽消耗和对资源的高效利用使其成为大规模物联网应用的理想选择。通过采用发布-订阅模式，MQTT 实现了发送者和接收者的解耦，从而有效地减少了网络流量和资源使用。此外，协议对不同 QoS 等级的支持使得消息传递可以根据需求进行定制，确保在各种场景下获得最佳的性能表现。

语言支持：物联网系统包含使用各种编程语言开发的设备和应用。MQTT 具有广泛的语言支持，使其能够轻松与多个平台和技术进行集成，从而实现了物联网生态系统中的无缝通信和互操作性。您可以阅读我们的 MQTT 客户端编程系列文章，学习如何在 PHP、Node.js、Python、Golang、Node.js 等编程语言中使用 MQTT。

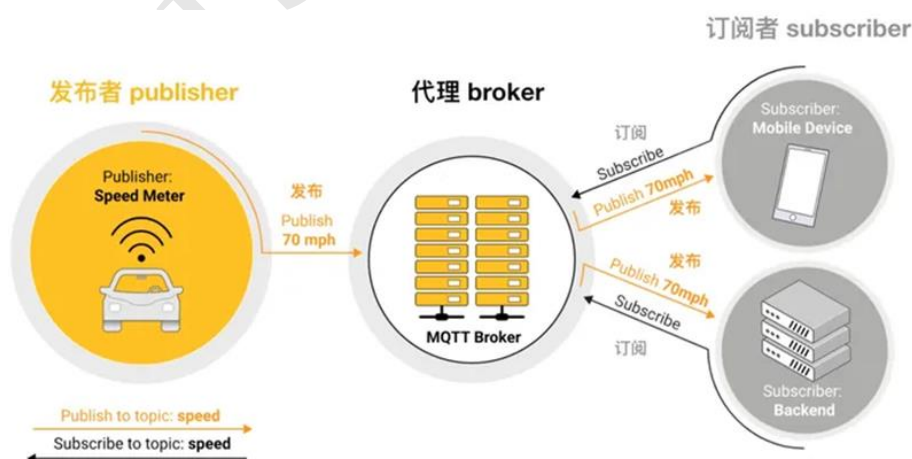
要了解 MQTT 的工作原理，首先需要掌握以下几个概念：MQTT 客户端、MQTT Broker、发布-订阅模式、主题、QoS。

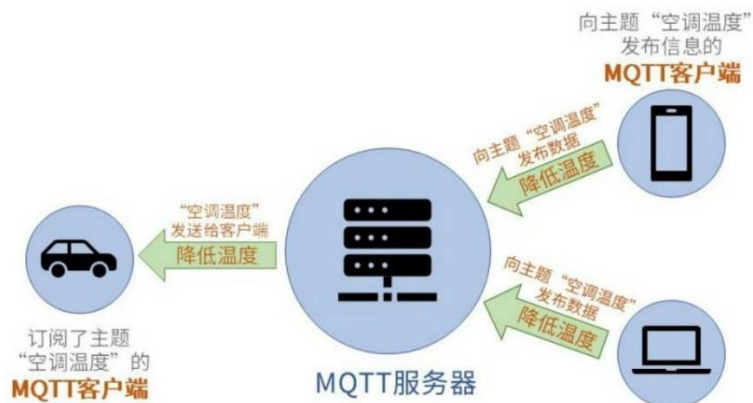
MQTT 客户端：任何运行 MQTT 客户端库的应用或设备都是 MQTT 客户端。例如，使用 MQTT 的即时通讯应用是客户端，使用 MQTT 上报数据的各种传感器是客户端，各种 MQTT 测试工具也是客户端。

MQTT Broker：MQTT Broker 是负责处理客户端请求的关键组件，包括建立连接、断开连接、订阅和取消订阅等操作，同时还负责消息的转发。一个高效强大的 MQTT Broker 能够轻松应对海量连接和百万级消息吞吐量，从而帮助物联网服务提供商专注于业务发展，快速构建可靠的 MQTT 应用。

发布-订阅模式：发布-订阅模式与客户端-服务器模式的不同之处在于，它将发送消息的客户端（发布者）和接收消息的客户端（订阅者）进行了解耦。发布者和订阅者之间无需建立直接连接，而是通过 MQTT Broker 来负责消息的路由和分发。

下图展示了 MQTT 发布/订阅过程。





pub/sub 最重要的方面是 publisher 与 subscriber 的解耦，这种耦合度有下面三个维度：

空间解耦：publisher 与 subscriber 并不知道对方的存在，例如不会有 IP 地址和端口的交互，也更不会有消息的交互。

时间解耦：publisher 与 subscriber 并不一定需要同时运行。

同步解耦：两个组件的操作都不会在发布或者接收过程中产生中断。

这种特点会带来如下的好处：

发布者与订阅者只需要知道同一个消息代理即可。

发布者和订阅者不需要直接交互。

发布者和订阅者不需要同时在线。

在 MQTT 中，话题名是由一串 UTF-8 编码的字符串组成的，例如：

kingdom/phylum/class/order/family/genus/species（界门纲目科属种）

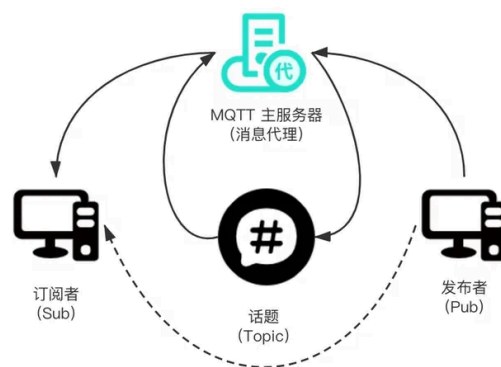
这同时也是一个多级主题的示例，在 MQTT 中，允许存在多级话题，可以向任何一级发布消息，每一个话题级别由/分割。也允许用通配符进行灵活的话题订阅和发布。

通配符分为两种，分别是单层通配符(+)和多层通配符(#):

单层通配符(+): 单层通配符可以替代主题名中的任意一个层级用来指代本层级的所有话题层级名，单层通配符前后必须是正斜杠/(单层通配符处于主题名末尾时除外)。如使用

kingdom/phylum/+/order 作为主题名订阅时，匹配结果如下

Topic	Status
kingdom/phylum/class/order	Yes
kingdom/phylum/aaaaa/order	Yes
kingdom/phylum/aaaaa/bbbbbb	No
kingdom/bbbbbbb/aaaaa/order	No
kingdom/phylum/aaaaa/bbbbbb/order	No



在 pub/sub 的架构模式中，Broker 扮演着至关重要的作用，其中非常重要的一点就是 Broker 能够对消息进行过滤，使每个订阅者只接收自己感兴趣的消息。Broker 有几个可以过滤的选项：

基于主题的过滤：MQTT 是基于 subject 的消息过滤的，每条消息都会有一个 topic，接收客户端会向 Broker 订阅感兴趣的 topic，订阅后，Broker 就会确保客户端收到发布到 topic 中的消息。

基于内容的过滤：在基于内容的过滤中，Broker 会根据特定的内容过滤消息，接收客户端会经过过滤他们感兴趣的内容。这种方法的一个显著的缺点就是必须事先知道消息的内容，不能加密或者轻易修改。

基于类型的过滤：在使用面向对象的语言时，基于消息（事件）的类型过滤是一种比较常见的过滤方式。

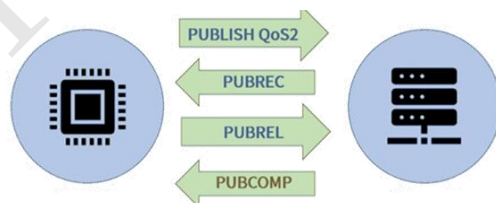
MQTT 提供了三种服务质量（QoS），在不同网络环境下保证消息的可靠性。

QoS 0：消息最多发 1 次，占用的网络资源最低。发送端一旦发送完消息后，就完成任务了。发送端不会检查发出的消息能否被正确接收到。

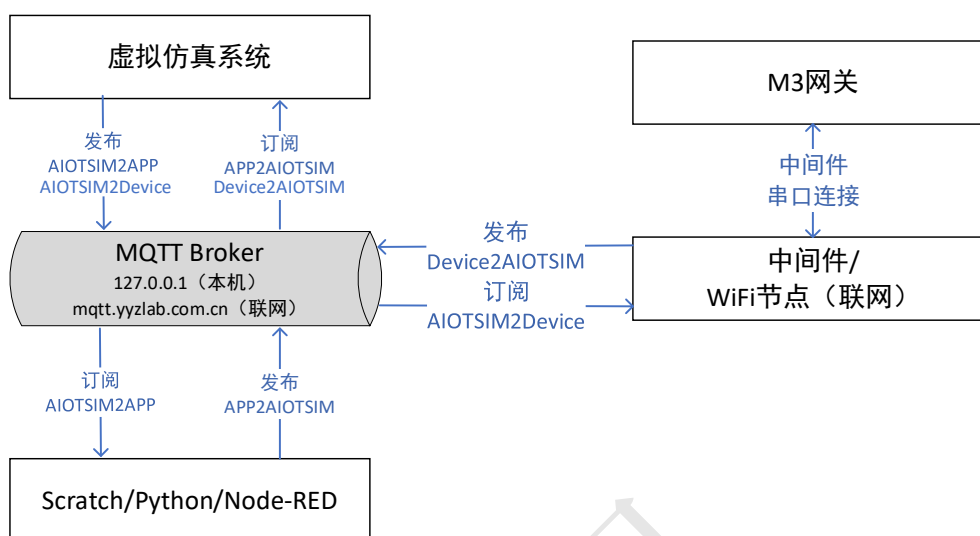
QoS 1：消息最少发 1 次，但是有可能出现接收端反复接收同一消息的情况。



QoS 2：消息保证收 1 次。

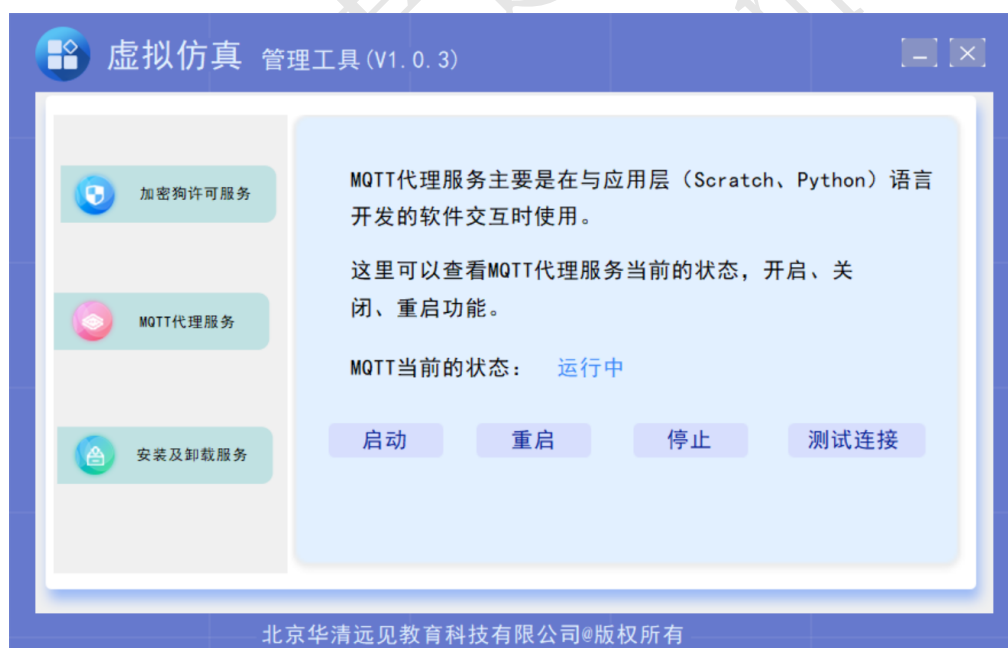


1.2 本系统 MQTT 架构



本系统包括：虚拟仿真系统、MQTT Broker、Scratch、Node-RED、Python、中间件、WiFi 节点、M3 网关等。

在安装单机版虚拟仿真系统时，已自动安装 MQTT Broker 服务如下图所示。网页版未在本地安装，也不能将本机作为 MQTT Broker 服务器，只能将 mqtt.yyzlab.com.cn 作为服务器。



因此，本机可作为 MQTT Broker 使用，从而存储转发其他软件订阅或发送的对应话题内的消息。mqtt.yyzlab.com.cn 也可作为 MQTT Broker 使用，供远程网络连接。

虚拟仿真系统中，网关或 WiFi 节点向 Broker 订阅 APP2AIOTSIM 话题，从而接收来自 Scratch 或 Python 发布的信息，网关向 Broker 发布 AIOTSIM2APP 话题，从而让 Scratch、Node-RED 或 Python 接收相关的话题。WiFi 节点只能通过网络连接到 mqtt.yyzlab.com.cn 上。注意，

当使用 127.0.0.1 本机地址作为 Broker 时，使用默认话题即可，当使用 mqtt.yyzlab.com.cn 作为 Broker 服务器时，话题和 ClientID 必须能够被唯一识别，从而避免与他人使用的话题和 ID 冲突。例如，可以通过点击“生成”按钮将话题和 ClientID 设置为下面的形式。在使用 Scratch、Node-RED、Python 时，ClientID 也不应与此相同，必须独立生成。另外需要注意单机版的虚拟仿真系统和网页版的虚拟仿真系统用的 MQTT 的端口不一样，单机版为 1883，网页版为 9002。



The image shows a 'Mqtt配置' (MQTT Configuration) window. It contains fields for MQTT settings: IP address (mqtt.yyzlab.com.cn), Port (1883), ClientID (31131f262ef8340e40c0c3e331e1d0ae), Username, and Password. There are buttons for '连接' (Connect), '生成' (Generate), and '订阅' (Subscribe). Below these are sections for '与应用层、云端交互的Topic:' (Topics for application layer and cloud interaction) and '与硬件交互的Topic:' (Topics for hardware interaction), each with '订阅' and '发布' (Publish) fields and '生成Topic' and '订阅' buttons. A watermark 'farsight.cn' is visible in the background.

MQTT: 网页版为9002

IP地址: mqtt.yyzlab.com.cn 端口: 1883

ClientID: 31131f262ef8340e40c0c3e331e1d0ae 生成

用户名: 请输入用户名

密码: 请输入密码

连接

与应用层、云端交互的Topic:

订阅: 9207/APP2AIOTSIM 复制

发布: 9207/AIOTSIM2APP 复制

生成Topic 订阅

与硬件交互的Topic:

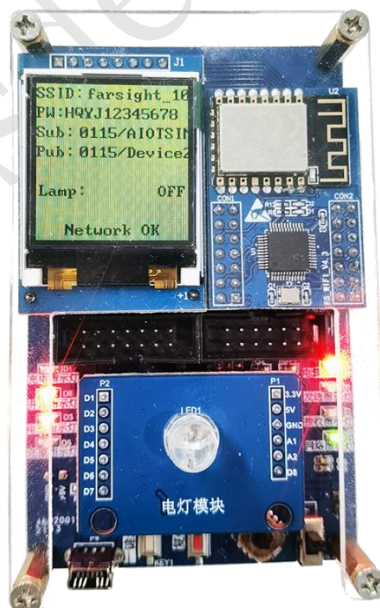
订阅: 0115/Device2AIOTSIM 复制

发布: 0115/AIOTSIM2Device 复制

生成Topic 订阅

微信小程序与虚拟仿真交互

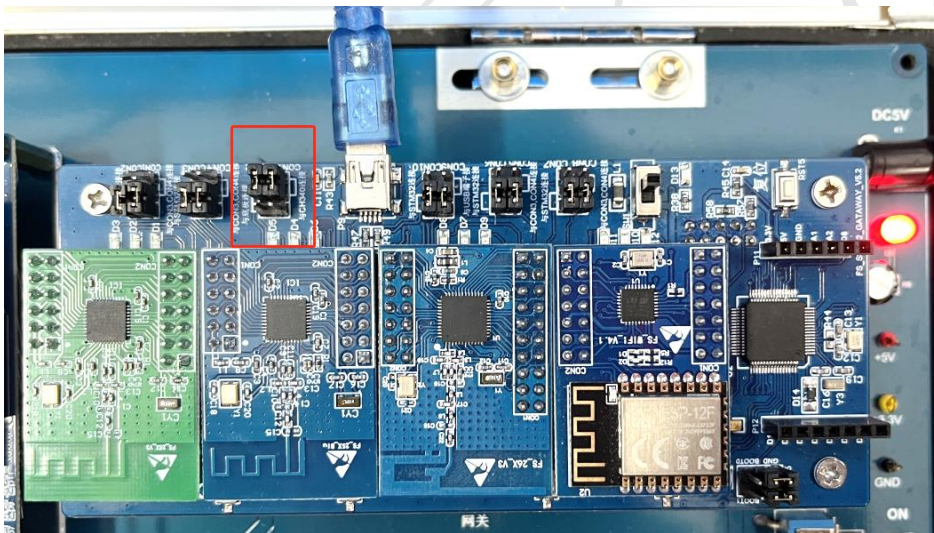
本系统可通过 mqtt.yyzlab.com.cn 作为 Broker，将 WiFi 节点和虚拟仿真系统进行连接。当虚拟仿真系统中的某个传感器被设置成实际硬件时，发往该传感器的命令或来自该传感器的数据将通过与硬件交互的 Topic 进行交互。WiFi 节点将订阅或发布对应的 Topic，从而将节点上的传感器的数据进行交互。



本系统还可以通过中间件连接真实的硬件设备，即 M3 网关。中间件界面如下。



M3 网关通过串口接收到中间件的数据后会转发给对应的传感器。



当需要接收硬件传感器发来的数据时，首先需要将虚拟仿真系统中的器件设置为实际硬件。



然后中间件会将通过串口接收到的数据转发给 Device2AIOTSIM 这个话题，供虚拟仿真系统接收。

当需要向硬件执行器发送命令时，首先需要将虚拟仿真系统中的器件设置为实际硬件，然后当利用 Scratch、Node-RED 或 Python 通过 APP2AIOTSIM 话题向该器件发送控制指令时，该指令会自动转发到 AIOTSIM2Device 上，供中间件接收，并通过串口转发给真实硬件。

1.3 本系统 MQTT 协议格式

MQTT 数据传输采用了 JSON 格式，格式如下：{"key":value,"id":int}。id 表示的是设备的 id，用于区分多个同一种类型的设备。例如：当同时存在多个 CO2 传感器（CO2 传感器_0、CO2 传感器_1、CO2 传感器_2），这里 id 分别使用 0、1、2。

M4 网关单元 JSON 数据格式			
设备名称	设备类型	数据传输格式({"key":value,"id":int})	说明
CO2 传感器	数值传感器	{"co2":float,"id":int}	传感器的数据传输格式： {"key":value,"id":int}，value 为 float 类型的传感器的数值，id 表示的是设备的 id，用于区分多个同一种类型的设备。
土壤温湿度传感器		{"soiltem":float,"soilhum":float,"id":int}	
光照传感器		{"light":float,"id":int}	
PM2.5 传感器		{"PM2.5":float,"id":int}	
空气温湿度传感器		{"tem":float,"hum":float,"id":int}	
可燃气体传感器	IO 传感器	{"flamGas":bool,"id":int}	超标:{"flamGas":true,"id":int} 未超标:{"flamGas":false,"id":int}
雨雪传感器		{"sleet":bool,"id":int}	有雨雪:{"sleet":true,"id":int} 无雨雪:{"sleet":false,"id":int}
烟雾传感器		{"smog":bool,"id":int}	超标:{"smog":true,"id":int} 未超标:{"smog":false,"id":int}
人体红外传感器		{"infrared":bool,"id":int}	有人:{"infrared":true,"id":int} 无人:{"infrared":false,"id":int}
火焰传感器	IO 执行器	{"flame":bool,"id":int}	未超标:{"flame":false,"id":int} 超标:{"flame":true,"id":int}
风扇		{"fan":bool,"id":int}	打开:{"fan":true,"id":int} 关闭:{"fan":false,"id":int}
喷淋		{"irrigated":bool,"id":int}	打开:{"irrigated":true,"id":int} 关闭:{"irrigated":false,"id":int}
遮阳板		{"sunshade":string,"id":int}	正转:{"sunshade":"forward","id":int} 反转:{"sunshade":"reverse","id":int} 停止:{"sunshade":"stop","id":int}
声光报警器		{"alarm":bool,"id":int}	打开:{"alarm":true,"id":int} 关闭:{"alarm":false,"id":int}
电灯		{"lamp":bool,"id":int}	打开:{"lamp":true,"id":int} 关闭:{"lamp":false,"id":int}
门锁		{"doorLock":bool,"id":int}	打开:{"doorLock":true,"id":int} 关闭:{"doorLock":false,"id":int}

道闸		{"barrierGate":bool,"id":int}	打开:{"barrierGate":true,"id":int} 关闭:{"barrierGate":false,"id":int}
蜂鸣器		{"beep":bool,"id":int}	打开:{"beep":true,"id":int} 关闭:{"beep":false,"id":int}
继电器		{"relay":bool,"id":int}	打开:{"relay":true,"id":int} 关闭:{"relay":false,"id":int}
RFID 识别模块	RFID 模块	{"RFID_125K":string,"id":int}	
		{"RFID_NFC":string,"id":int}	
		{"RFID_24G":string,"id":int}	
		{"RFID_1356M":string,"id":int}	
		{"RFID_915M":string,"id":int}	
一键还原节点控制板 JSON 数据格式			
设备名称	设备类型	数据传输格式({"key":value,"id":int})	说明
温湿度传感器	数值传感器	{"obn_tem":float,"obn_hum":float,"id":int }	{"obn_tem":-27.5,"obn_hum":2.5,"id":0}
光敏传感器		{"obn_photosensitive": float,"id": int }	{"obn_photosensitive":99.3,"id":0}
霍尔传感器		{"obn_hallElement": float,"id": int }	{"obn_hallElement":1.0,"id":0}
三轴传感器		{"obn_xAxis": float,"obn_yAxis": float,"obn_zAxis": float,"id":int}	{"obn_xAxis":325.6,"obn_yAxis":325.6,"obn_zAxis":325.6,"id":0}
九轴传感器		{"obn_pitchAxis": float,"obn_yawAxis": float,"obn_rollAxis": float,"id":0}	{"obn_pitchAxis":286.0,"obn_yawAxis":286.0,"obn_rollAxis":286.0,"id":0}
心率传感器		{"obn_heartRate":float,"id":0}	{"obn_heartRate":68.2,"id":0}
雨雪传感器		{"obn_sleet":float,"id":int}	
可燃气体传感器		{"obn_flamGas":float,"id":int}	{"obn_flamGas":2159.6,"id":0}
温湿度传感器		{"obn_tem":float,"obn_hum":float,"id":int}	
火焰传感器		{"obn_flame":float,"id":int}	
人体红外传感器	IO 传感器	{"obn_infrared":bool,"id":int}	有人:{"obn_infrared":true,"id":0 } 无人:{"obn_infrared":false,"id":0 }
触摸按键		{"obn_touch_key":bool,"id":int}	按下:{"obn_touch_key":true,"id":0} 抬起:{"obn_touch_key":false,"id":0}
声控开关		{"obn_voice_switch":bool,"id":int}	有声:{"obn_voice_switch":true,"id":0} 无声:{"obn_voice_switch":false,"id":0}
震动传感器		{"obn_vshake":bool,"id":int}	有震动:{"obn_vshake":true,"id":0} 无震动:{"obn_vshake":false,"id":0}
光电开关		{"obn_photoSwitch":bool,"id":int}	有光 电:{"obn_photoSwitch":true,"id":0} 无光 电:{"obn_photoSwitch":false,"id":0}
继电器	执行器	{"obn_relay":bool,"id":int}	开启:{"obn_relay":true,"id":0} 关闭:{"obn_relay":false,"id":0}
数码管		{"obn_nixieTube":str,"id":int}	{"obn_nixieTube":"0","id":0}

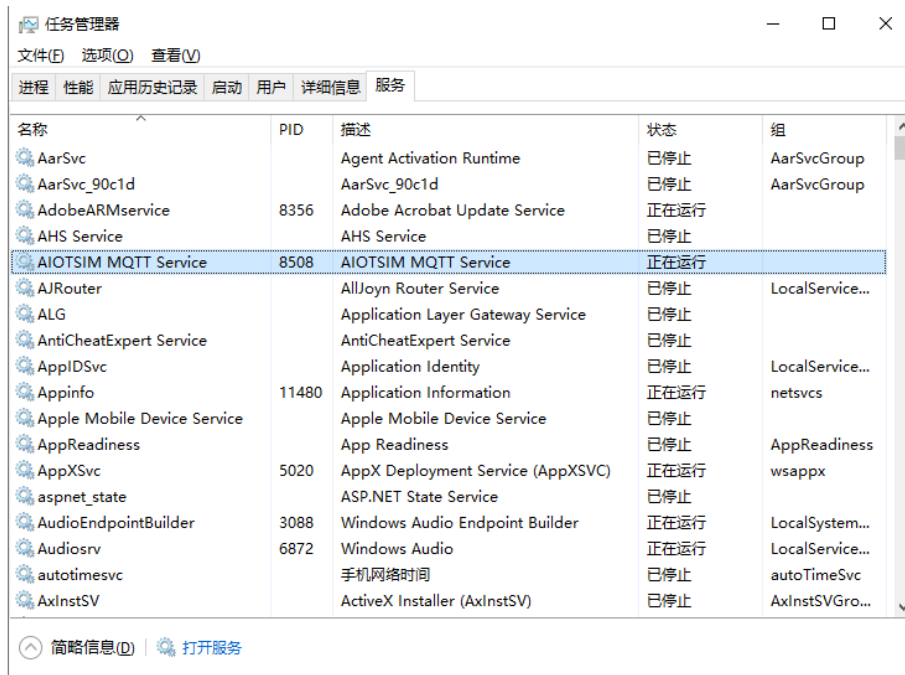
第 2 章 MQTT 代理服务器部署及客户端软件安装

2.1 实验目的

部署本地 MQTT 代理服务器，安装 MQTT 客户端软件，以便进行 MQTT 实验。

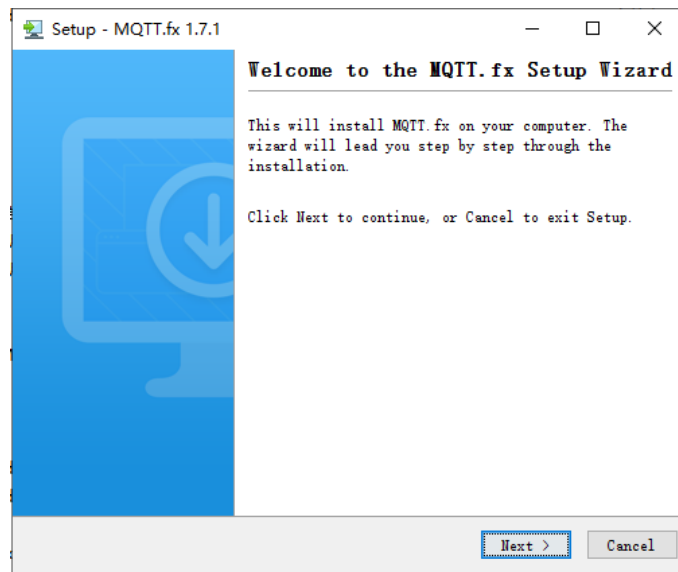
2.2 实验步骤

右键桌面下方任务栏，选择任务管理器，选择服务，找到 AIOTSIM MQTT Service 右键选择开始，打开 MQTT 本地代理服务。有此服务才能使用本地 MQTT 代理。

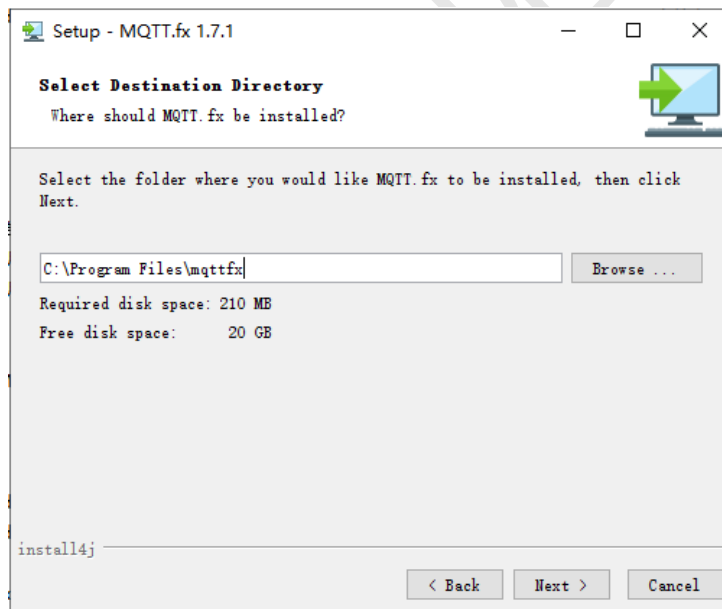


安装 MQTT 客户端软件 MQTT.fx

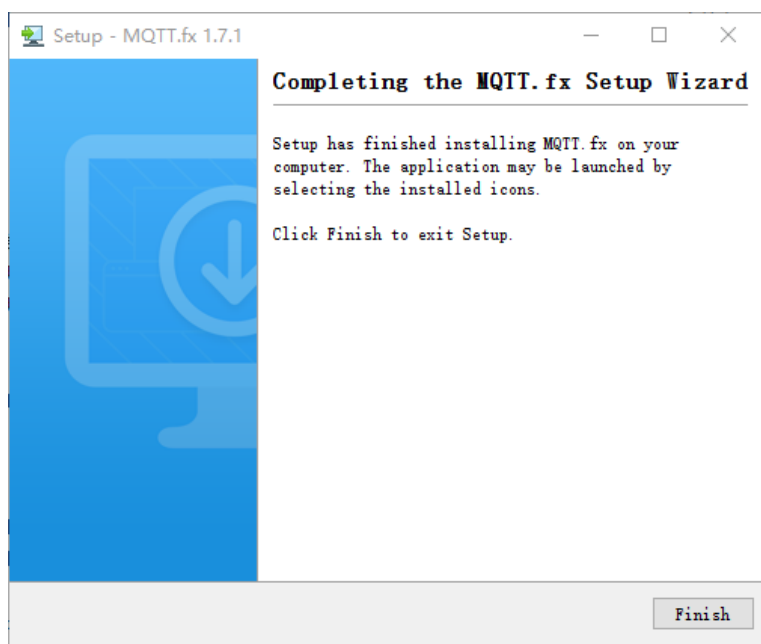
mqttfx-1.7.1-windows-x64.exe	2021/1/13 13:55	应用程序	52,209 KB
------------------------------	-----------------	------	-----------



选择安装路径，点击 Next



点击 Finish 完成安装

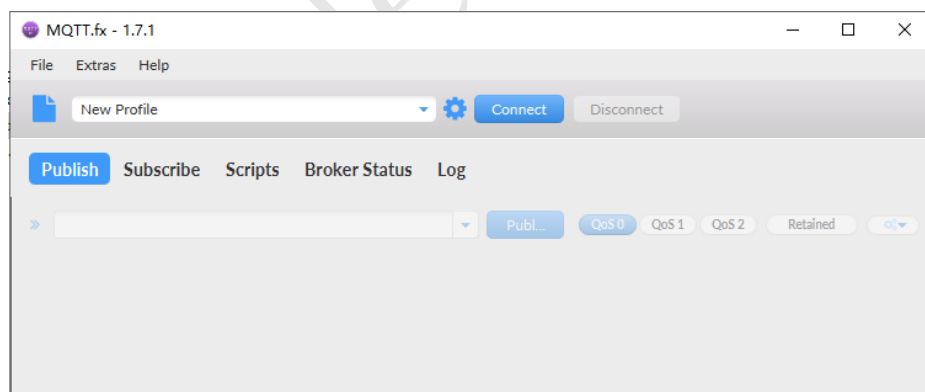


2.3 实验结果

双击 MQTT.fx 安装路径下的 MQTT.fx.exe 或桌面上的快捷方式，可打开 MQTT 客户端软件

> mqttfx >

名称	修改日期	类型	大小
.install4j	2021/1/13 15:10	文件夹	
jre	2021/1/13 15:09	文件夹	
lib	2021/1/13 15:09	文件夹	
MQTT.fx-afx.jar	2018/9/28 20:26	Executable Jar File	1,585 KB
mqttfx.exe	2018/9/28 20:26	应用程序	431 KB
uninstall.exe	2018/9/28 20:26	应用程序	699 KB



第 3 章 物联网网关 MQTT 通讯协议配置实验

3.1 实验目的

熟悉网关 MQTT 通讯协议的配置。

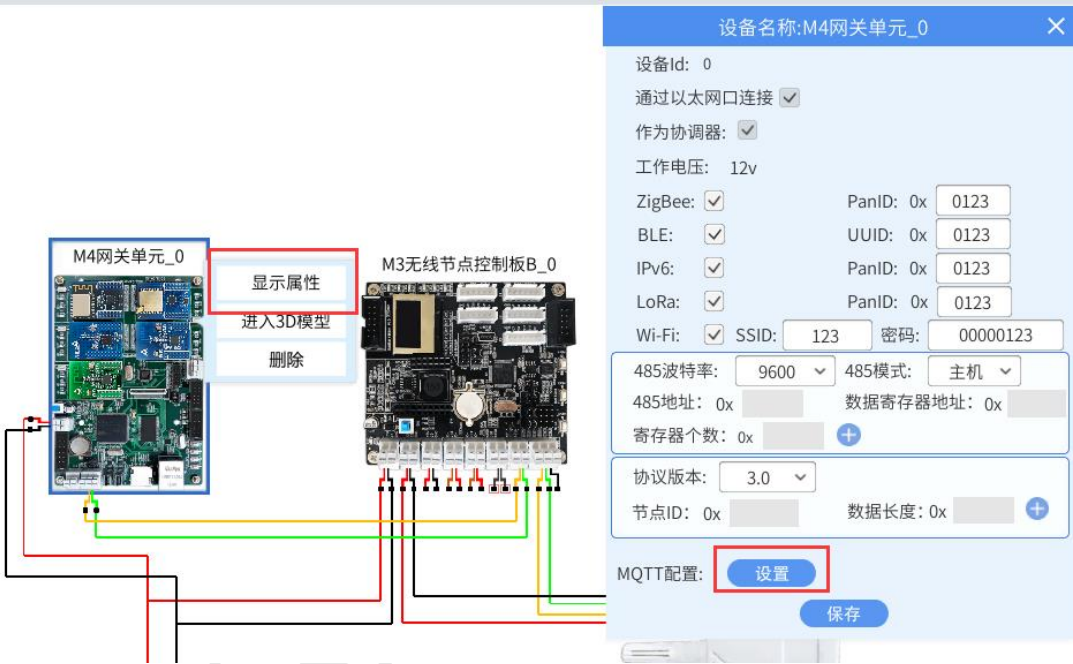
3.2 实验步骤

新建实验进行接线并配置通信协议，或打开已有实验，本章实验以预设实验为例，打开预设实验中 1_智能风扇控制系统。

预设实验

实验名称		
1_智能风扇控制系统	打开	移除
2_智能家居	打开	移除
3_智能农业	打开	移除
4_智能图书馆	打开	移除

打开预设实验后点击 M4 网关选择显示属性，点击 MQTT 配置的设置按钮



在 MQTT 配置界面 IP 地址输入本机 IP，默认为 127.0.0.1 即为本地回环地址，端口号默认为 1883，ClientID 点击生成即随机生成，不填系统默认补充，用户名和密码默认为空，点击连接按钮即可看到提示连接成功，如提示连接失败，确认上一节实验 MQTT 本地代理服务打开。订阅和发布的 topic 输入框填写了默认的 topic，可自定义修改。这里的订阅指的是接收其他客户端发布的话题，发布指的是自己要发布的话题，可被其他客户端订阅。在 MQTT 连接成功后，点击订阅即完成 topic 订阅和发布 topic 的设置。

MQTT配置窗口，包含以下配置项：

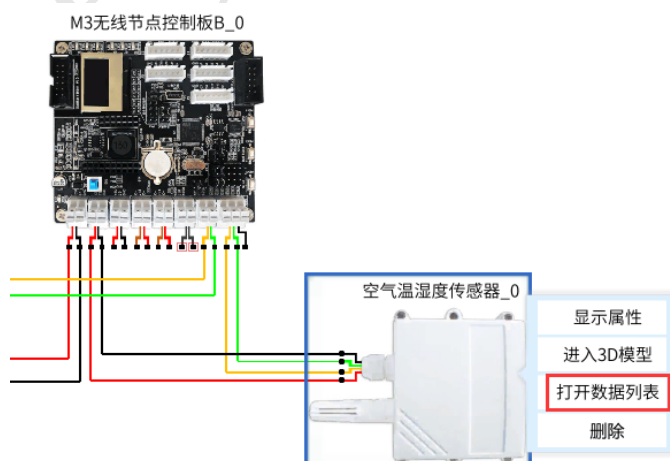
- MQTT:
 - IP地址: 127.0.0.1
 - 端口: 1883
 - ClientID: 请输入或者生成ClientID (生成按钮)
 - 用户名: 请输入用户名
 - 密码: 请输入密码
 - 连接按钮 (红色框)
- 与应用层、云端交互的Topic:
 - 订阅: APP2AIOTSIM (复制按钮)
 - 发布: AIOTSIM2APP (复制按钮)
 - 生成Topic按钮
 - 订阅按钮 (红色框)
- 与硬件交互的Topic:

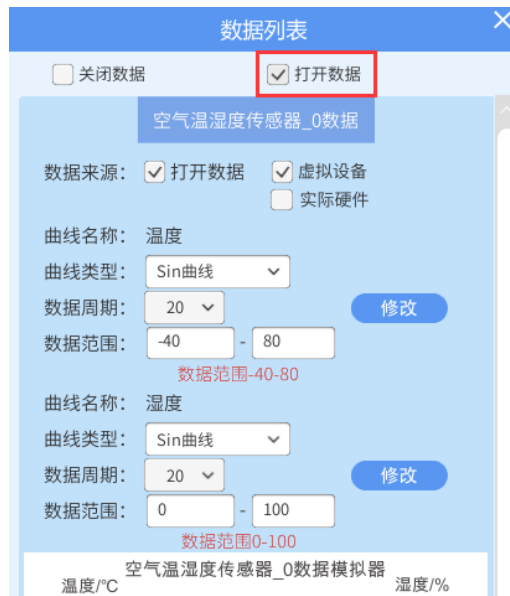
注意，单机版软件的 IP 地址 127.0.0.1 代表本机地址，端口为 1883。网页版中需要输入 MQTT.yyzlab.com.cn 作为服务器地址，端口为 9002。同时，在网页版中，需要将话题设置为独一无二的名称，防止与其他用户发布的话题消息冲突。如下图所示。

MQTT配置窗口，包含以下配置项：

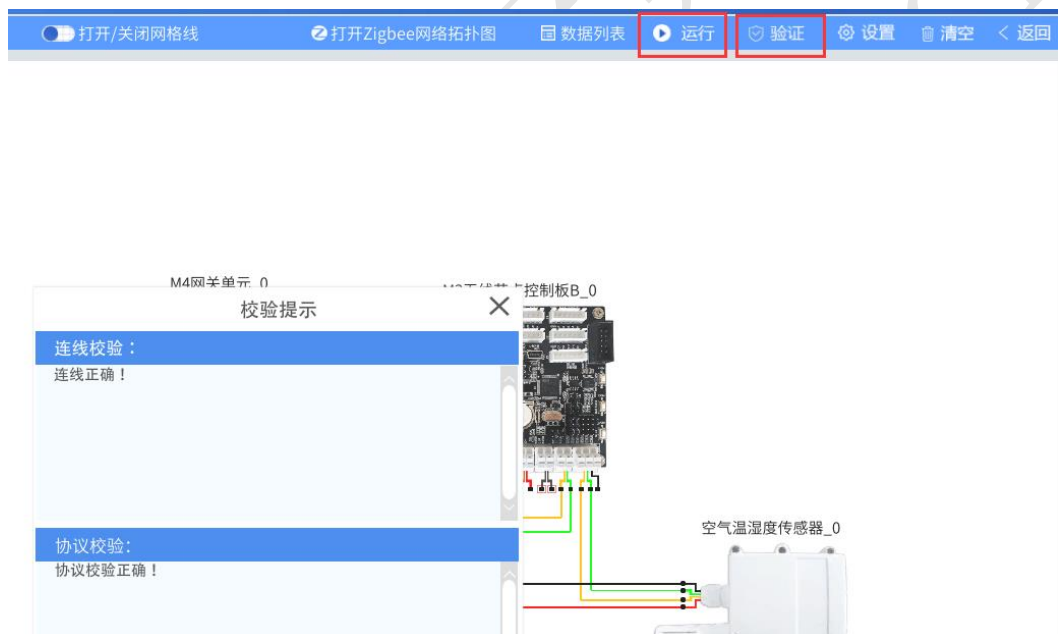
- MQTT:
 - IP地址: wss:// mqtt.yyzlab.com.cn
 - Path: mqtt
 - 端口: 9002
 - MQTT版本: 3.1.1
 - ClientID: 33574cfa21e85f0e7a505ccabd84575 (生成按钮)
 - 用户名: 请输入用户名
 - 密码: 请输入密码
 - 连接按钮
- 与应用层、云端交互的Topic:
 - 订阅: 1703059472688/APP2AIOTSIM (复制按钮)
 - 发布: 1703059472688/AIOTSIM2APP (复制按钮)
 - 生成topic按钮
 - 订阅按钮

点击空气温湿度传感器，选择打开数据列表，或上方的数据列表按键，点击打开数据，数据变化曲线可自定义修改。





点击验证，验证通过后，点击运行。



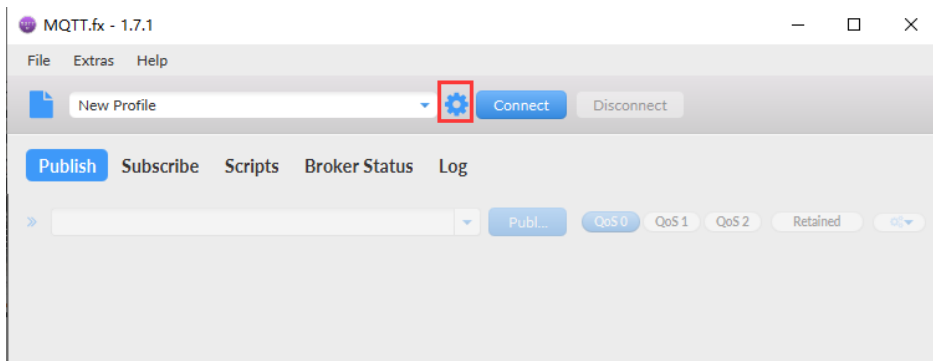
第 4 章 MQTT 消息订阅实验

4.1 实验目的

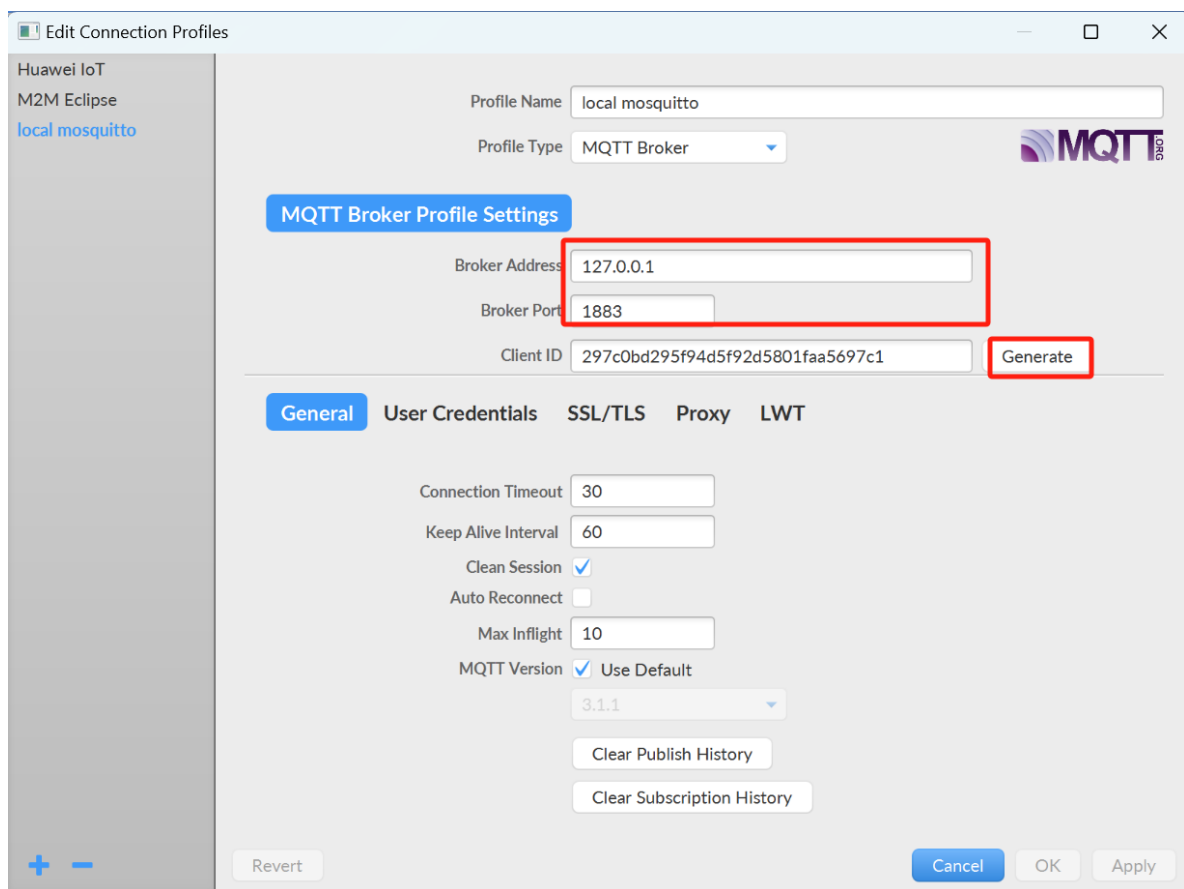
学习了解 MQTT 消息订阅的流程。

4.2 实验步骤

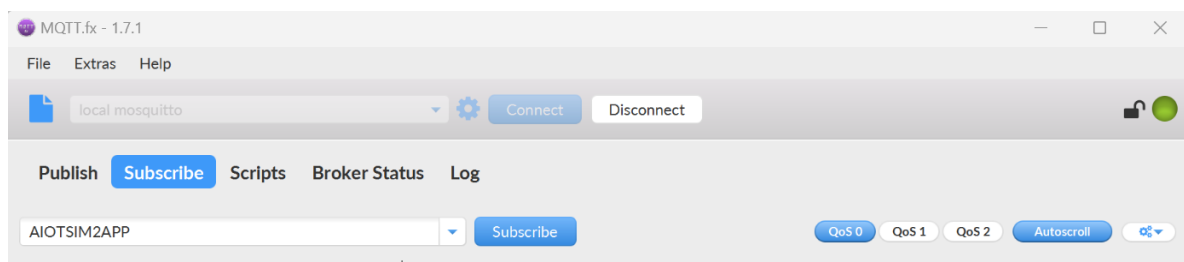
在上一节实验配置好网关 MQTT 配置，并点击运行后，找到安装好的 MQTT.fx 客户端软件，双击打开，点击设置



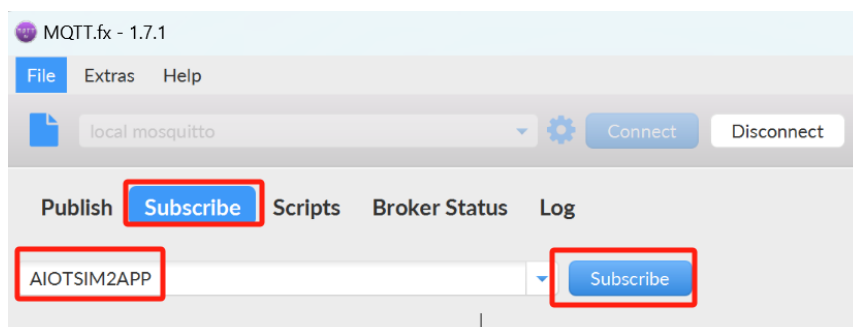
点击添加连接，Broker Address 填写代理服务器 IP 即本机 IP，默认 127.0.0.1，Port 1883，随机生成一个 ClientID，点击 OK。



点击 Connect 连接。连接成功后右侧会变成绿灯。

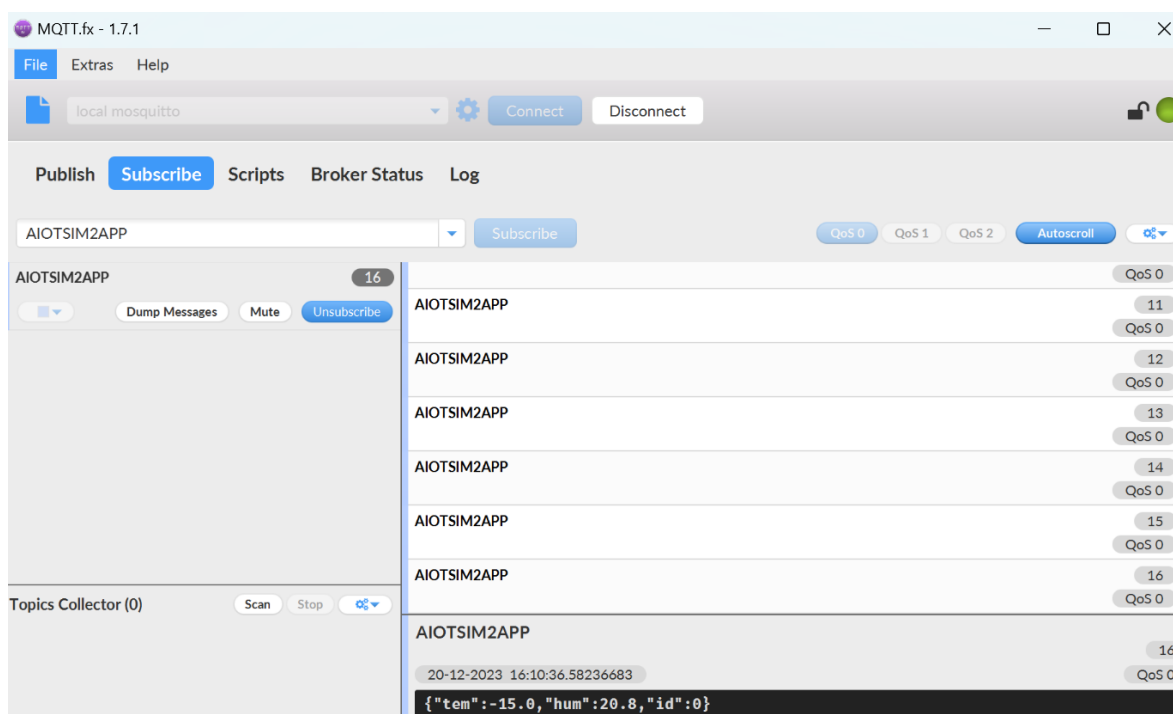


选择 Subscribe，订阅 Topic 输入 AIOTSIM2APP，点击 Subscribe



4.3 实验结果

虚拟仿真软件运行的情况下，可在 MQTT.fx 客户端软件上看到接收到的 MQTT 消息。



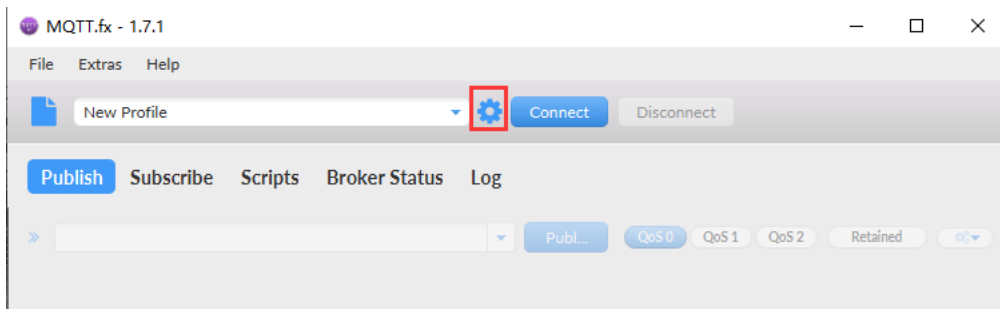
第 5 章 MQTT 消息发布实验

5.1 实验目的

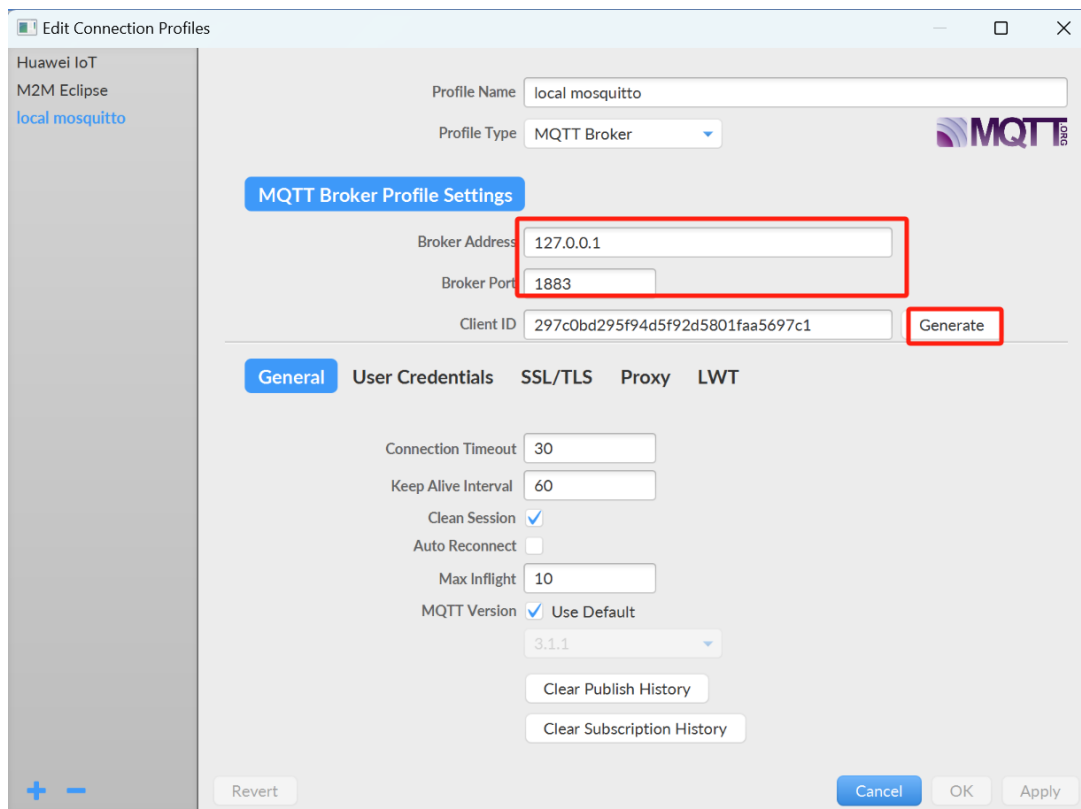
学习了解 MQTT 消息发布的流程。

5.2 实验步骤

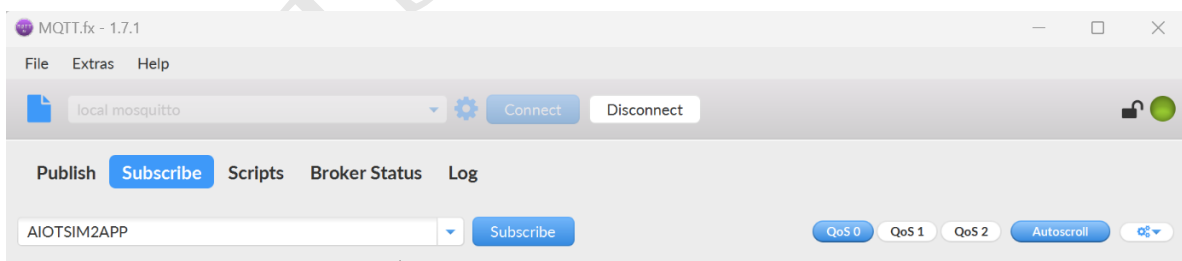
在配置好网关 MQTT 配置，并点击运行后，找到安装好的 MQTT.fx 客户端软件，双击打开，点击设置



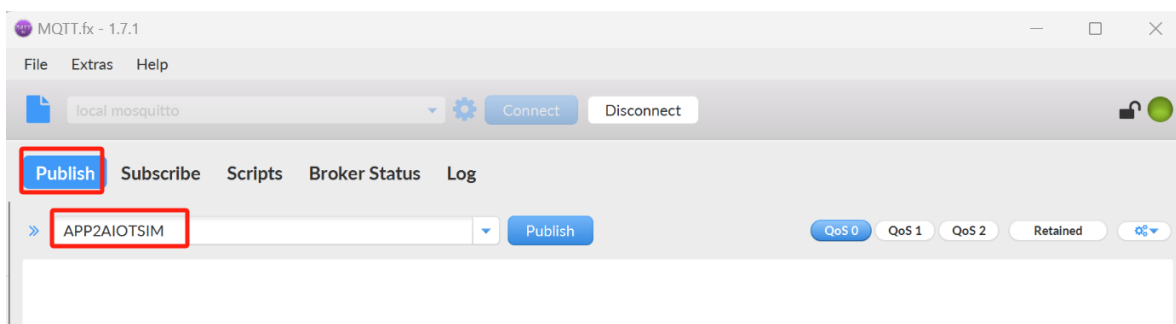
点击添加连接，Broker Address 填写代理服务器 IP 即本机 IP，Address 默认 127.0.0.1，Port 1883，生成一个随机的 ClientID，点击 OK



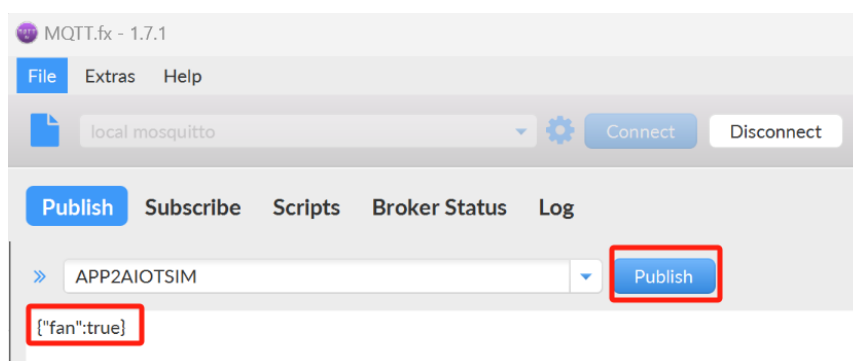
点击 Connect 连接。连接成功后右侧会变成绿灯。



选择 Publish，发布 Topic 输入 APP2AIOTSIM，即虚拟仿真系统订阅的话题。

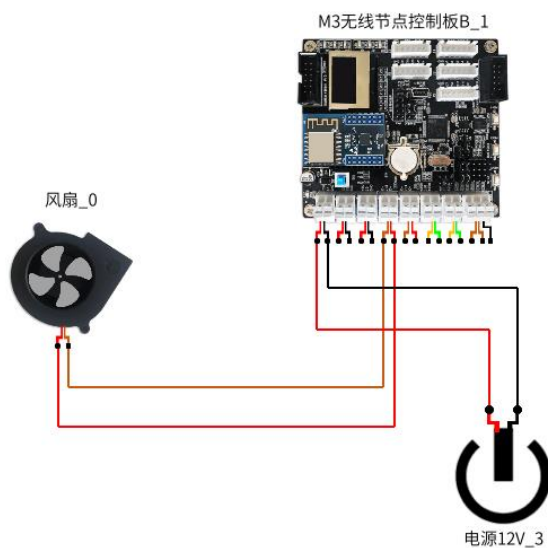


发布消息框输入{"fan":true}或{"fan":false}，点击 Publish。



5.3 实验结果

点击 Publish 发布消息后，可看到风扇转动或停止转动。



第 6 章 排风系统控制实验

6.1 实验目的

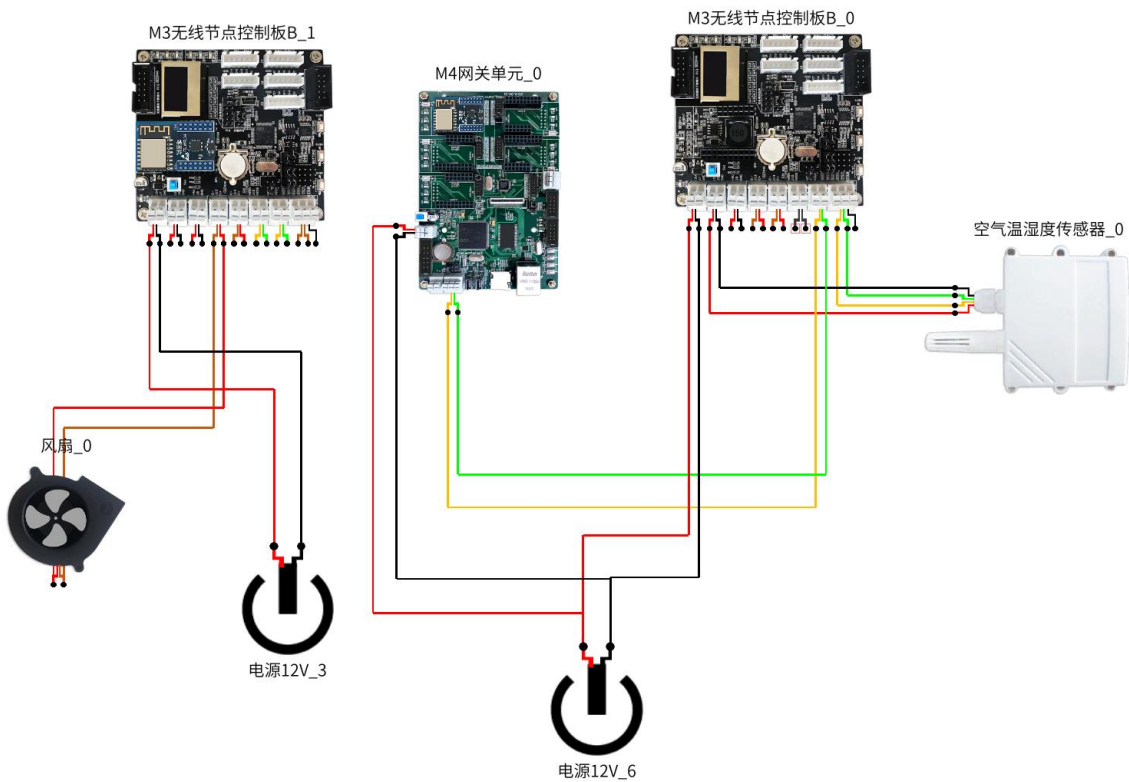
了解利用 MQTT 获取温湿度传感器信息，并根据显示的数值进行风扇的控制。

6.2 实验步骤

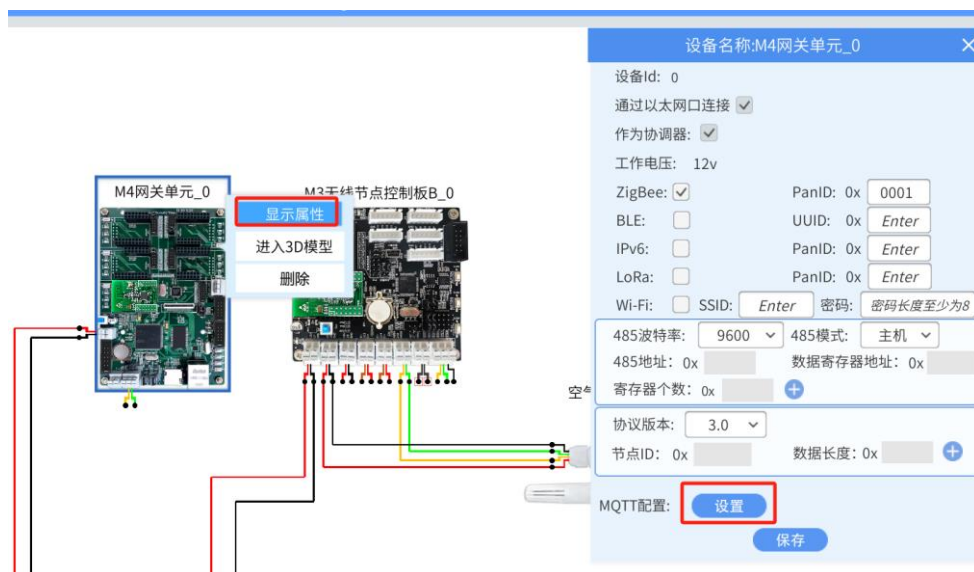
打开预设实验的智能风扇控制系统。



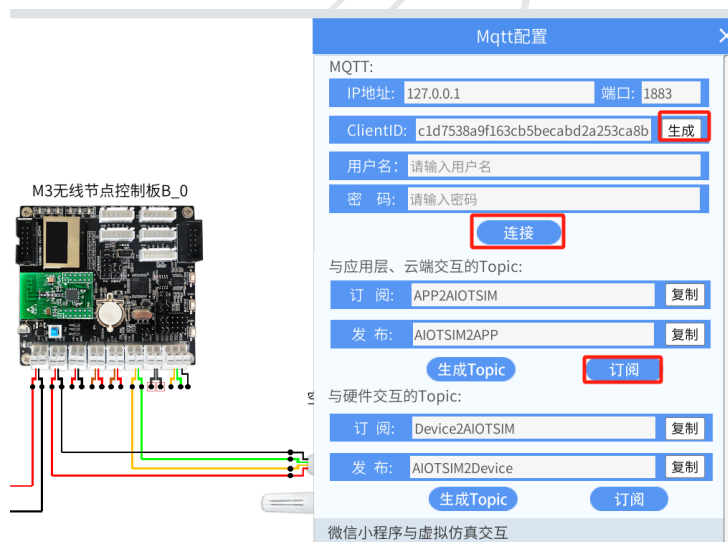
打开后显示如下界面。其中的连线和协议已经设置完毕，我们直接配置 MQTT 即可。



左键点击网关单元，选择属性。在打开的页面中点击 MQTT 配置：设置。



打开 MQTT 配置后，按照下图所示进行配置。生成一个 ClientID 作为本机的客户端 ID。该 ID 不可与其他应用设置的 ClientID（如 MQTT.fx）重复，否则会产生冲突。检查订阅和发布的话题是否正确。然后点击连接和订阅按钮。



注意，单机版软件的 IP 地址 127.0.0.1 代表本机地址，端口为 1883。也可以填入 MQTT.yyzlab.com.cn，端口号为 1883，来连接到互联网服务器中。网页版中需要输入 MQTT.yyzlab.com.cn 作为服务器地址，端口为 9002。如下图所示。连接到互联网时注意点击生成 Topic 按钮生成独一无二的 Topic，避免和其他客户端的 Topic 重复。

Mqtt配置

MQTT:

IP地址: wss:// mqtt.yyzlab.com.cn

Path: mqtt

端口: 9002 MQTT版本: 3.1.1

ClientID: 33574cfa21e85f0e7a505ccabd84575 生成

用户名: 请输入用户名

密码: 请输入密码

连接

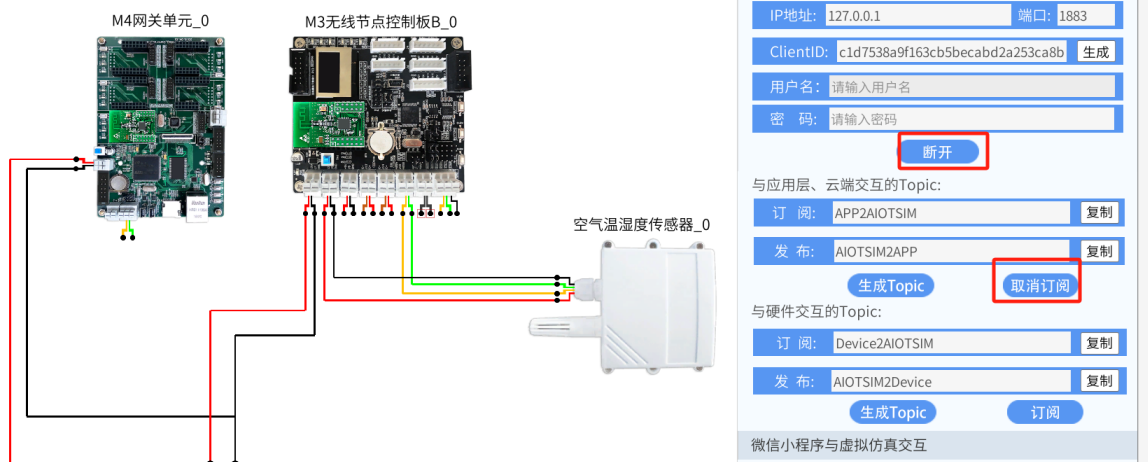
与应用层、云端交互的Topic:

订阅: 1703059472688/APP2AIOTSIM 复制

发布: 1703059472688/AIOTSIM2APP 复制

生成topic 订阅

连接和订阅成功后会显示如下按钮状态:



为了让传感器上传数据，我们需要打开数据生成选项。传感器数据就会按照曲线所示自动生成，然后发送到 MQTT 的对应的发布话题上去。

打开Zigbee网络拓扑图 数据列表 运行 验证 设置 清空 返回

数据列表

☐ 关闭数据 ☒ 打开数据

空气温湿度传感器_0数据

数据来源: ☒ 打开数据 ☒ 虚拟设备 ☐ 实际硬件

曲线名称: 温度

曲线类型: Sin曲线

数据周期: 20 修改

数据范围: -40 - 80 数据范围-40-80

曲线名称: 湿度

曲线类型: Sin曲线

数据周期: 20 修改

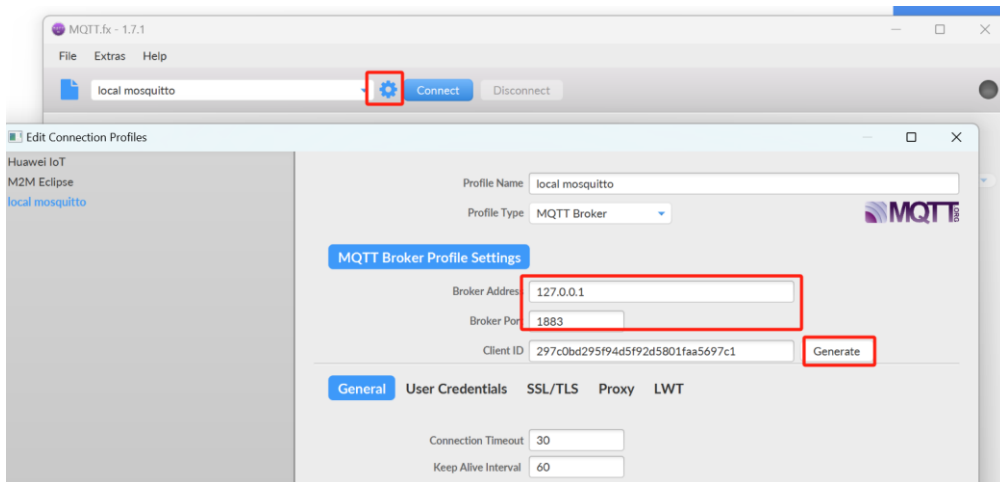
数据范围: 0 - 100 数据范围0-100

温度/°C 空气温湿度传感器_0数据模拟器 湿度/%

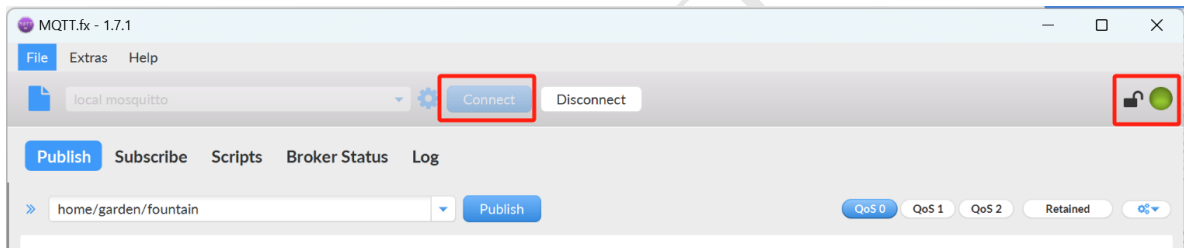
80 100

我们打开 MQTT.fx，点击设置按钮，然后使用 127.0.0.1 和 1883 作为 Broker Address 和

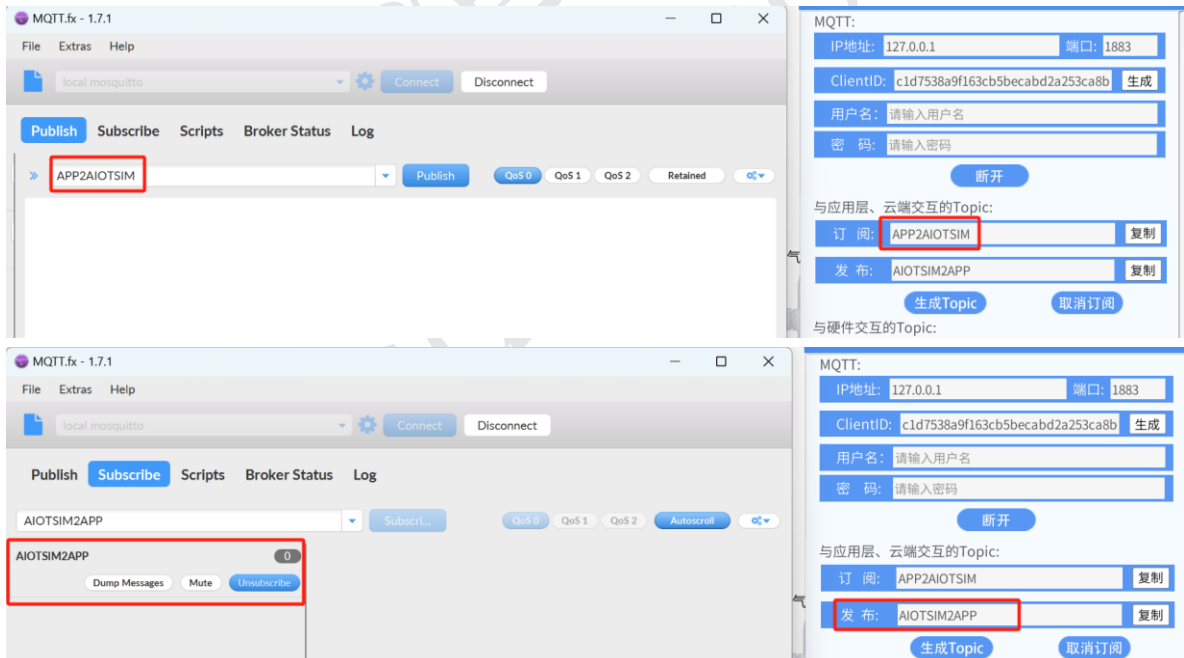
Port。注意生成一个独一无二的 ClientID，避免与其他 ID 冲突。然后点击 OK 保存。



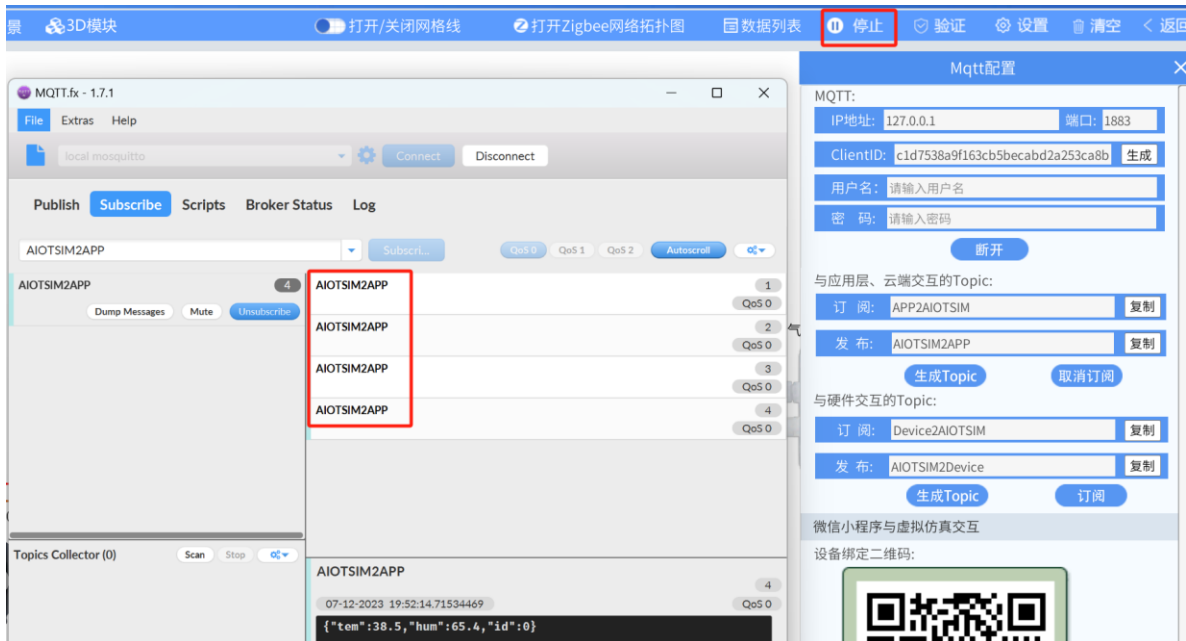
然后我们点击 Connect 按钮，当右侧的指示灯变绿时，表示我们连接到了对应的 Broker 上。



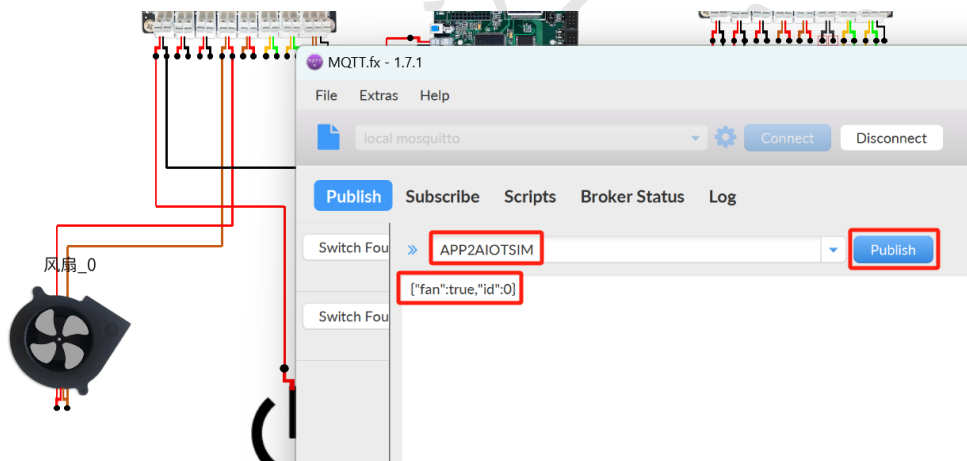
下面我们就需要设置发布和订阅的话题了。在 MQTT.fx 上发布的话题是虚拟仿真系统里面订阅的话题，在 MQTT.fx 上订阅的话题是虚拟仿真系统里面发布的话题。



我们点击虚拟仿真系统里的运行按钮，即可让数据不断产生并发送到 MQTT Broker 上，由 MQTT.fx 接收到。



我们可以依据传感器上传的值，来决定什么时候向虚拟仿真系统发布消息，让其打开风扇。我们先在 MQTT.fx 上发布话题让其打开和关闭，内容为{"fan":true,"id":0}或{"fan":false,"id":0}。id 为 0 代表其为虚拟仿真系统里面的 0 号风扇。



在实际应用中，我们需要通过编程语言来对传感器产生的数值进行判断，从而决定向执行器发送什么指令。比如我们可以使用 Python 语言，如下所示：

```
# -*- coding:utf-8 -*-
2 import paho.MQTT.client as MQTT
3 from queue import Queue
4 import JSON
5 import base64
6 import requests
7 import time
8
9 pass
10
11 class HQYJMQTTClient:
```



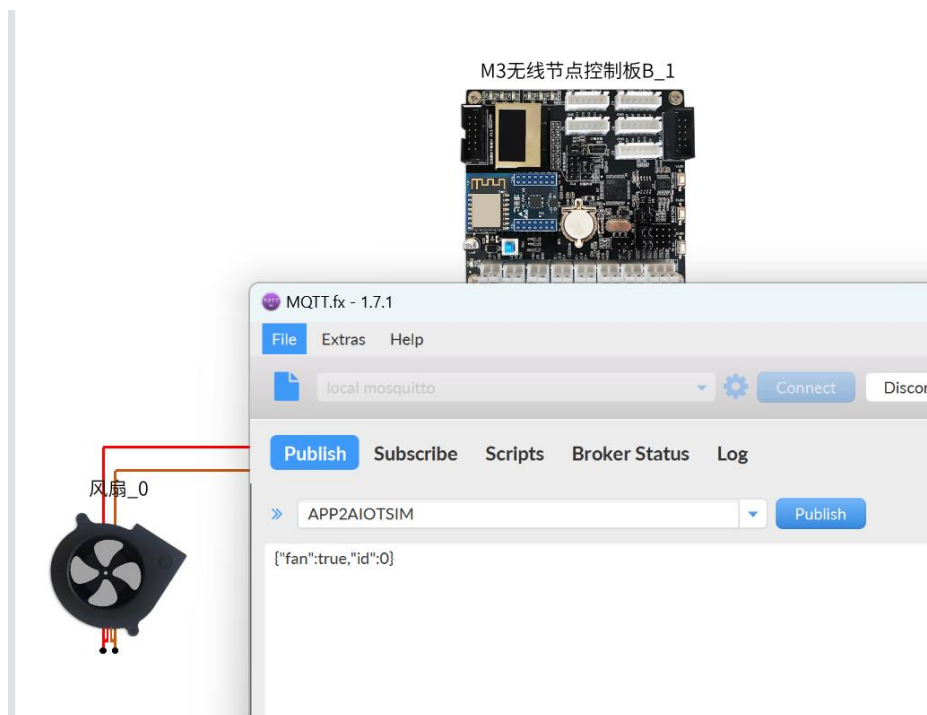
```

12     def __init__(self, Broker_ip: str, Broker_port: int):
13         self.MQTT_queue = Queue(255)
14         self.is_connected = False
15         self.client = MQTT.Client()
16         self.client.on_message = self.on_message
17         self.client.on_connect = self.on_connect
18         self.rc = 100
19         try:
20             self.client.connect(Broker_ip, Broker_port, 3)
21         except Exception as e:
22             print(e)
23
24     def on_message(self, client, userdata, message):
25         msg = JSON.loads(message.payload.decode())
26         self.MQTT_queue.put(msg)
27
28     def on_connect(self, client, userdata, flags, rc):
29         print("result code:", rc)
30         self.rc = rc
31
32 # 注意: 在 websocket 端口是 9001, 如使用 Python 代码, 端口需手动改成 1883。
33 hqyj_MQTT_clt = HQYJMqttClient('127.0.0.1', 1883)
34 hqyj_MQTT_clt.client.loop_start()
35 time.sleep(3)
36 if hqyj_MQTT_clt.rc == 0:
37     print('MQTT connected')
38 hqyj_MQTT_clt.client.subscribe('AIOTSIM2APP', qos=0)
39
40 while True:
41     MQTT_data = hqyj_MQTT_clt.MQTT_queue.get()
42
43     if ('tem' in MQTT_data) and ('id' in MQTT_data) and (MQTT_data['id'] == 0):
44         if (float(MQTT_data["tem"])) > 8:
45             hqyj_MQTT_clt.client.publish('APP2AIOTSIM', payload=JSON.dumps({"fan":True, "id":0}),ensure_ascii=False))
46         if (float(MQTT_data["tem"])) < 8:
47             hqyj_MQTT_clt.client.publish('APP2AIOTSIM', payload=JSON.dumps({"fan":False, "id":0}),ensure_ascii=False))

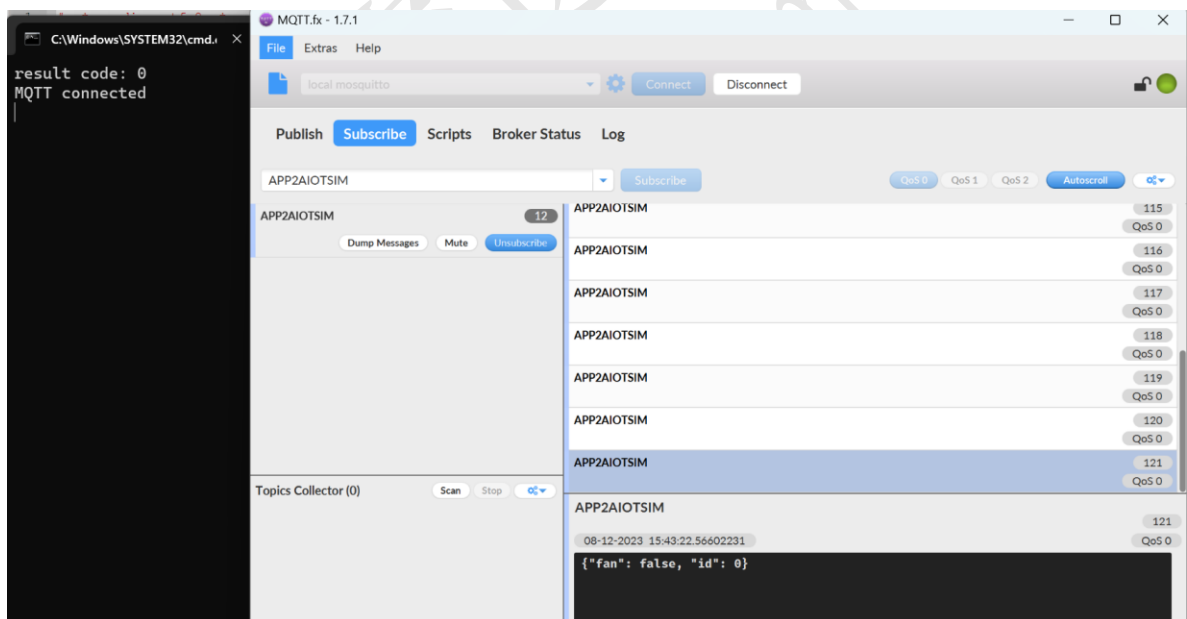
```

6.3 实验结果

当用 MQTT.fx 发送{"fan":true,"id":0}后, 风扇会转。



运行 Python 程序后，终端中显示如下字符，并且程序会依据传感器上传的数据判断是否要打开或关闭风扇。



第 7 章 智能报警系统控制实验

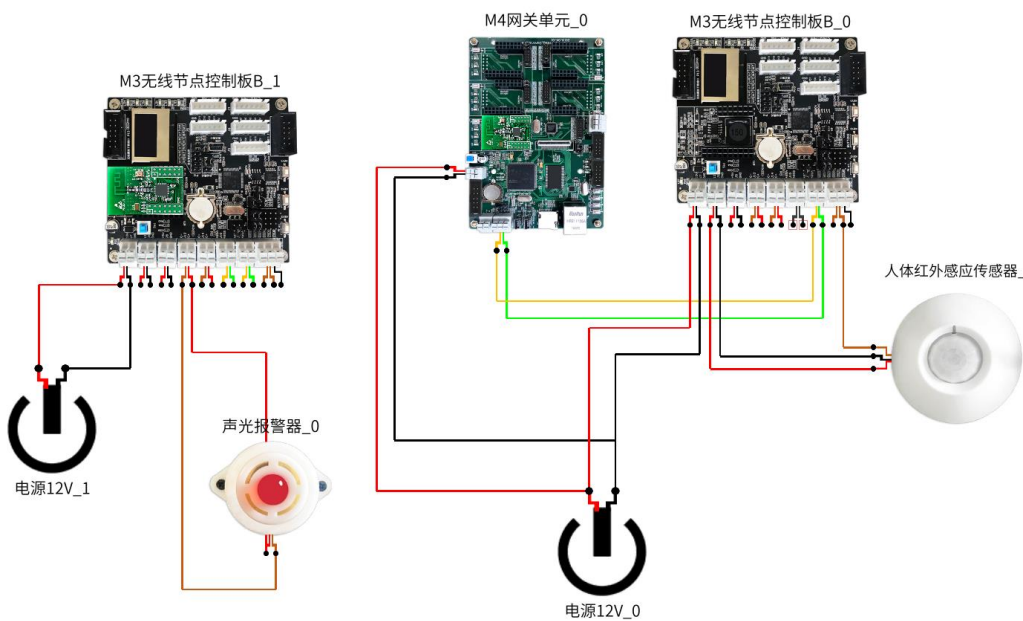
7.1 实验目的

利用 MQTT 获取人体红外传感器信息，并根据状态进行报警。

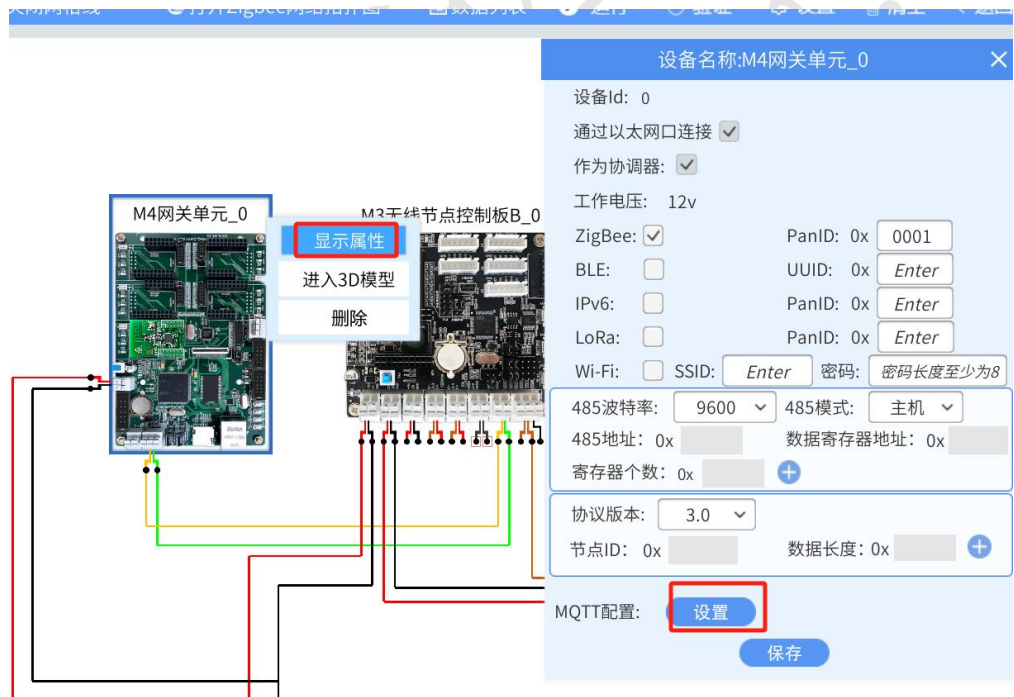
7.2 实验步骤

打开 01-资料光盘镜像\01-程序源码\物联网虚拟仿真软件实验\MQTT 实验下的 2_智能报警系

统控制实验.AIOTSIM。下面我们将依据人体红外感应传感器产生的信息进行声光报警器的控制。



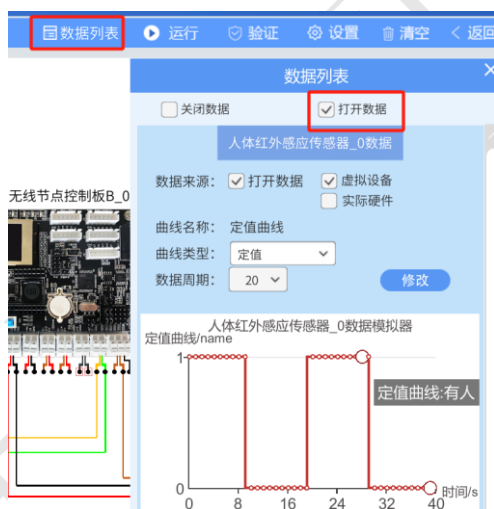
左键点击网关单元，选择属性



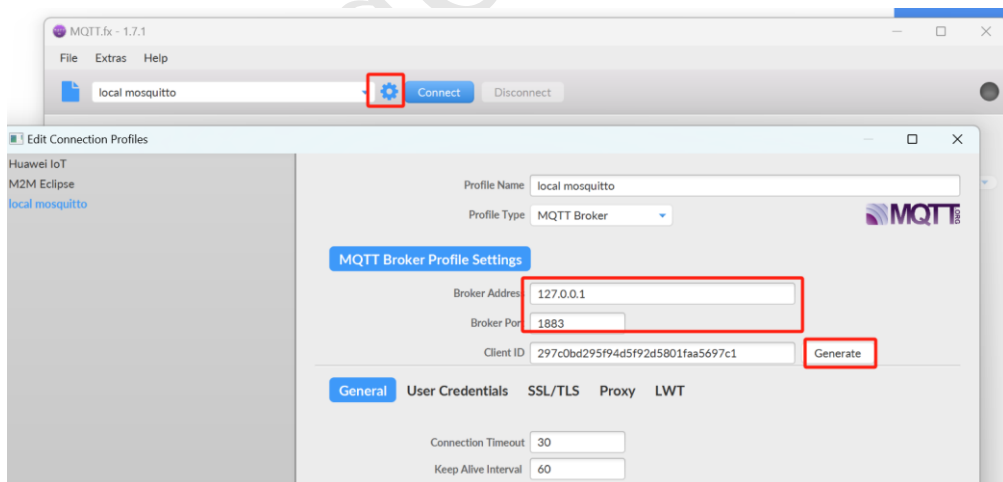
连接本地 MQTT Broker，订阅相关话题。



点击数据列表，打开数据。然后点击数据列表旁边的运行按钮。

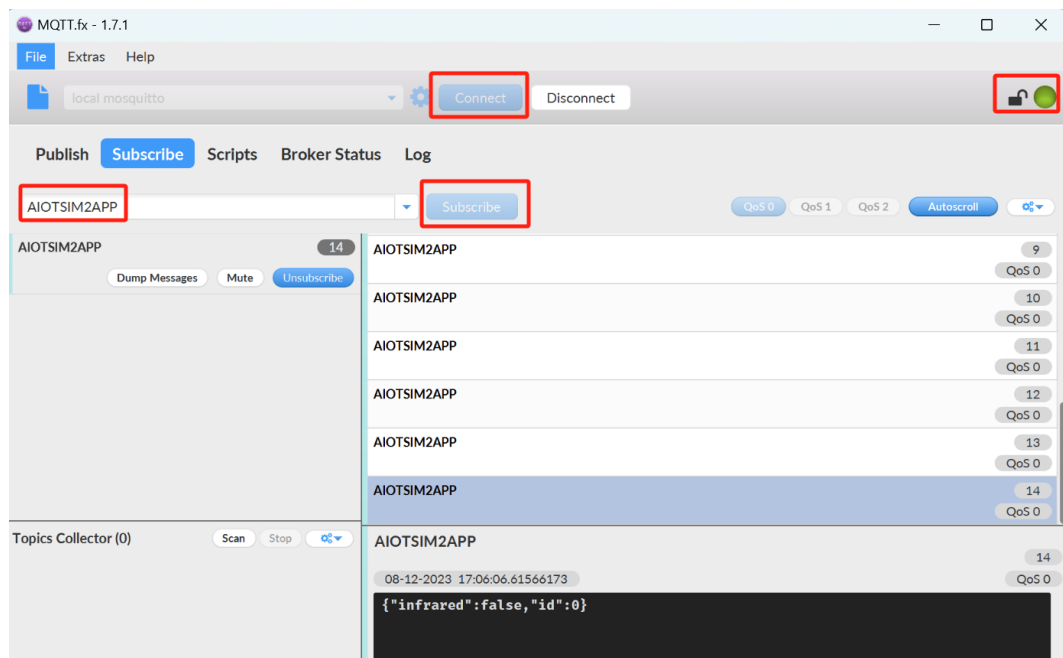


我们打开 MQTT.fx，点击设置按钮，然后使用 127.0.0.1 和 1883 作为 Broker Address 和 Port。注意生成一个独一无二的 ClientID，避免与其他 ID 冲突。然后点击 OK 保存。

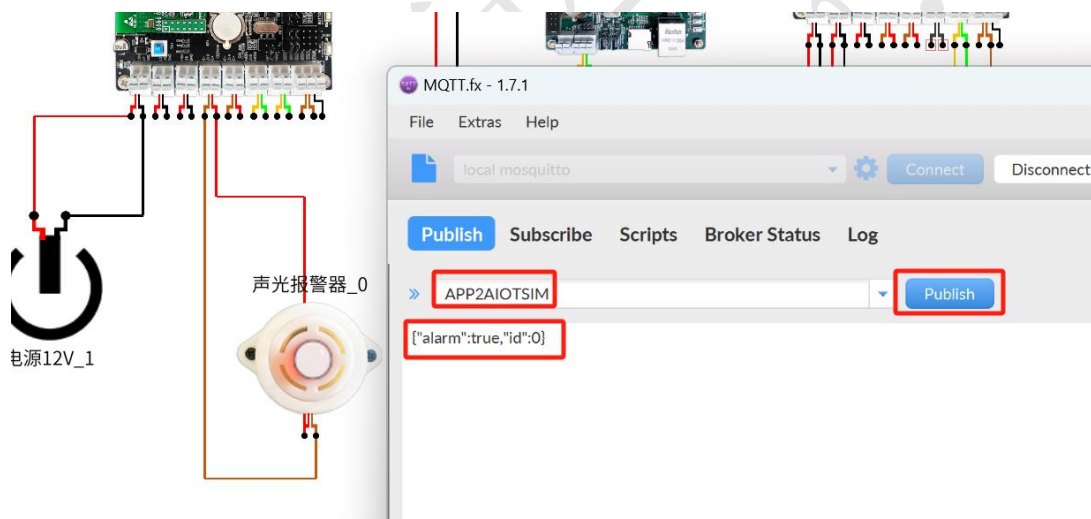


点击 Connect 按钮后，如果连接 Broker 成功，则右侧绿灯会亮。然后我们订阅一个话题。注意该订阅的话题名称要与虚拟仿真中的发布的话题名称一致。订阅成功后，即可收到虚拟仿真发

来的消息。



在 Publish 页面里面，我们向虚拟仿真订阅的话题发布消息 {"alarm":true,"id":0}，即可打开声光报警器。



在 Python 中，我们可以按照下面的代码进行自动控制。

```
# -*- coding:utf-8 -*-
2 import paho.MQTT.client as MQTT
3 from queue import Queue
4 import JSON
5 import base64
6 import requests
7
8 import time
9
10 class HQYJMQTTClient:
```

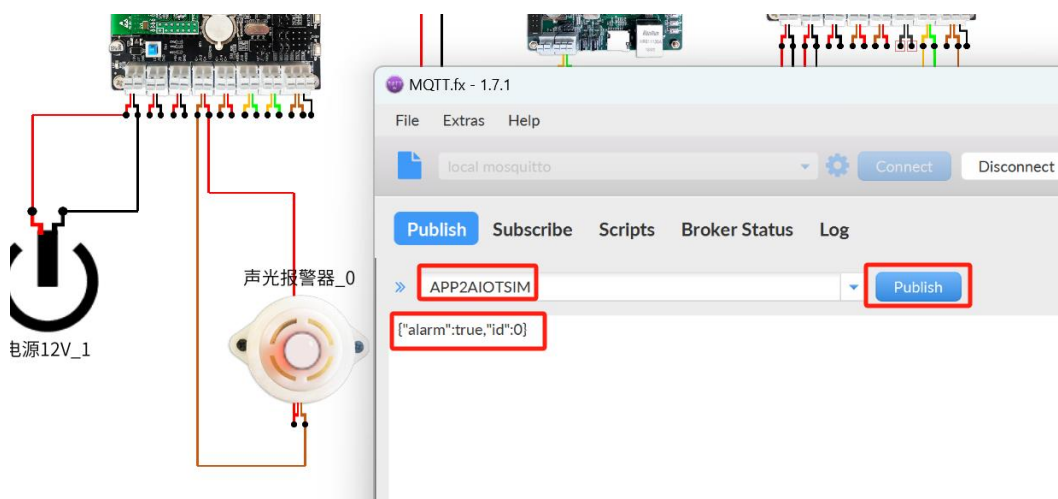
```

11 def __init__(self, Broker_ip: str, Broker_port: int):
12     self.MQTT_queue = Queue(255)
13     self.is_connected = False
14     self.client = MQTT.Client()
15     self.client.on_message = self.on_message
16     self.client.on_connect = self.on_connect
17     self.rc = 100
18     try:
19         self.client.connect(Broker_ip, Broker_port, 3)
20     except Exception as e:
21         print(e)
22
23 def on_message(self, client, userdata, message):
24     msg = JSON.loads(message.payload.decode())
25     self.MQTT_queue.put(msg)
26
27 def on_connect(self, client, userdata, flags, rc):
28     print("result code", rc)
29     self.rc = rc
30
31
32 # 注意: 在 websocket 端口是 9001, 如使用 Python 代码, 端口需手动改成 1883。
33 hqyj_MQTT_clt = HQYJMqttClient('127.0.0.1', 1883)
34 hqyj_MQTT_clt.client.loop_start()
35 time.sleep(3)
36 if hqyj_MQTT_clt.rc == 0:
37     print('MQTT connected')
38     hqyj_MQTT_clt.client.subscribe('AIOTSIM2APP', qos=0)
39
40 while True:
41     MQTT_data = hqyj_MQTT_clt.MQTT_queue.get()
42
43     if ('infrared' in MQTT_data) and ('id' in MQTT_data) and (MQTT_data['id'] == 0):
44         if (float(MQTT_data["infrared"])) > 0:
45             hqyj_MQTT_clt.client.publish('APP2AIOTSIM', payload=JSON.dumps({"alarm":True, "id":0}),ensure_ascii=False))
46         if (float(MQTT_data["infrared"])) < 1:
47             hqyj_MQTT_clt.client.publish('APP2AIOTSIM', payload=JSON.dumps({"alarm":False, "id":0}),ensure_ascii=False))

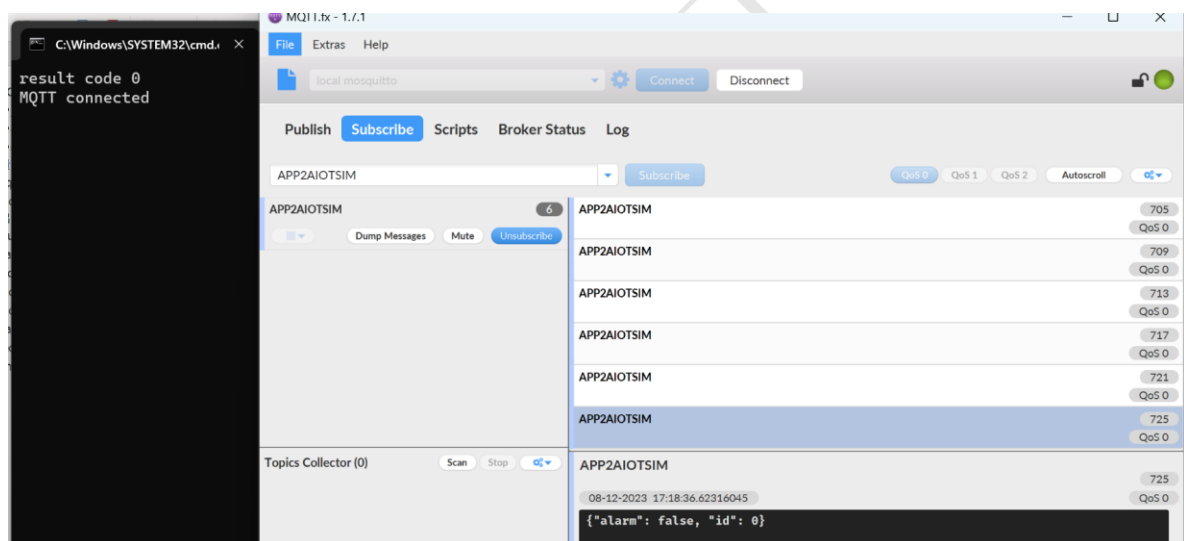
```

7.3 实验结果

当用 MQTT.fx 发送{"alarm":true,"id":0}后, 报警器会报警。



运行 Python 程序后，终端中显示如下字符，并且程序会依据传感器上传的数据判断是否要打开或关闭声光报警器。



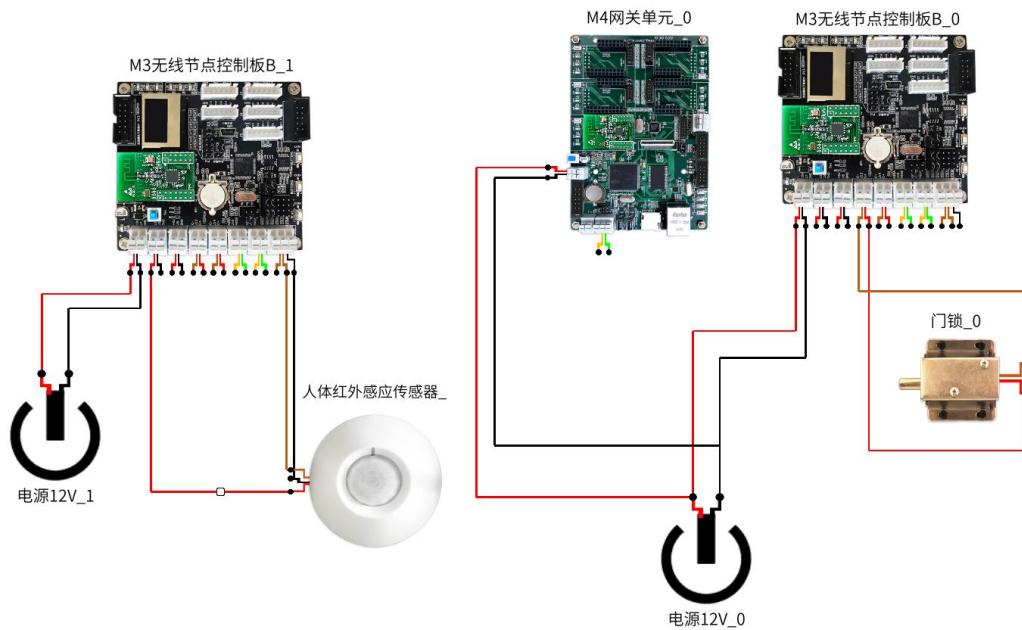
第 8 章 微型电控锁系统控制实验

8.1 实验目的

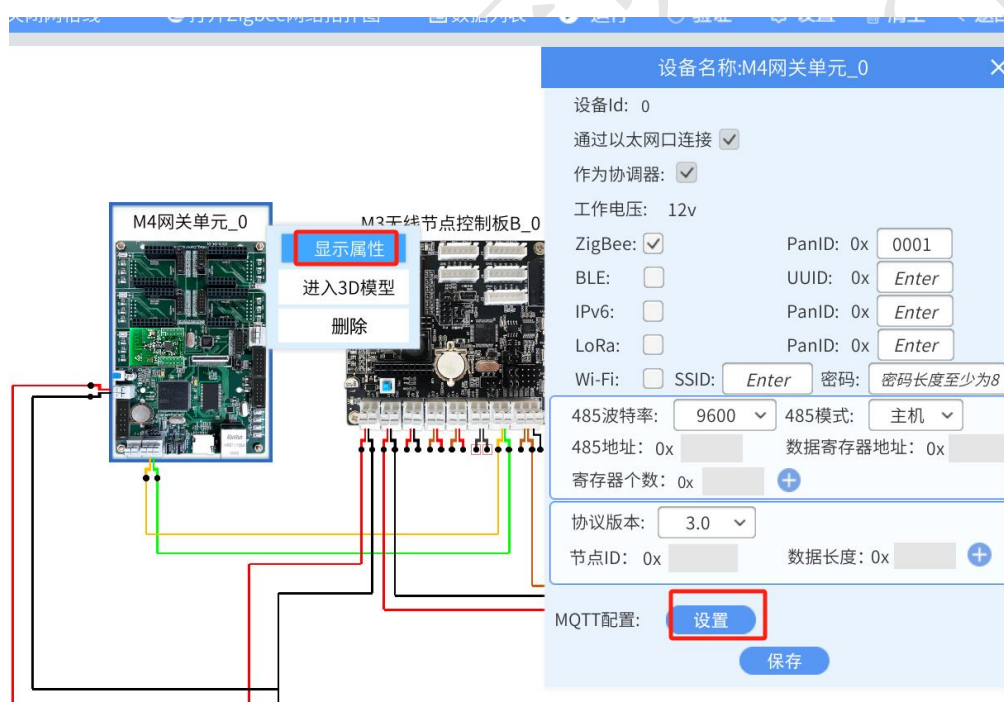
利用 MQTT 获取人体红外传感器信息，并根据状态进行门锁的控制。

8.2 实验步骤

打开 01-资料光盘镜像\01-程序源码\物联网虚拟仿真软件实验\MQTT 实验下的 3_微型电控锁系统控制实验.AIOTSIM。下面我们将依据人体红外感应传感器产生的信息进行门锁的控制。



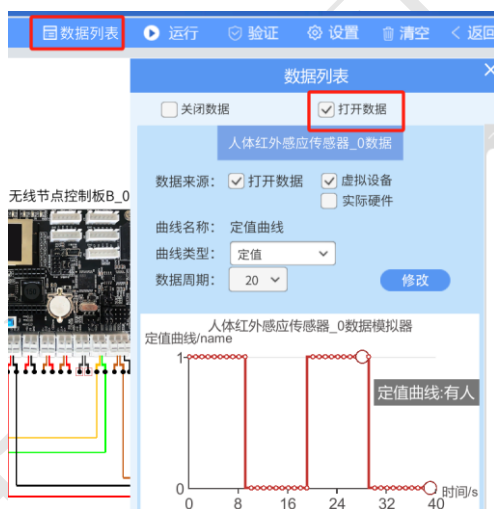
左键点击网关单元，选择属性



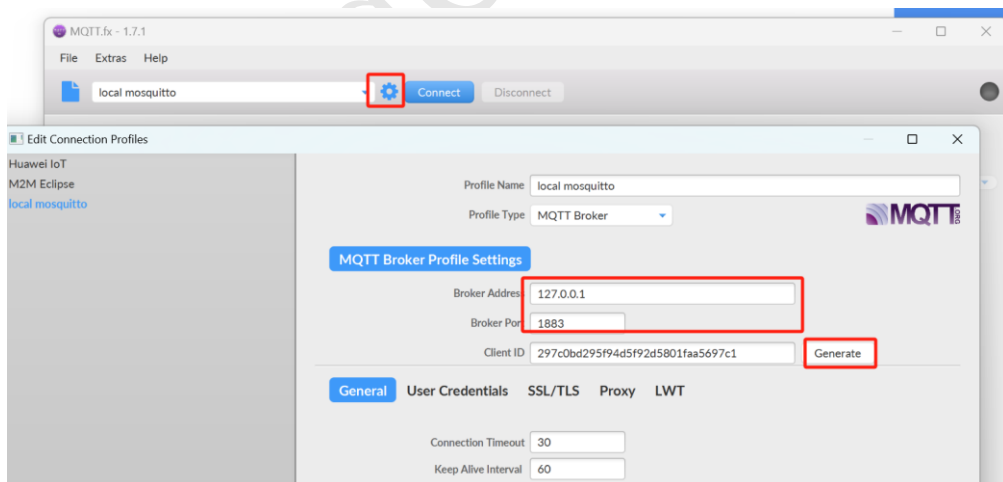
连接本地 MQTT Broker，订阅相关话题。



点击数据列表，打开数据。然后点击数据列表旁边的运行按钮。

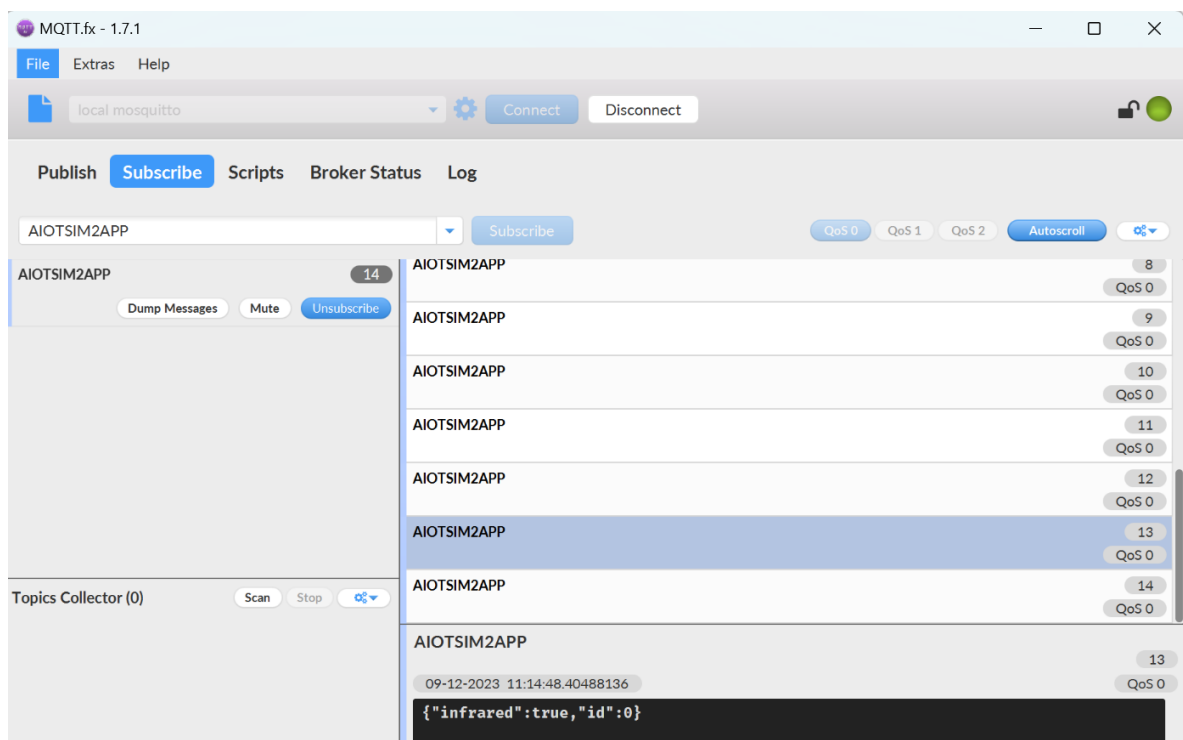


我们打开 MQTT.fx，点击设置按钮，然后使用 127.0.0.1 和 1883 作为 Broker Address 和 Port。注意生成一个独一无二的 ClientID，避免与其他 ID 冲突。然后点击 OK 保存。

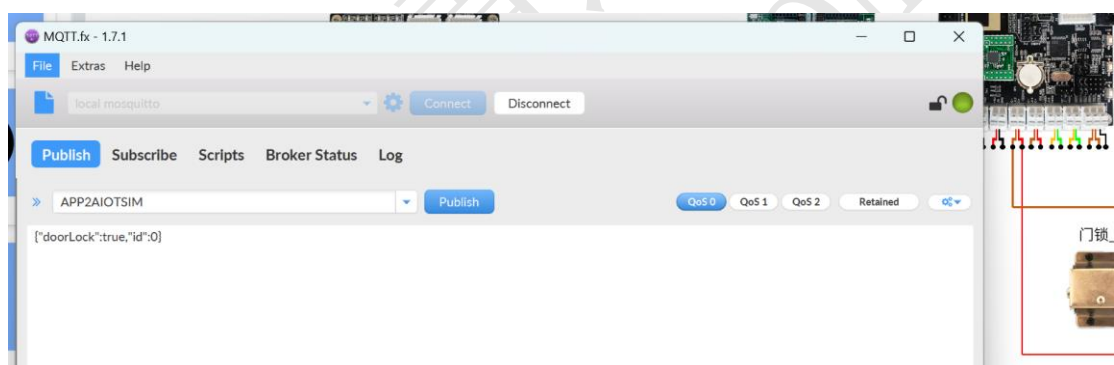


点击 Connect 按钮后，如果连接 Broker 成功，则右侧绿灯会亮。然后我们订阅一个话题。注意该订阅的话题名称要与虚拟仿真中的发布的话题名称一致。订阅成功后，即可收到虚拟仿真发

来的消息。



在 Publish 页面里面，我们向虚拟仿真订阅的话题发布消息{"doorLock":true,"id":0}，即可打开门锁。



在 Python 中，我们可以按照下面的代码进行自动控制。

```
# -*- coding:utf-8 -*-
2 import paho.MQTT.client as MQTT
3 from queue import Queue
4 import JSON
5 import base64
6 import requests
7
8 import time
9
10 class HQYJMQTTClient:
11     def __init__(self, broker_ip: str, broker_port: int):
12         self.MQTT_queue = Queue(255)
```

```

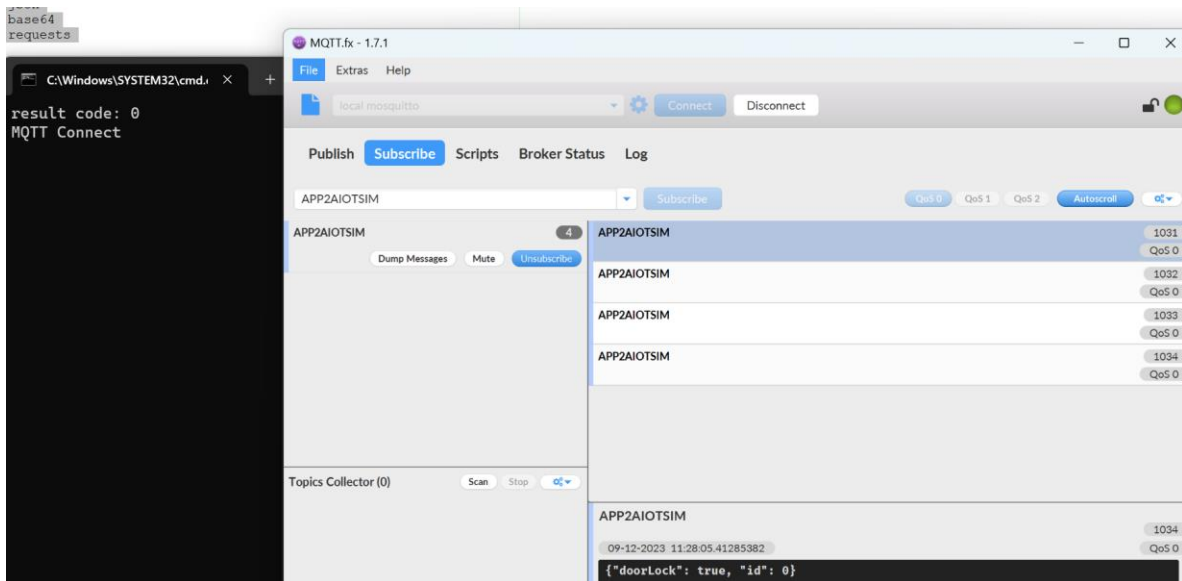
13     self.is_connected = False
14     self.client = MQTT.Client()
15     self.client.on_message = self.on_message
16     self.client.on_connect = self.on_connect
17     self.rc = 100
18     try:
19         self.client.connect(broker_ip, broker_port, 3)
20     except Exception as e:
21         print(e)
22
23     def on_message(self, client, userdata, message):
24         msg = JSON.loads(message.payload.decode())
25         self.MQTT_queue.put(msg)
26
27     def on_connect(self, client, userdata, flags, rc):
28         print("result code:", rc)
29         self.rc = rc
30
31
32 # 注意: 在 websocket 端口是 9001, 如使用 Python 代码, 端口需手动改成 1883。
33 hqyj_MQTT_clt = HQYJMQttClient('127.0.0.1', 1883)
34 hqyj_MQTT_clt.client.loop_start()
35 time.sleep(3)
36 if hqyj_MQTT_clt.rc == 0:
37     print('MQTT Connect')
38     hqyj_MQTT_clt.client.subscribe('AIOTSIM2APP', qos=0)
39
40     while True:
41         MQTT_data = hqyj_MQTT_clt.MQTT_queue.get()
42
43         if ('infrared' in MQTT_data) and ('id' in MQTT_data) and (MQTT_data['id'] == 0):
44             if (MQTT_data["infrared"]) > 0:
45                 hqyj_MQTT_clt.client.publish('APP2AIOTSIM', payload=JSON.dumps({"doorLock":
46 True, "id":0}),ensure_ascii=False))
47             if (MQTT_data["infrared"]) < 1:
48                 hqyj_MQTT_clt.client.publish('APP2AIOTSIM', payload=JSON.dumps({"doorLock":
False, "id":0}),ensure_ascii=False))

```

8.3 实验结果

当用 MQTT.fx 发送{"doorLock":true,"id":0}后, 门锁会打开。

运行 Python 程序后, 终端中显示如下字符, 并且程序会依据传感器上传的数据判断是否要打开或关闭门锁。



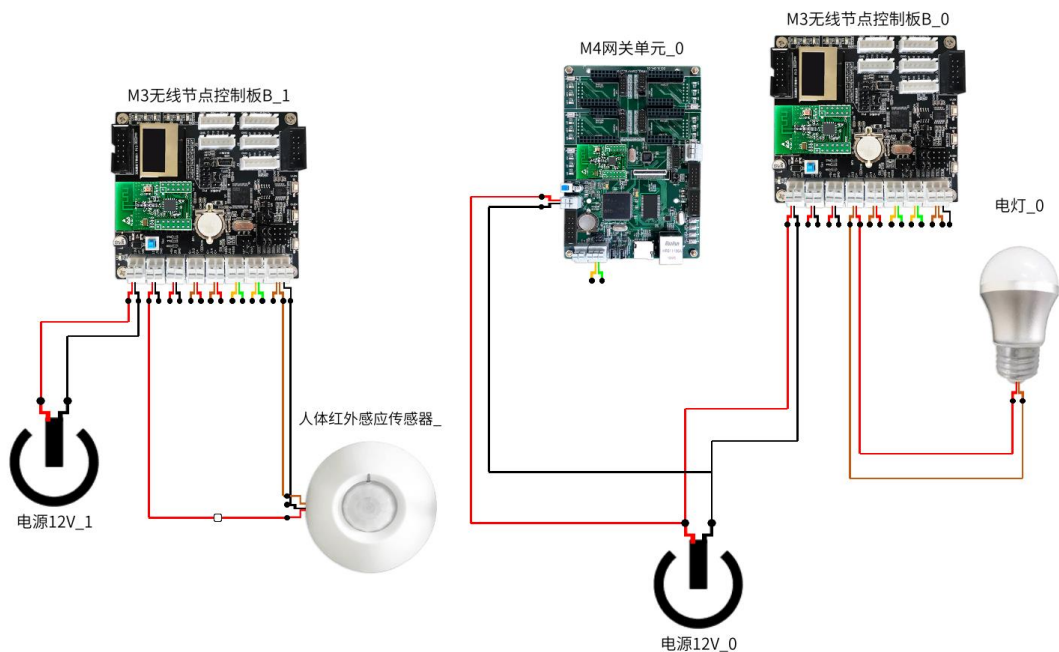
第 9 章 电灯系统控制实验

9.1 实验目的

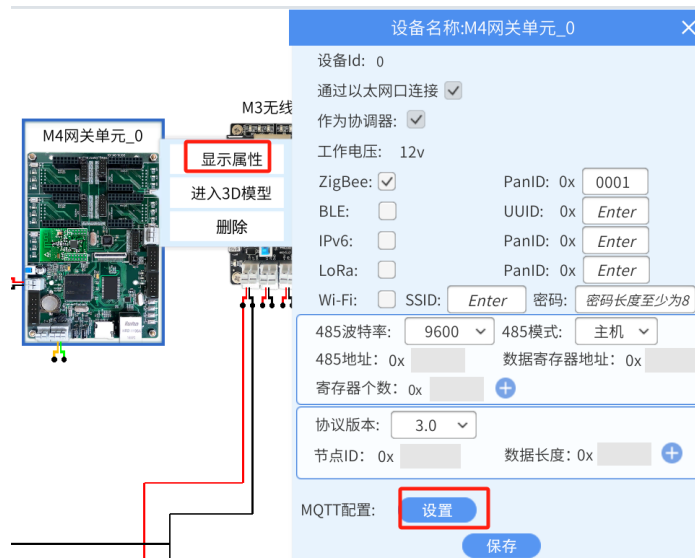
利用 MQTT 获取人体红外传感器信息，并根据状态进行电灯的控制。

9.2 实验步骤

打开 01-资料光盘镜像\01-程序源码\物联网虚拟仿真软件实验\MQTT 实验下的 4_电灯系统控制实验.AIOTSIM。下面我们将依据人体红外感应传感器产生的信息进行电灯的控制。



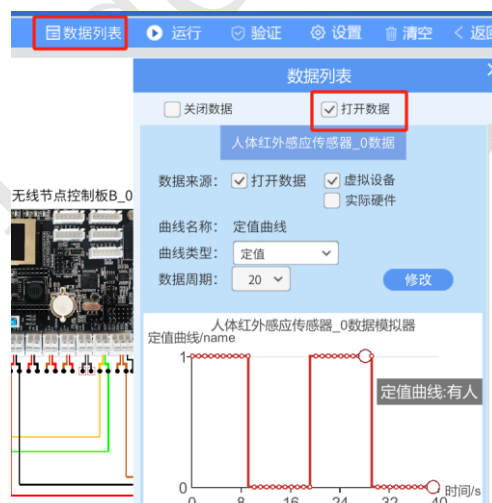
左键点击网关单元，选择属性



连接本地 MQTT Broker，订阅相关话题。

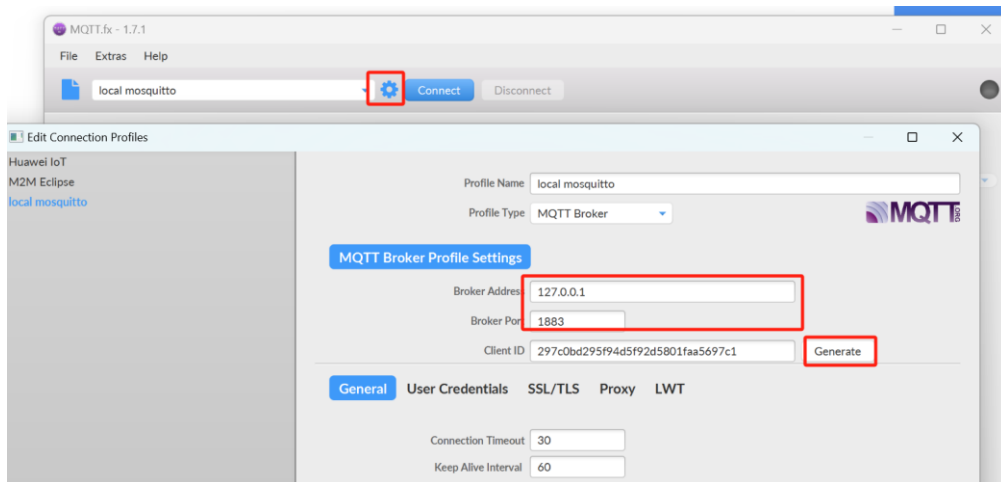


点击数据列表，打开数据。然后点击数据列表旁边的运行按钮。

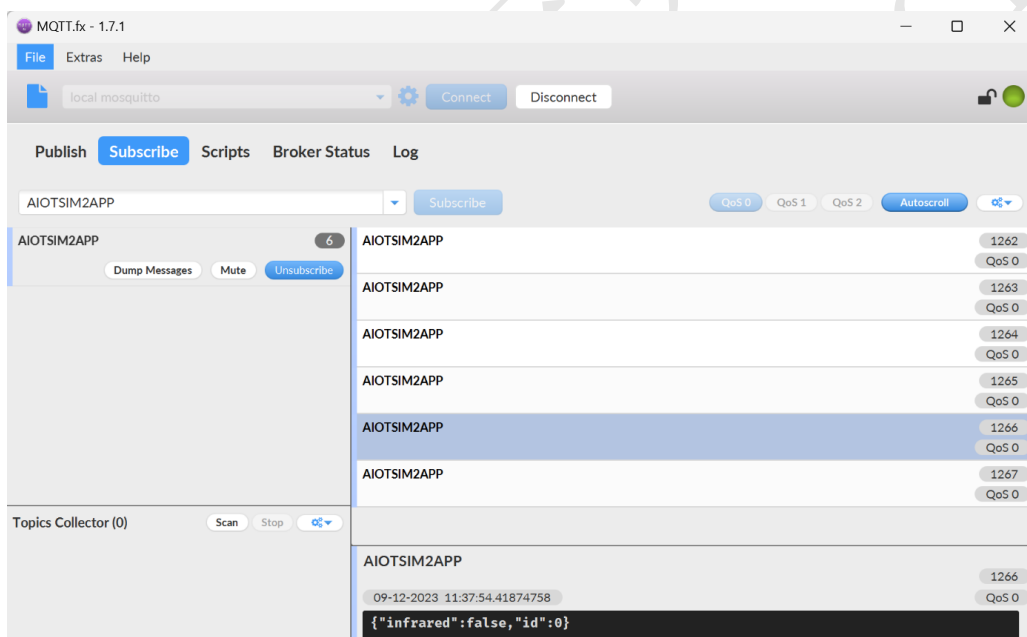


我们打开 MQTT.fx，点击设置按钮，然后使用 127.0.0.1 和 1883 作为 Broker Address 和

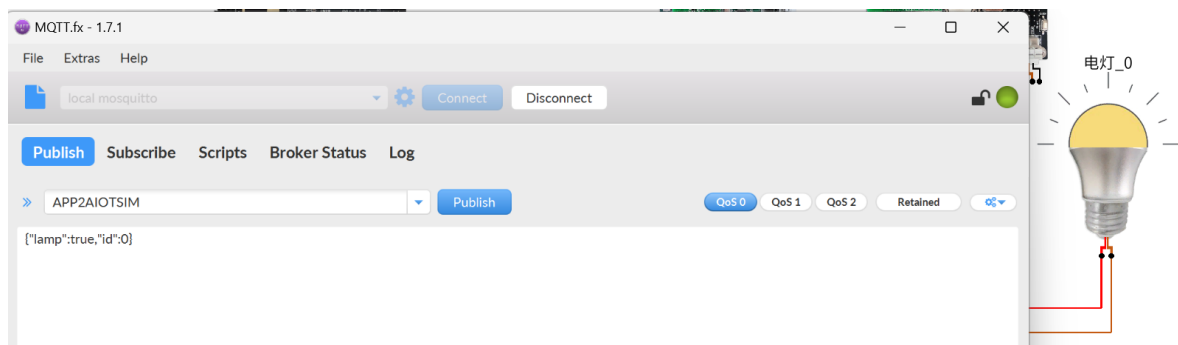
Port。注意生成一个独一无二的 ClientID，避免与其他 ID 冲突。然后点击 OK 保存。



点击 **Connect** 按钮后，如果连接 **Broker** 成功，则右侧绿灯会亮。然后我们订阅一个话题。注意订阅的话题名称要与虚拟仿真中的发布的话题名称一致。订阅成功后，即可收到虚拟仿真发来的消息。



在 **Publish** 页面里面，我们向虚拟仿真订阅的话题发布消息 `{"lamp":true,"id":0}`，即可打开电灯。



在 Python 中，我们可以按照下面的代码进行自动控制。

```
# -*- coding:utf-8 -*-
2 import paho.MQTT.client as MQTT
3 from queue import Queue
4 import JSON
5 import base64
6 import requests
7
8 import time
9
10 class HQYJMQTTClient:
11     def __init__(self, broker_ip: str, broker_port: int):
12         self.MQTT_queue = Queue(255)
13         self.is_connected = False
14         self.client = MQTT.Client()
15         self.client.on_message = self.on_message
16         self.client.on_connect = self.on_connect
17         self.rc = 100
18         try:
19             self.client.connect(broker_ip, broker_port, 3)
20         except Exception as e:
21             print(e)
22
23     def on_message(self, client, userdata, message):
24         msg = JSON.loads(message.payload.decode())
25         self.MQTT_queue.put(msg)
26
27     def on_connect(self, client, userdata, flags, rc):
28         print("result code:", rc)
29         self.rc = rc
30
31
32 # 注意：在 websocket 端口是 9001，如使用 Python 代码，端口需手动改成 1883。
33 hqyj_MQTT_clt = HQYJMQTTClient('127.0.0.1', 1883)
34 hqyj_MQTT_clt.client.loop_start()
35 time.sleep(3)
36 if hqyj_MQTT_clt.rc == 0:
37     print('MQTT Connect')
38     hqyj_MQTT_clt.client.subscribe('AIOTSIM2APP', qos=0)
39
40     while True:
41         MQTT_data = hqyj_MQTT_clt.MQTT_queue.get()
42
43         if ('infrared' in MQTT_data) and ('id' in MQTT_data) and (MQTT_data['id'] == 0):
```

```

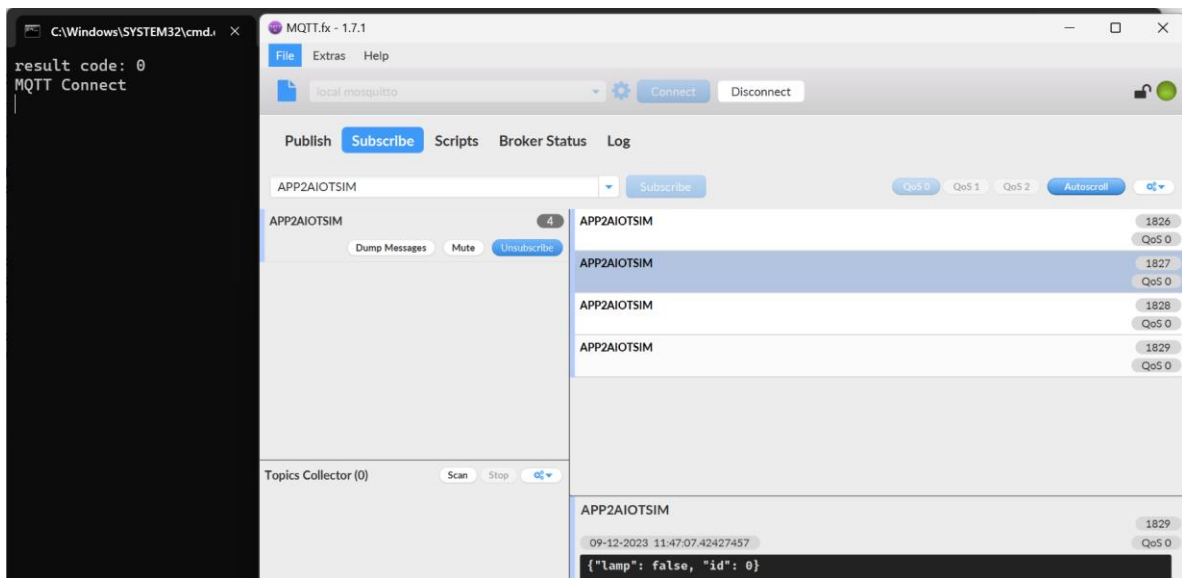
44     if (MQTT_data["infrared"]) > 0:
45         hqyj_MQTT_clt.client.publish('APP2AIOTSIM', payload=JSON.dumps({"lamp":True
46 , "id":0}),ensure_ascii=False))
47     if (MQTT_data["infrared"]) < 1:
48         hqyj_MQTT_clt.client.publish('APP2AIOTSIM', payload=JSON.dumps({"lamp":False, "id":0}),ensure_ascii=False))

```

9.3 实验结果

当用 MQTT.fx 发送{"lamp":true,"id":0}后，电灯会打开。

运行 Python 程序后，终端中显示如下字符，并且程序会依据传感器上传的数据判断是否要打开或关闭电灯。



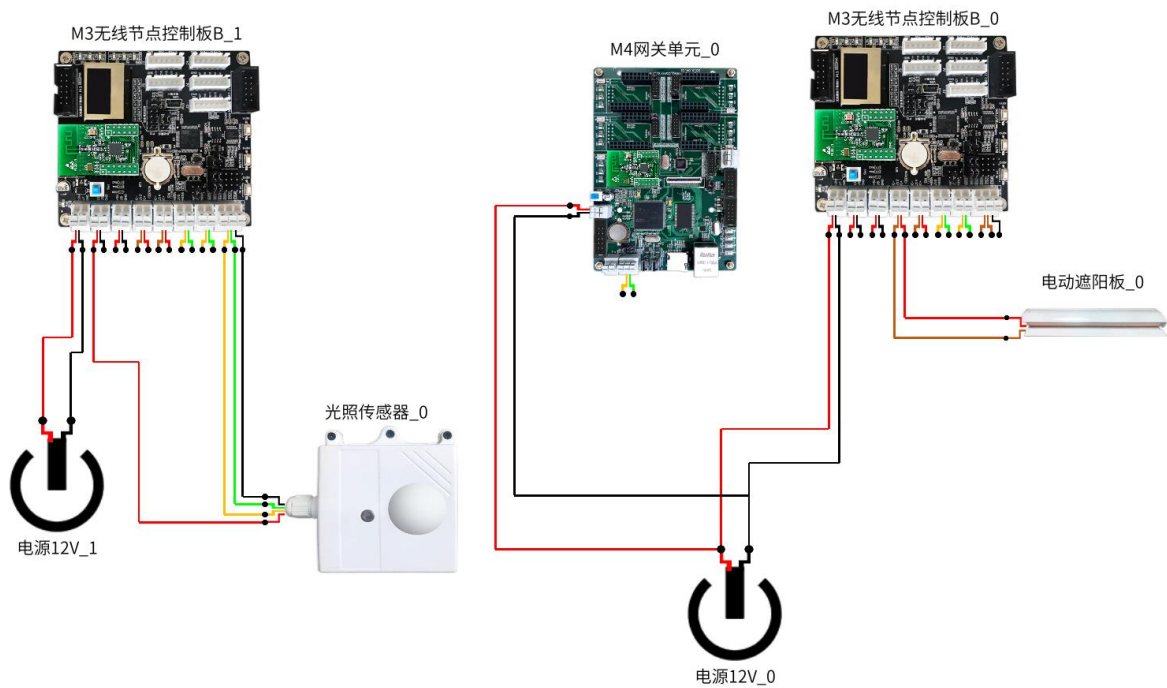
第 10 章 电动窗帘控制实验

10.1 实验目的

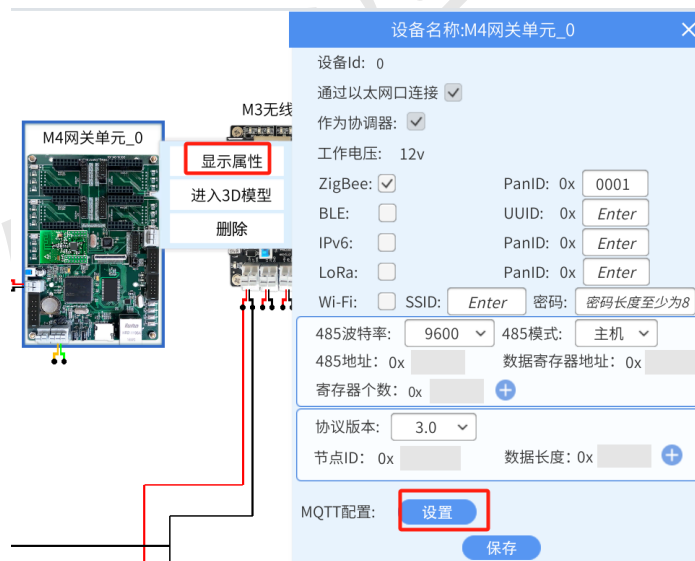
利用 MQTT 获取光照传感器信息，并根据状态进行电动窗帘的控制。当户外的光照传感器识别到光线强度高时，打开窗帘；强度低时，拉上窗帘。

10.2 实验步骤

打开 01-资料光盘镜像\01-程序源码\物联网虚拟仿真软件实验\MQTT 实验下的 5_电动窗帘控制实验.AIOTSIM。下面我们将依据光照传感器产生的信息进行电动窗帘的控制。



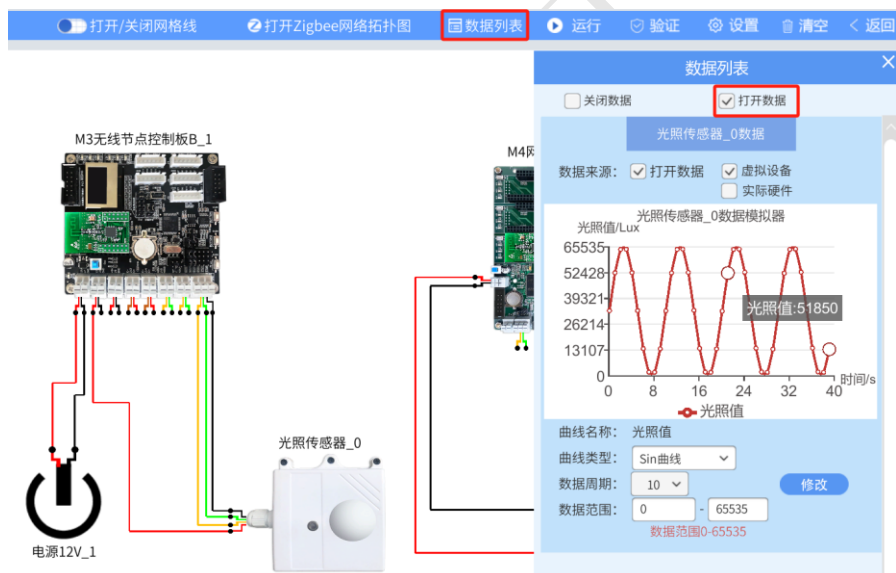
左键点击网关单元，选择属性



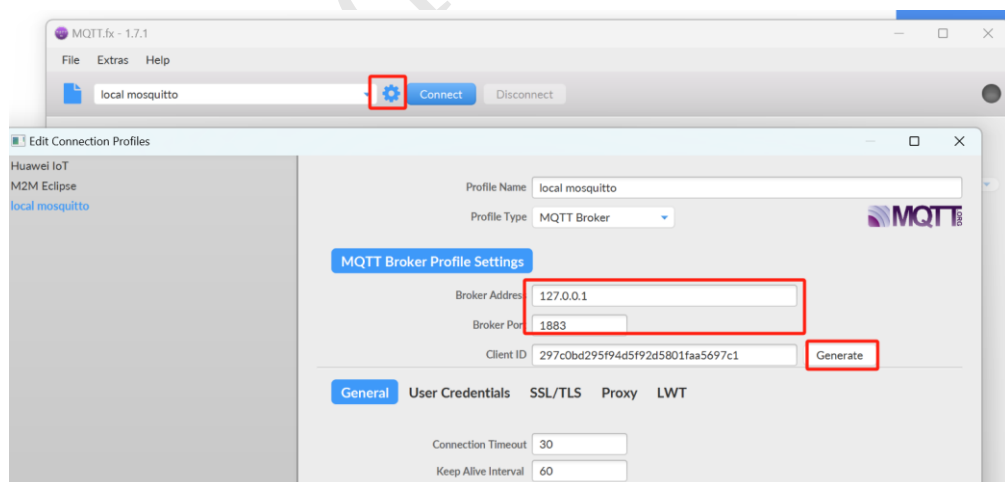
连接本地 MQTT Broker，订阅相关话题。



点击数据列表，打开数据。然后点击数据列表旁边的运行按钮。

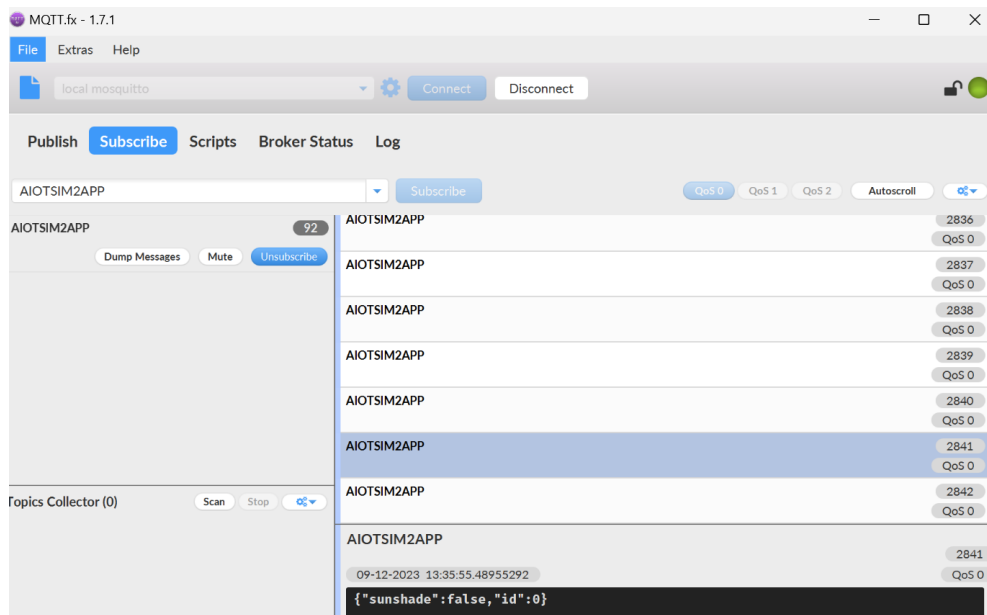


我们打开 MQTT.fx，点击设置按钮，然后使用 127.0.0.1 和 1883 作为 Broker Address 和 Port。注意生成一个独一无二的 ClientID，避免与其他 ID 冲突。然后点击 OK 保存。



点击 Connect 按钮后，如果连接 Broker 成功，则右侧绿灯会亮。然后我们订阅一个话题。注

意该订阅的话题名称要与虚拟仿真中的发布的话题名称一致。订阅成功后，即可收到虚拟仿真发来的消息。



在 Publish 页面里面，我们向虚拟仿真订阅的话题发布消息{"sunshade":true,"id":0}，即可打开电动遮阳板。

在 Python 中，我们可以按照下面的代码进行自动控制。

```
# -*- coding:utf-8 -*-
2 import paho.MQTT.client as MQTT
3 from queue import Queue
4 import JSON
5 import base64
6 import requests
7
8 import time
9
10 class HQYJMQTTClient:
11     def __init__(self, broker_ip: str, broker_port: int):
12         self.MQTT_queue = Queue(255)
13         self.is_connected = False
14         self.client = MQTT.Client()
15         self.client.on_message = self.on_message
16         self.client.on_connect = self.on_connect
17         self.rc = 100
18         try:
19             self.client.connect(broker_ip, broker_port, 3)
20         except Exception as e:
21             print(e)
22
23     def on_message(self, client, userdata, message):
```

```

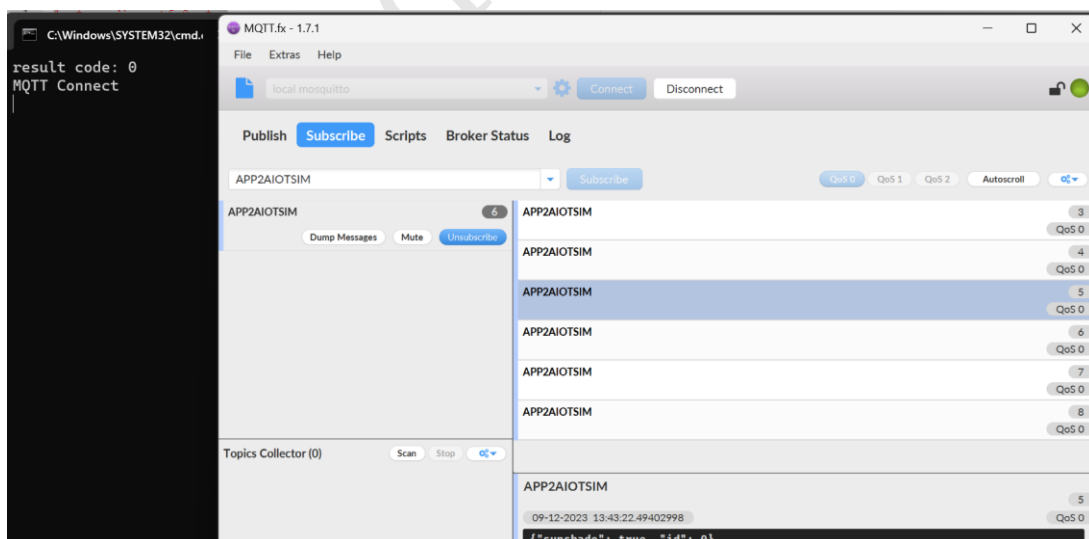
24     msg = JSON.loads(message.payload.decode())
25     self.MQTT_queue.put(msg)
26
27     def on_connect(self, client, userdata, flags, rc):
28         print("result code:", rc)
29         self.rc = rc
30
31 # 注意: 在 websocket 端口是 9001, 如使用 Python 代码, 端口需手动改成 1883。
32 hqyj_MQTT_clt = HQYJMqttClient('127.0.0.1', 1883)
33 hqyj_MQTT_clt.client.loop_start()
34 time.sleep(3)
35 if hqyj_MQTT_clt.rc == 0:
36     print('MQTT Connect')
37     hqyj_MQTT_clt.client.subscribe('AIOTSIM2APP', qos=0)
38
39     while True:
40         MQTT_data = hqyj_MQTT_clt.MQTT_queue.get()
41
42         if ('light' in MQTT_data) and ('id' in MQTT_data) and (MQTT_data['id'] == 0):
43             if (float(MQTT_data["light"])) > 30000:
44                 hqyj_MQTT_clt.client.publish('APP2AIOTSIM', payload=JSON.dumps({"sunshade":
45 False, "id":0}),ensure_ascii=False))
46             if (float(MQTT_data["light"])) < 30000:
47                 hqyj_MQTT_clt.client.publish('APP2AIOTSIM', payload=JSON.dumps({"sunshade":
48 True, "id":0}),ensure_ascii=False))

```

10.3 实验结果

当用 MQTT.fx 发送{"sunshade":true,"id":0}后, 电动遮阳板会打开。

运行 Python 程序后, 终端中显示如下字符, 并且程序会依据传感器上传的数据判断是否要打开或关闭电动遮阳板。



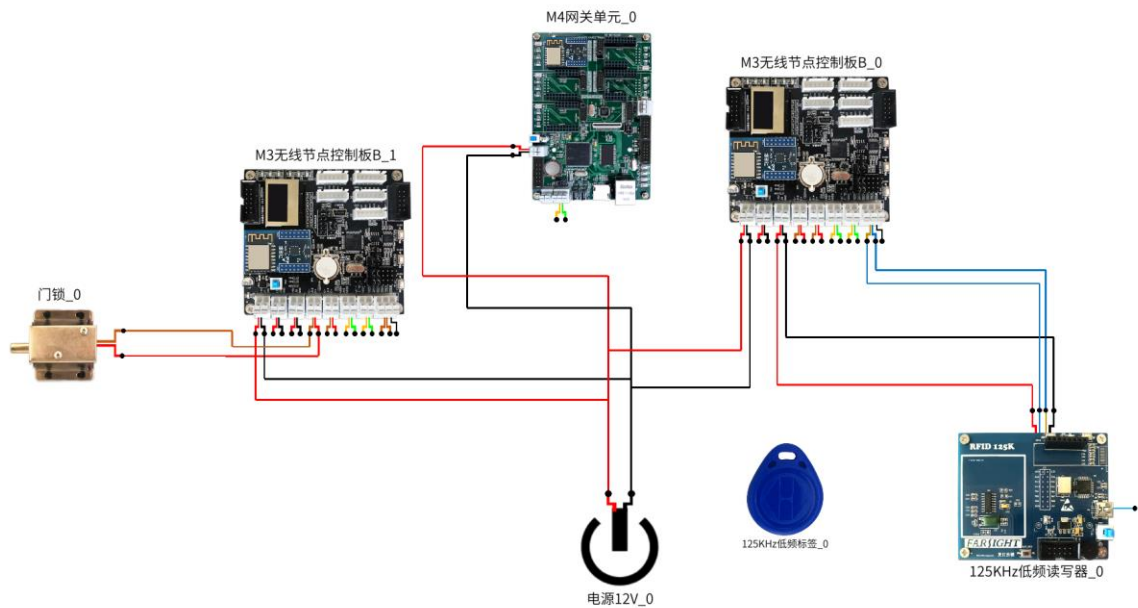
第 11 章 RFID 远程控制门禁实验

11.1 实验目的

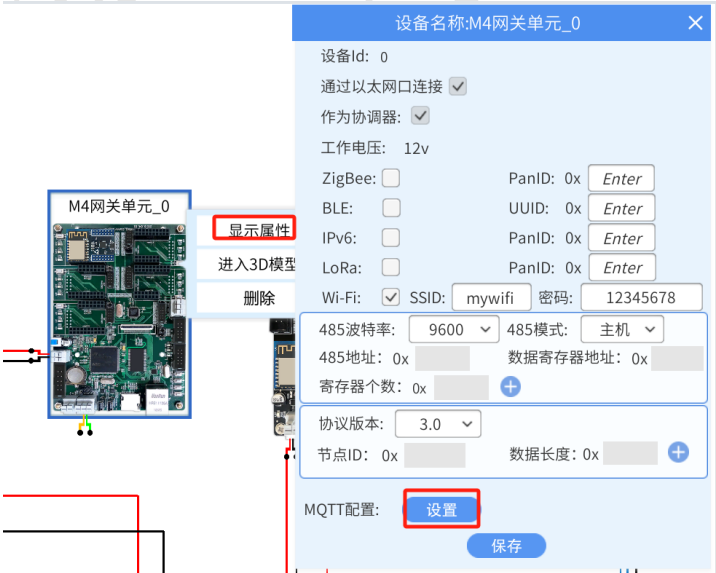
利用 MQTT 获取 125KHz RFID 标签的信息，并根据标签信息进行门禁的控制。

11.2 实验步骤

打开 01-资料光盘镜像\01-程序源码\物联网虚拟仿真软件实验\MQTT 实验下的 6_RFID 远程控制门禁实验.AIOTSIM。



左键点击网关单元，选择属性



连接本地 MQTT Broker，订阅相关话题。

MQTT配置

MQTT:

IP地址: 端口:

ClientID:

用户名:

密码:

与应用层、云端交互的Topic:

订阅:

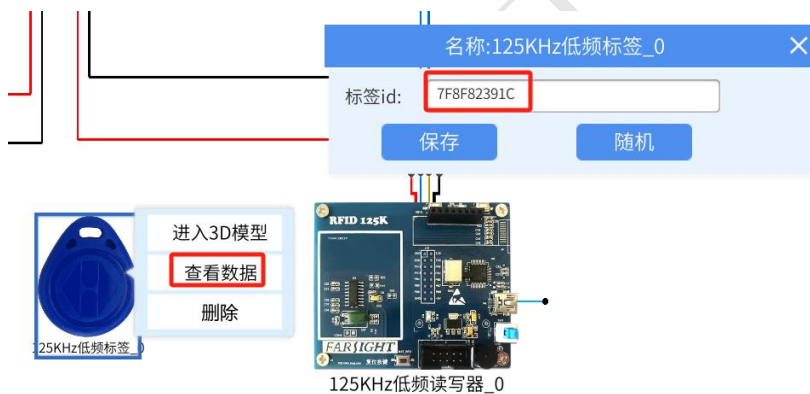
发布:

与硬件交互的Topic:

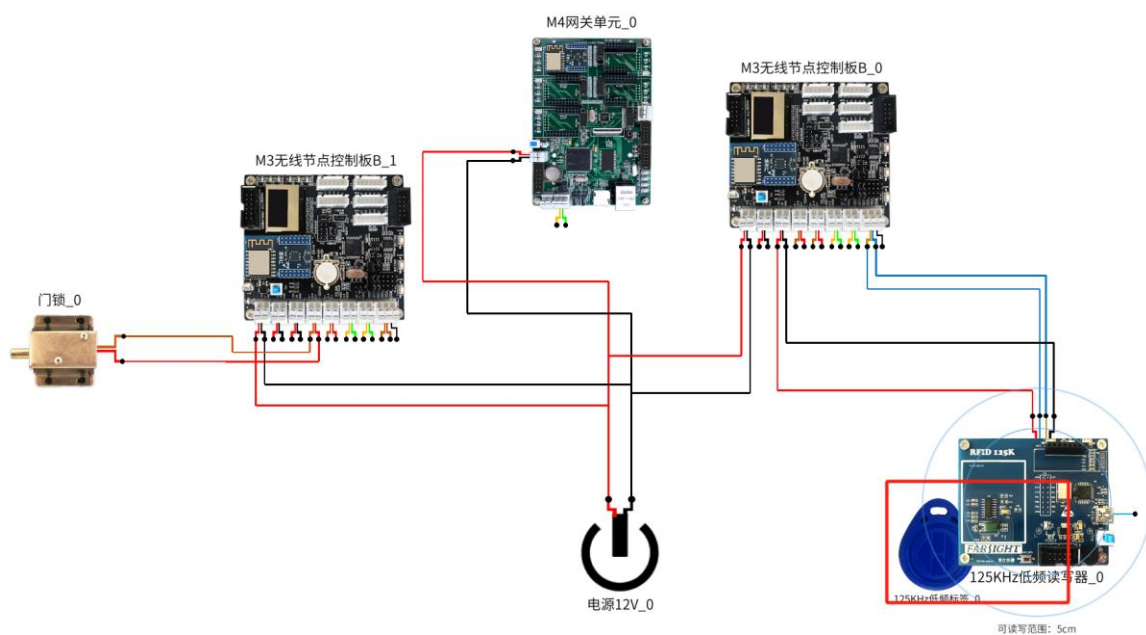
订阅:

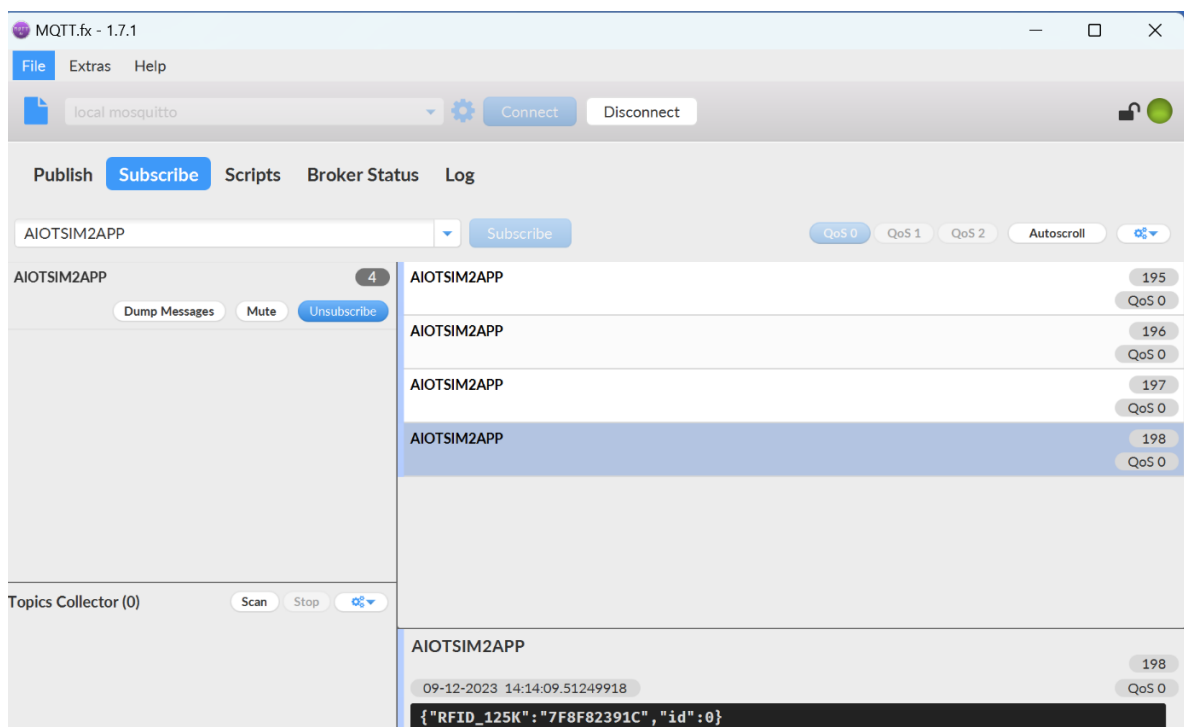
发布:

左键点击 125KHz 标签，选择查看数据，设置一个标签 ID，并保存。

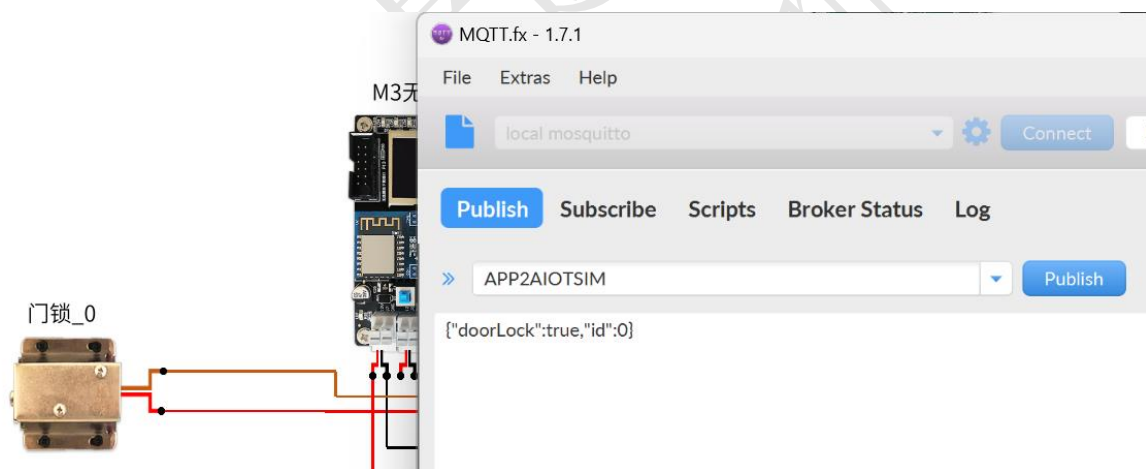


点击运行，并将标签靠近读卡器，即可听到读卡器读卡的声音，并且读到的信息可以通过 MQTT.fx 显示出来。





在 Publish 页面里面，我们向虚拟仿真订阅的话题发布消息 {"doorLock":true,"id":0} 后，门锁可被打开。



我们可以使用下面的 Python 程序实现自动控制。当 MQTT 的消息中包含标签的 ID 时，发布打开门锁的消息。

```
# -*- coding:utf-8 -*-
2 import paho.MQTT.client as MQTT
3 from queue import Queue
4 import JSON
5 import base64
6 import requests
7
8 import time
9
```

```

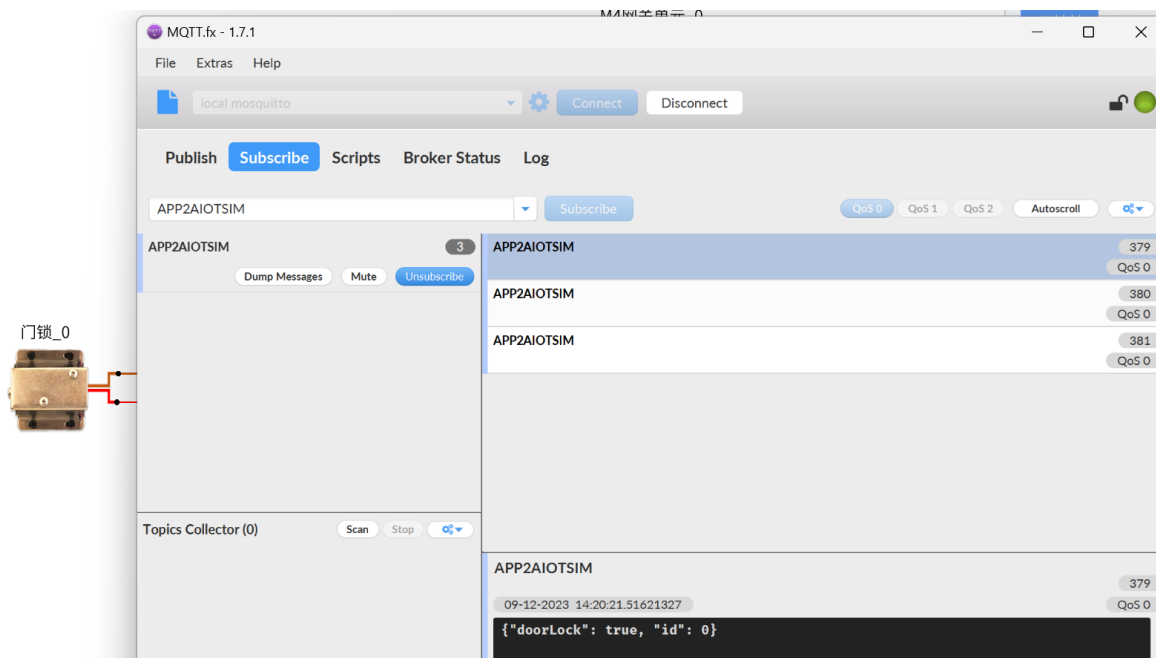
10 class HQYJMQTTClient:
11     def __init__(self, broker_ip: str, broker_port: int):
12         self.MQTT_queue = Queue(255)
13         self.is_connected = False
14         self.client = MQTT.Client()
15         self.client.on_message = self.on_message
16         self.client.on_connect = self.on_connect
17         self.rc = 100
18         try:
19             self.client.connect(broker_ip, broker_port, 3)
20         except Exception as e:
21             print(e)
22
23     def on_message(self, client, userdata, message):
24         msg = JSON.loads(message.payload.decode())
25         self.MQTT_queue.put(msg)
26
27     def on_connect(self, client, userdata, flags, rc):
28         print("result code:", rc)
29         self.rc = rc
30
31
32 # 注意: 在 websocket 端口是 9001, 如使用 Python 代码, 端口需手动改成 1883。
33 hqyj_MQTT_clt = HQYJMQTTClient('127.0.0.1', 1883)
34 hqyj_MQTT_clt.client.loop_start()
35 time.sleep(3)
36 if hqyj_MQTT_clt.rc == 0:
37     print('MQTT connected')
38     hqyj_MQTT_clt.client.subscribe('AIOTSIM2APP', qos=0)
39
40     while True:
41         MQTT_data = hqyj_MQTT_clt.MQTT_queue.get()
42
43         if ('RFID_125K' in MQTT_data) and ('id' in MQTT_data) and (MQTT_data['id'] == 0)
44 :
45             print((MQTT_data["RFID_125K"]))
46             if str((MQTT_data["RFID_125K"])).find(str('7F8F82391C')) > -1:
47                 hqyj_MQTT_clt.client.publish('APP2AIOTSIM', payload=JSON.dumps({"doorLock":
48 True, "id":0}),ensure_ascii=False))
49             else:
50                 hqyj_MQTT_clt.client.publish('APP2AIOTSIM', payload=JSON.dumps({"doorLock":
False, "id":0}),ensure_ascii=False))

```


11.3 实验结果

当用 MQTT.fx 发送{"doorLock":true,"id":0}后，门锁会打开。

运行 Python 程序后，终端中显示如下字符，并且程序会依据传感器上传的数据判断是否要打开或关闭门锁。



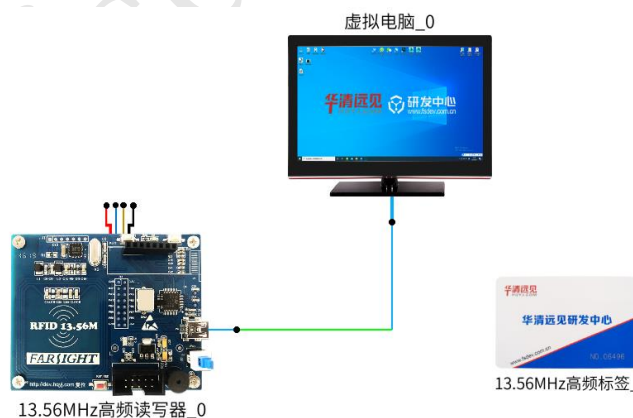
第 12 章 13.56MHz RFID 标签远程读取实验

12.1 实验目的

利用 MQTT 远程读取 13.56MHz 标签的特定扇区特定块。

12.2 实验步骤

打开 01-资料光盘镜像\01-程序源码\物联网虚拟仿真软件实验\MQTT 实验下的 7_13.56MHzRFID 标签远程读取实验.AIOTSIM。



左键点击标签，选择查看数据。



修改对应扇区对应块的值，和密码，点击保存按钮。注意不要轻易修改控制位，否则容易导致数据无法读出。

设备名称:13.56MHz高频标签_0						
扇区	0块	1块	2块	A密码	控制位	B密码
扇区0	F8A2D2870F0804006263646566676869	00000000000000000000000000000000	00000000000000000000000000000000	FFFFFFFFFFFF	FF078069	FFFFFFFFFFFF
扇区1	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	FFFFFFFFFFFF	FF078069	FFFFFFFFFFFF
扇区2	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	FFFFFFFFFFFF	FF078069	FFFFFFFFFFFF
扇区3	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	FFFFFFFFFFFF	FF078069	FFFFFFFFFFFF
扇区4	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	FFFFFFFFFFFF	FF078069	FFFFFFFFFFFF
扇区5	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	FFFFFFFFFFFF	FF078069	FFFFFFFFFFFF

左键点击读写器，选择读写标签，打开相应页面后，检查密钥是否与设置的一致。



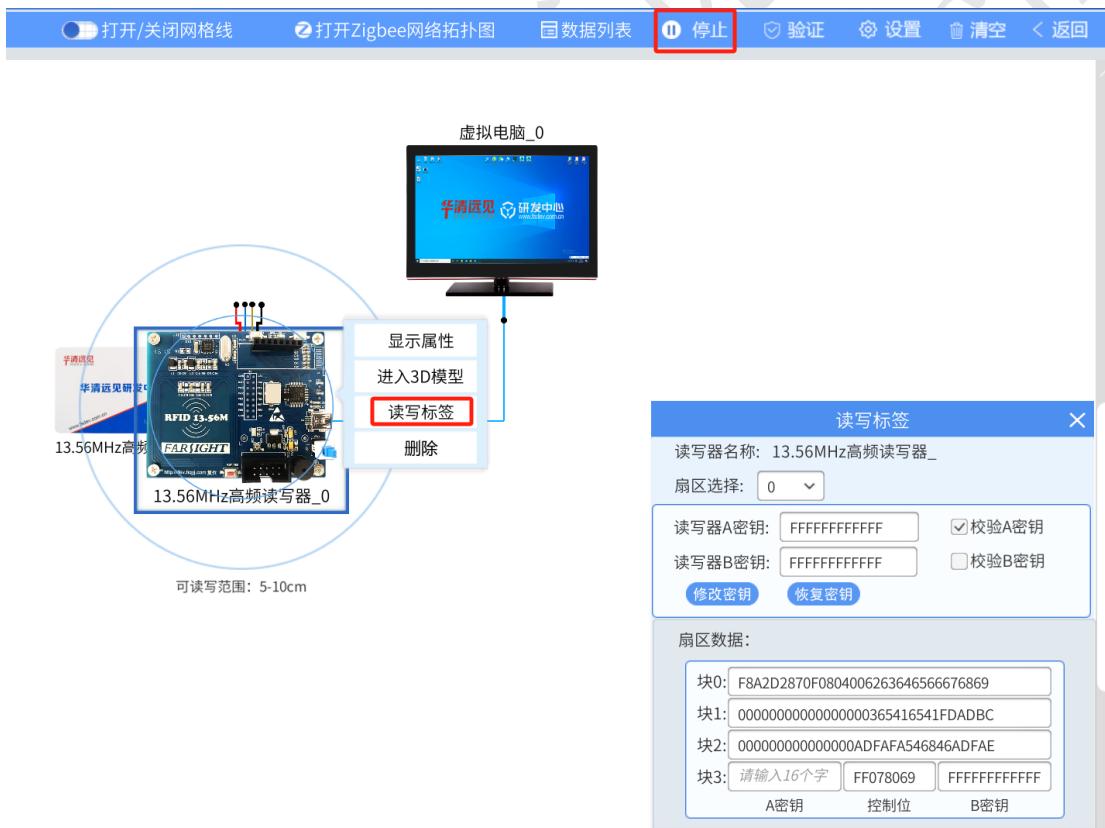
左键点击虚拟电脑，选择显示属性。点击里面的 MQTT 设置按钮。



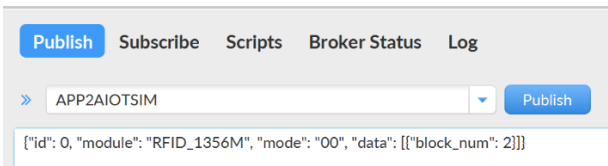
连接本地 MQTT Broker 并订阅相关的话题。

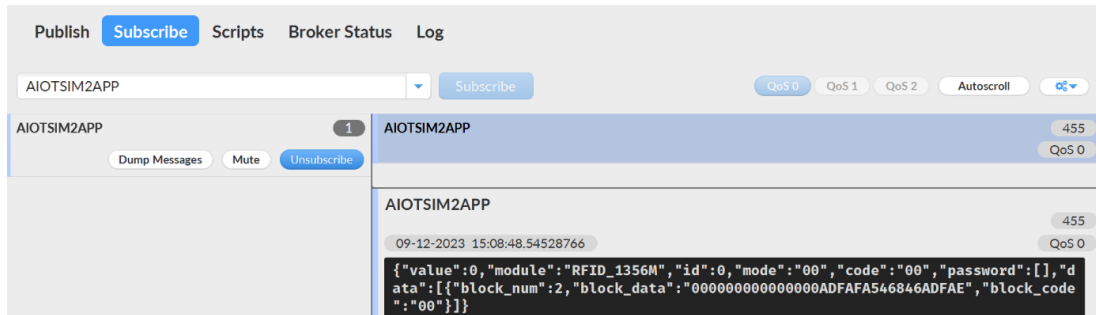


点击运行按钮，将标签靠近读写器，并左键点击读写器选择读写标签，即可看到对应扇区的信息。



在 MQTT.fx 中向对应的话题发布 `{ "id": 0, "module": "RFID_1356M", "mode": "00", "data": [{"block_num": 2}] }`，即可在订阅的话题中接收到返回的信息。





创建 Python 工程，并输入以下代码。该代码的作用是读取扇区 0 块 2 的数据。

```

1 # -*- coding:utf-8 -*-
2 import paho.MQTT.client as MQTT
3 from queue import Queue
4 import JSON
5 import base64
6 import requests
7
8 import time
9
10 class HQYJMqttClient:
11     def __init__(self, broker_ip: str, broker_port: int):
12         self.MQTT_queue = Queue(255)
13         self.is_connected = False
14         self.client = MQTT.Client()
15         self.client.on_message = self.on_message
16         self.client.on_connect = self.on_connect
17         self.rc = 100
18         try:
19             self.client.connect(broker_ip, broker_port, 3)
20         except Exception as e:
21             print(e)
22
23     def on_message(self, client, userdata, message):
24         msg = JSON.loads(message.payload.decode())
25         self.MQTT_queue.put(msg)
26
27     def on_connect(self, client, userdata, flags, rc):
28         print("result code:", rc)
29         self.rc = rc
30
31 # 注意：在 websocket 端口是 9001，如使用 Python 代码，端口需手动改成 1883。
32 hqyj_MQTT_clt = HQYJMqttClient('127.0.0.1', 1883)
33 hqyj_MQTT_clt.client.loop_start()
34 time.sleep(3)
35 if hqyj_MQTT_clt.rc == 0:
36     hqyj_MQTT_clt.client.publish('APP2AIOTSIM', payload=JSON.dumps({'id':0, 'module':"

```

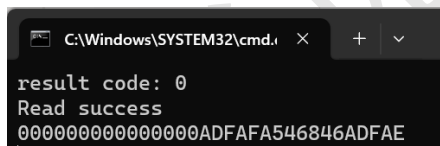
```

36 RFID_1356M", "mode": "00", "data": [{"block_num": 2}]}
37 ), ensure_ascii=False))
38 hqyj_MQTT_clt.client.subscribe('AIOTSIM2APP', qos=0)
39
40 while True:
41     MQTT_data = hqyj_MQTT_clt.MQTT_queue.get()
42
43     if not (MQTT_data) == '':
44         if ('code' in MQTT_data) and (MQTT_data['code'] == "00"):
45             print('Read success')
46             card_data = MQTT_data['data']
47             for card_data in MQTT_data["data"]:
48                 print((card_data['block_data']))

```

12.3 实验结果

运行 Python 程序后，会在终端界面里看到读取到的卡内的数据。



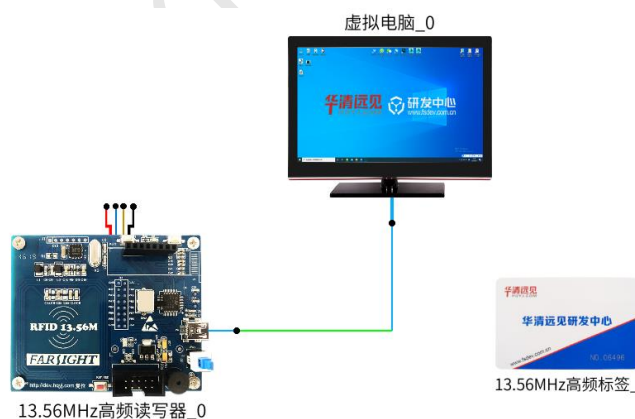
第 13 章 13.56MHz RFID 电子钱包实验

13.1 实验目的

利用 MQTT 远程读取 13.56MHz 标签的特定扇区特定块。

13.2 实验步骤

打开 01-资料光盘镜像\01-程序源码\物联网虚拟仿真软件实验\MQTT 实验下的 7_13.56MHzRFID 标签远程读取实验.AIOTSIM。



左键点击虚拟电脑，点击显示属性后点击设置 MQTT。



连接 MQTT 并订阅相关话题。



将标签靠近读写器后，点击运行按钮。



利用 MQTT.fx 向对应话题发布消息用于初始化

`{"id":0,"module":"RFID_1356M","value":0,"mode":"05","data":[{"block_num":5}]}`，会发现对应扇区对应块的数据变成了 M1 S50 卡规范中电子钱包格式的数据：

设备名称:13.56MHz高频标签_1						
扇区	0块	1块	2块	A密码	控制位	B密码
扇区0	1D553599E40804006263646566676869	00000000000000000000000000000000	00000000000000000000000000000000	FFFFFFFFFFFF	FF078069 →	FFFFFFFFFFFF
扇区1	00000000000000000000000000000000	00000000FFFFFFFF000000005FA05FA	00000000000000000000000000000000	FFFFFFFFFFFF	FF078069 →	FFFFFFFFFFFF
扇区2	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	FFFFFFFFFFFF	FF078069 →	FFFFFFFFFFFF

发布消息用于充值 100 元

{ "id":0,"module":"RFID_1356M","value":100,"mode":"03","data":[{"block_num":5}] }。之后查看标签数据:

设备名称:13.56MHz高频标签_1						
扇区	0块	1块	2块	A密码	控制位	B密码
扇区0	1D553599E40804006263646566676869	00000000000000000000000000000000	00000000000000000000000000000000	FFFFFFFFFFFF	FF078069 →	FFFFFFFFFFFF
扇区1	00000000000000000000000000000000	640000009BFFFFFFFF6400000005FA05FA	00000000000000000000000000000000	FFFFFFFFFFFF	FF078069 →	FFFFFFFFFFFF
扇区2	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	FFFFFFFFFFFF	FF078069 →	FFFFFFFFFFFF
扇区3	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	FFFFFFFFFFFF	FF078069 →	FFFFFFFFFFFF

发布消息用于扣费 10 元

{ "id":0,"module":"RFID_1356M","value":10,"mode":"04","data":[{"block_num":5}] }。之后查看标签数据:

设备名称:13.56MHz高频标签_1						
扇区	0块	1块	2块	A密码	控制位	B密码
扇区0	1D553599E40804006263646566676869	00000000000000000000000000000000	00000000000000000000000000000000	FFFFFFFFFFFF	FF078069 →	FFFFFFFFFFFF
扇区1	00000000000000000000000000000000	5A000000A5FFFFFFFF5A00000005FA05FA	00000000000000000000000000000000	FFFFFFFFFFFF	FF078069 →	FFFFFFFFFFFF
扇区2	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	FFFFFFFFFFFF	FF078069 →	FFFFFFFFFFFF
扇区3	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	FFFFFFFFFFFF	FF078069 →	FFFFFFFFFFFF

我们编写 Python 程序, 让标签在充值 100 元后每隔 3 秒持续扣费 10 元。

```
# -*- coding:utf-8 -*-
2 import paho.MQTT.client as MQTT
3 from queue import Queue
4 import JSON
5 import base64
6 import requests
7 import time
8
9 class HQYJMqttClient:
10     def __init__(self, broker_ip: str, broker_port: int):
11         self.MQTT_queue = Queue(255)
12         self.is_connected = False
13         self.client = MQTT.Client()
14         self.client.on_message = self.on_message
15         self.client.on_connect = self.on_connect
16         self.rc = 100
17         try:
18             self.client.connect(broker_ip, broker_port, 3)
19         except Exception as e:
```



```

print(e)

def on_message(self, client, userdata, message):
    msg = JSON.loads(message.payload.decode())
    self.MQTT_queue.put(msg)

def on_connect(self, client, userdata, flags, rc):
    print("result code:", rc)
    self.rc = rc

# 注意: 在 websocket 端口是 9001, 如使用 Python 代码, 端口需手动改成 1883。
hqyj_MQTT_clt = HQYJMQttClient('127.0.0.1', 1883)
hqyj_MQTT_clt.client.loop_start()
time.sleep(3)
if hqyj_MQTT_clt.rc == 0:
    print('MQTT Connect')
    hqyj_MQTT_clt.client.subscribe('AIOTSIM2APP', qos=0)

    hqyj_MQTT_clt.client.publish('APP2AIOTSIM', payload=JSON.dumps(({ "id":0, "module": "RFID_1356M", "value":0, "mode": "05", "data": [{ "block_num":5 }] }, ensure_ascii=False))
    time.sleep(3)
    hqyj_MQTT_clt.client.publish('APP2AIOTSIM', payload=JSON.dumps(({ "id":0, "module": "RFID_1356M", "value":100, "mode": "03", "data": [{ "block_num":5 }] }, ensure_ascii=False))
    print('Recharge 100')
    time.sleep(3)
    while True:
        hqyj_MQTT_clt.client.publish('APP2AIOTSIM', payload=JSON.dumps(({ "id":0, "module": "RFID_1356M", "value":10, "mode": "04", "data": [{ "block_num":5 }] }, ensure_ascii=False))
        print('Deduct 10')
        time.sleep(3)

```

13.3 实验结果

运行 Python 程序后，我们不断查看标签内的数据，会发现对应块内的数据会发生变化。

设备名称:13.56MHz高频标签_1						
扇区	0块	1块	2块	A密码	控制位	B密码
扇区0	1D53599E4080406263646566676869	00000000000000000000000000000000	00000000000000000000000000000000	FFFFFFFFFFFF	FF078069 →	FFFFFFFFFFFF
扇区1	00000000000000000000000000000000	5A000000A5FFFFFFFF5A00000005FA05FA	00000000000000000000000000000000	FFFFFFFFFFFF	FF078069 →	FFFFFFFFFFFF
扇区2	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	FFFFFFFFFFFF	FF078069 →	FFFFFFFFFFFF

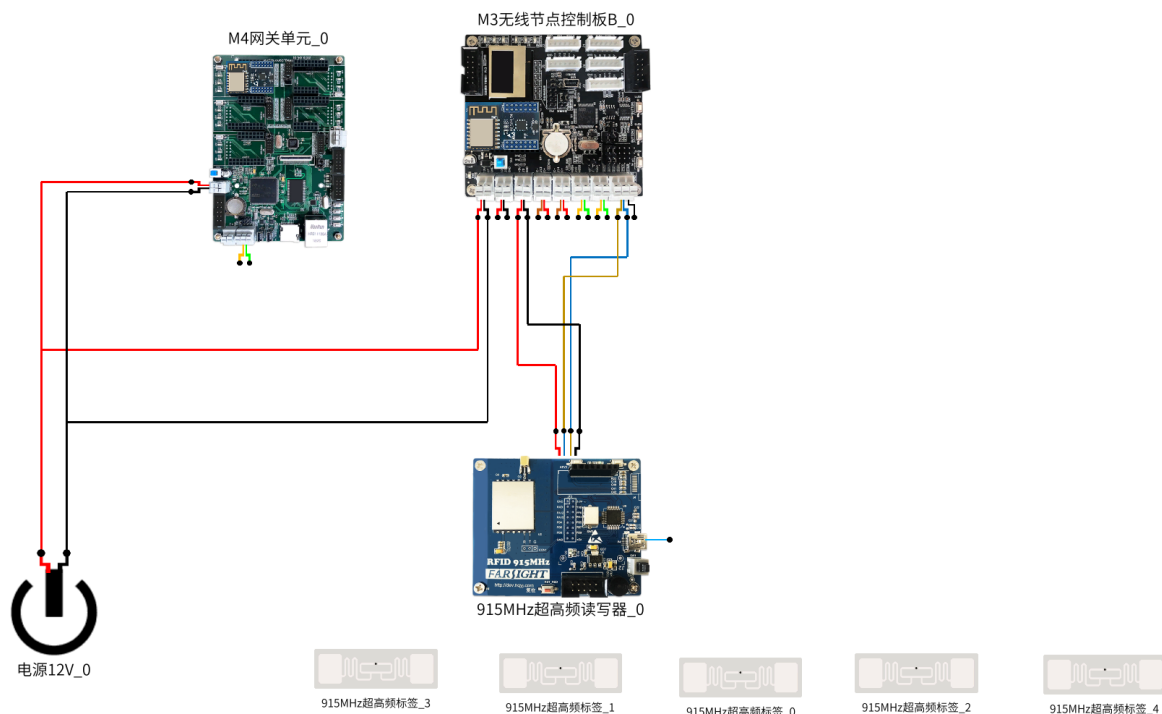
第 14 章 915MHz RFID 库存盘点实验

14.1 实验目的

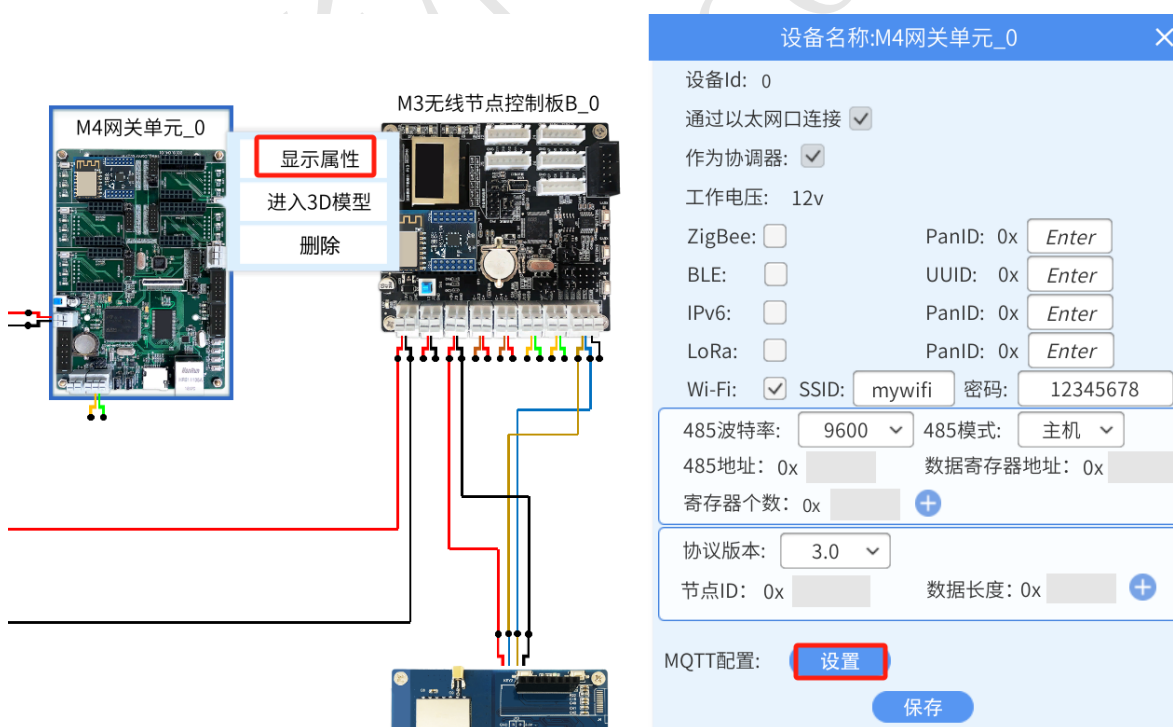
利用虚拟仿真里的 915MHz 读写器和多个标签模拟库存盘点，将识别到的标签的 EPC 发送给 MQTT 相应的话题，并打印识别到的标签数量。

14.2 实验步骤

打开 01-资料光盘镜像\01-程序源码\物联网虚拟仿真软件实验\MQTT 实验下的 9_915MHzRFID 库存盘点实验.AIOTSIM。



左键点击网关单元，选择显示属性，点击 MQTT 设置。



连接 MQTT Broker 并订阅相关话题。

Mqtt配置

MQTT:

IP地址: 127.0.0.1 端口: 1883

ClientID: c1d7538a9f163cb5becabd2a253ca8b 生成

用户名: 请输入用户名

密码: 请输入密码

断开

与应用层、云端交互的Topic:

订阅: APP2AIOTSIM 复制

发布: AIOTSIM2APP 复制

生成Topic 取消订阅

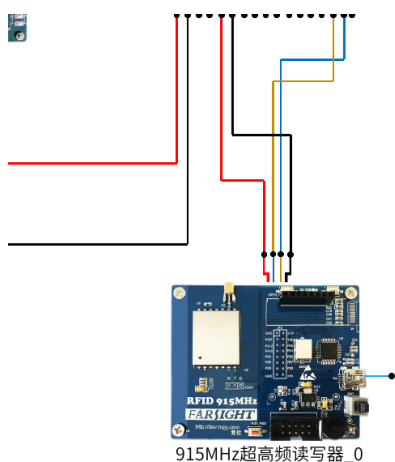
与硬件交互的Topic:

订阅: Device2AIOTSIM 复制

发布: AIOTSIM2Device 复制

生成Topic 订阅

记住每个标签的 EPC 的前 4 个字节。



名称:915MHz超高频标签_1

EPC区: 29F5 3000 C5F538F3500EBB4F12F614ED0000

CRC16 PC

温馨提示: CRC16为只读, 由程序自动计算,不需要手动输入, PC为计算EPC长度。

TID区: 4851594A500EBB4F12F614ED

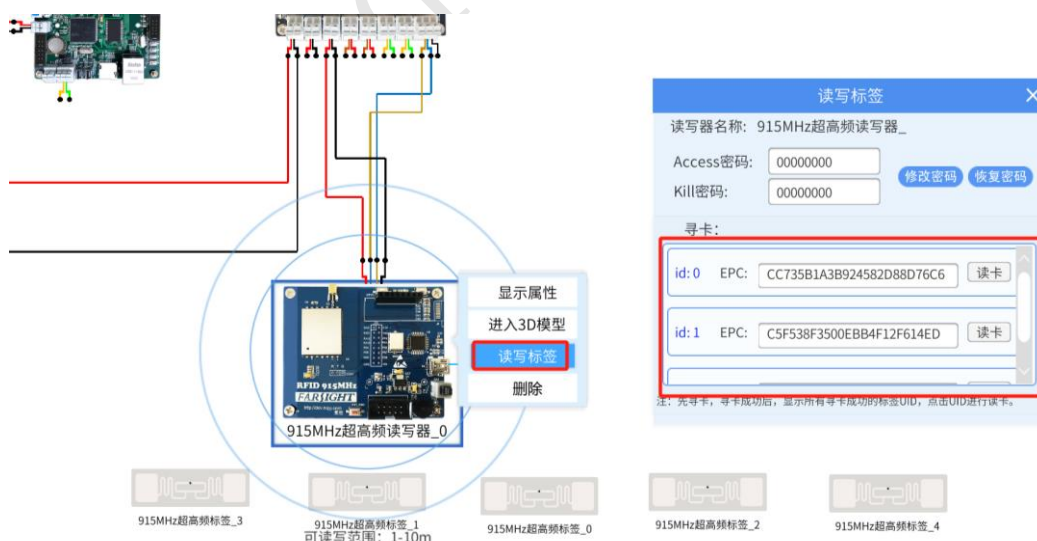
Password区: 0000000000000000

User区: 00000000000000000000000000000000

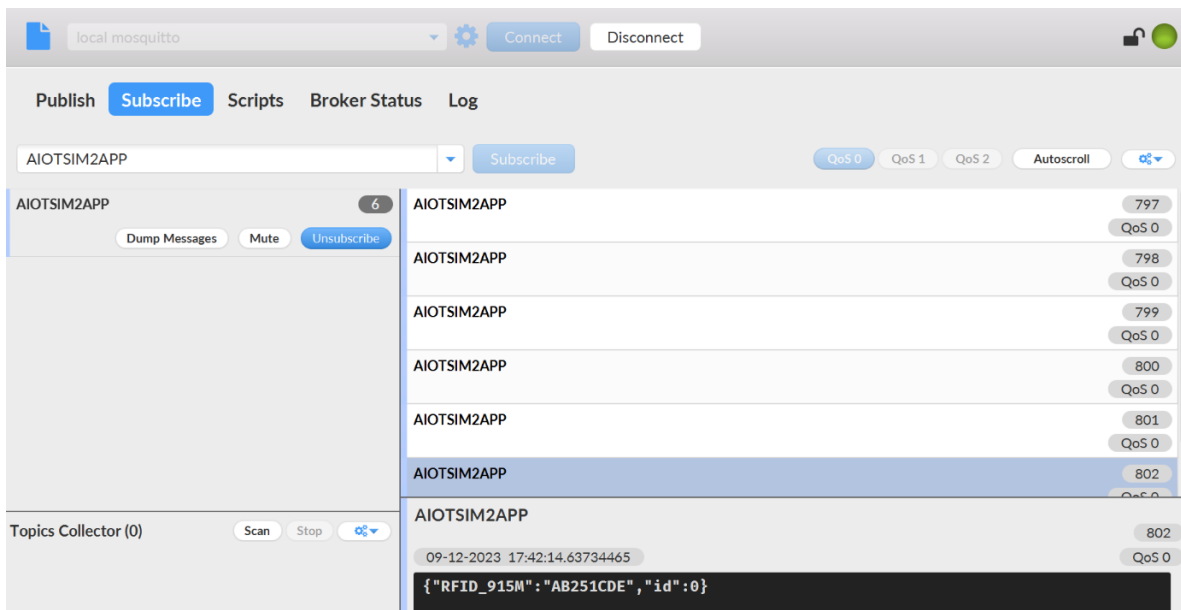
随机 保存



运行后，左键点击读写器，选择读写标签，即可在右侧看到读到的多张卡的 EPC。



在 MQTT.fx 上，可以通过订阅对应的话题得到仿真系统发送给 MQTT 的消息。



我们编写 Python 程序，让程序识别读卡器读到了几张标签。

```
1 # -*- coding:utf-8 -*-
2 import paho.MQTT.client as MQTT
3 from queue import Queue
4 import JSON
5 import base64
6 import requests
7 import time
8
9 class HQYJMqttClient:
10     def __init__(self, broker_ip: str, broker_port: int):
11         self.MQTT_queue = Queue(255)
12         self.is_connected = False
13         self.client = MQTT.Client()
14         self.client.on_message = self.on_message
15         self.client.on_connect = self.on_connect
16         self.rc = 100
17         try:
18             self.client.connect(broker_ip, broker_port, 3)
19         except Exception as e:
20             print(e)
21
22     def on_message(self, client, userdata, message):
23         msg = JSON.loads(message.payload.decode())
24         self.MQTT_queue.put(msg)
25
26     def on_connect(self, client, userdata, flags, rc):
27         print("result code:", rc)
```

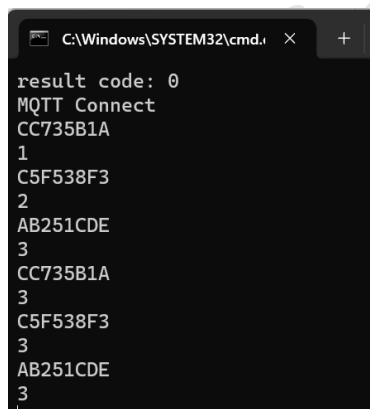
```

27     self.rc = rc
28 # 注意: 在 websocket 端口是 9001, 如使用 Python 代码, 端口需手动改成 1883。
29 hqyj_MQTT_clt = HQYJMQTTClient('127.0.0.1', 1883)
30 hqyj_MQTT_clt.client.loop_start()
31 time.sleep(3)
32 if hqyj_MQTT_clt.rc == 0:
33     print('MQTT Connect')
34     card_num = int('0')
35     data_base = []
36     data_base.append('CC735B1A')
37     data_base.append('9986124D')
38     data_base.append('C5F538F3')
39     data_base.append('AB251CDE')
40     data_base.append('9986124D1')
41     hqyj_MQTT_clt.client.subscribe('AIOTSIM2APP', qos=0)
42
43     while True:
44         MQTT_data = hqyj_MQTT_clt.MQTT_queue.get()
45
46         if ('RFID_915M' in MQTT_data) and ('id' in MQTT_data) and (MQTT_data['id'] ==
47 0):
48             print((MQTT_data["RFID_915M"]))
49             if str((data_base)).find(str((MQTT_data["RFID_915M"]))) > -1:
50                 card_num = int(((card_num) + 1))
51                 while not not str((data_base)).find(str((MQTT_data["RFID_915M"]))) > -1:
52                     data_base.remove((MQTT_data["RFID_915M"]))
53             print((card_num))
54

```

14.3 实验结果

运行程序后, 可以看见中断中会打印出对应的标签 EPC 和总共读到的标签数量。



```

C:\Windows\SYSTEM32\cmd.exe
result code: 0
MQTT Connect
CC735B1A
1
C5F538F3
2
AB251CDE
3
CC735B1A
3
C5F538F3
3
AB251CDE
3

```

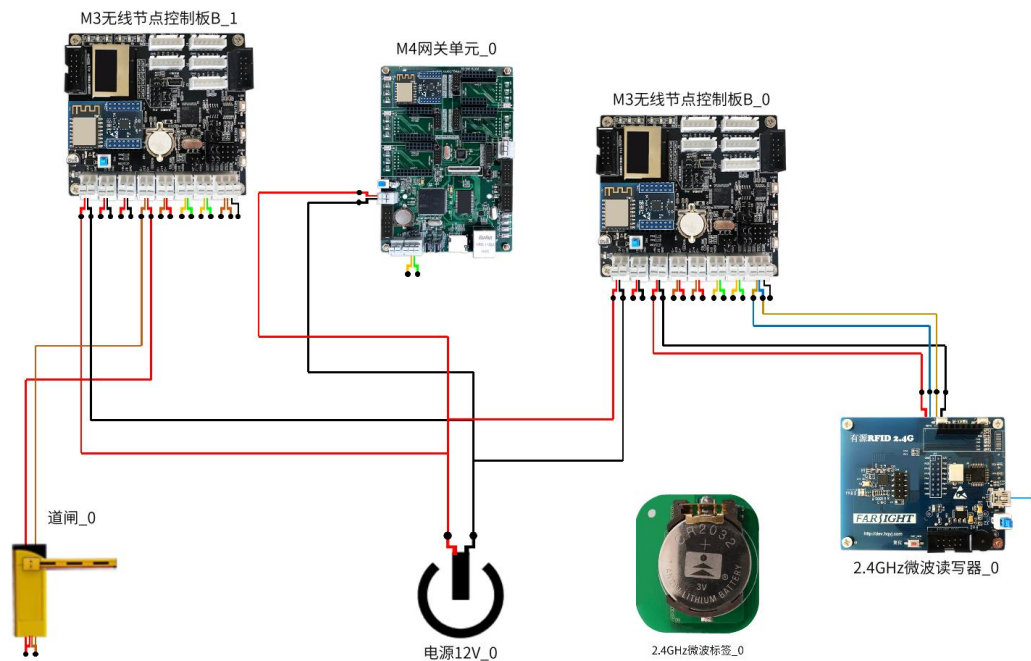
第 15 章 2.4GHz RFID 停车场闸机控制系统实验

15.1 实验目的

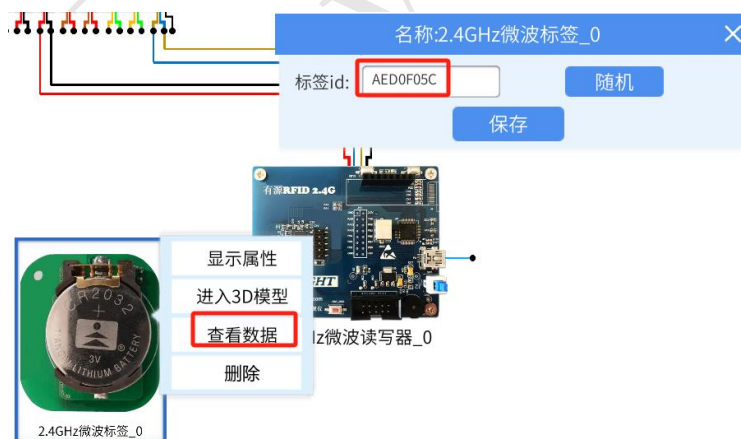
使用 2.4GHz 有源 RFID 标签和读卡器，结合 MQTT，对道闸进行远程控制。

15.2 实验步骤

打开 01-资料光盘镜像\01-程序源码\物联网虚拟仿真软件实验\MQTT 实验下的 10_RFID 停车场闸机控制系统实验.AIOTSIM。



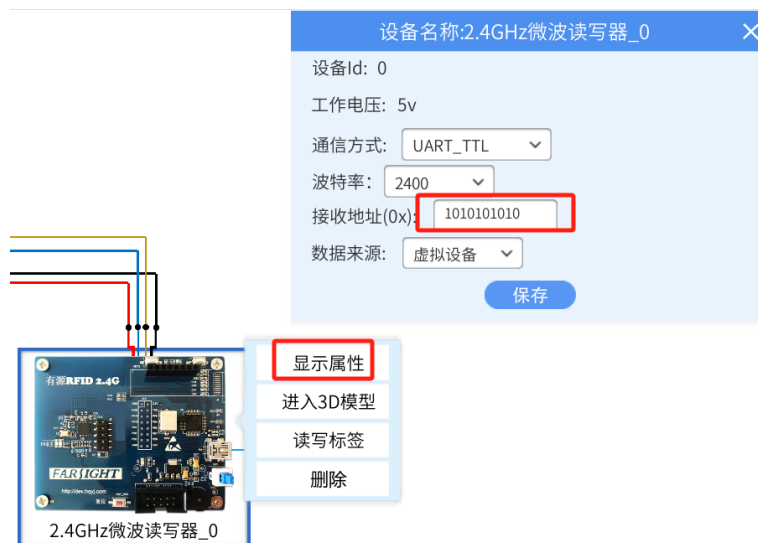
为 2.4GHz 有源标签随机生成一个 ID。



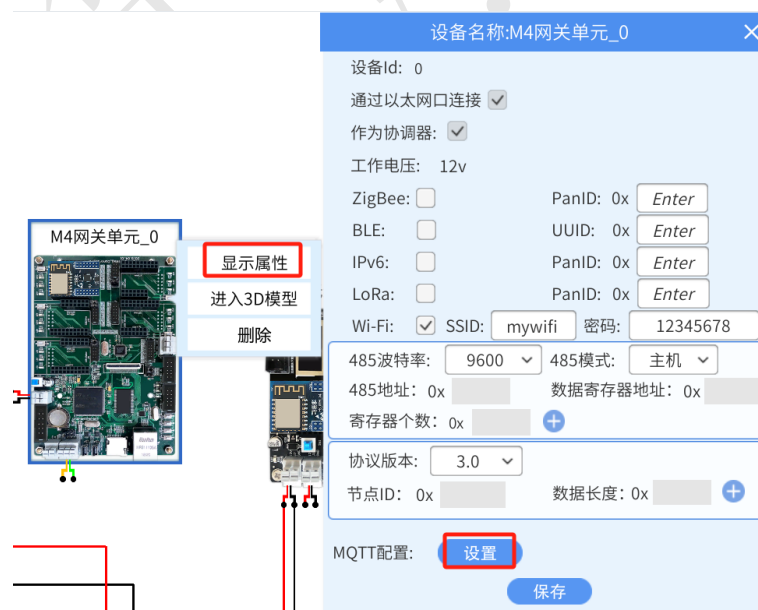
再设置一个发送地址。



将该发送地址设置为读写器的接收地址。



左键点击网关单元，选择显示属性，点击 MQTT 设置。



连接 MQTT 并订阅相关话题。



MQTT配置

MQTT:

IP地址: 127.0.0.1 端口: 1883

ClientID: c1d7538a9f163cb5becabd2a253ca8b 生成

用户名: 请输入用户名

密码: 请输入密码

断开

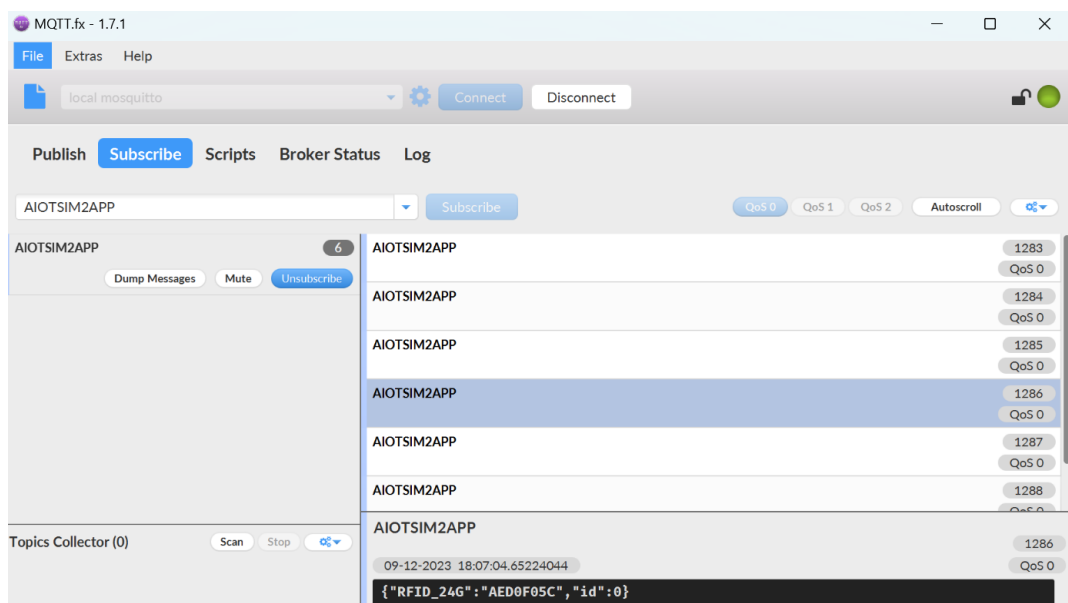
与应用层、云端交互的Topic:

订阅: APP2AIOTSIM 复制

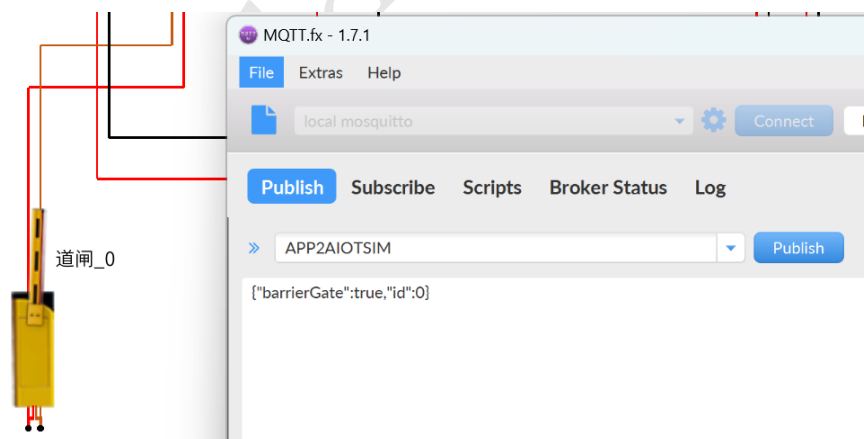
发布: AIOTSIM2APP 复制

生成Topic 取消订阅

点击运行后，可以在 MQTT.fx 上看到相关话题的信息。



向对应话题发送{"barrierGate":true,"id":0}后，可开启道闸。



下面我们用 Python 实现相应的控制。

```
# -*- coding:utf-8 -*-
2 import paho.MQTT.client as MQTT
3 from queue import Queue
```

```

4 import JSON
5 import base64
6 import requests
7 import time
8
9 class HQYJMQTTClient:
10     def __init__(self, broker_ip: str, broker_port: int):
11         self.MQTT_queue = Queue(255)
12         self.is_connected = False
13         self.client = MQTT.Client()
14         self.client.on_message = self.on_message
15         self.client.on_connect = self.on_connect
16         self.rc = 100
17         try:
18             self.client.connect(broker_ip, broker_port, 3)
19         except Exception as e:
20             print(e)
21
22     def on_message(self, client, userdata, message):
23         msg = JSON.loads(message.payload.decode())
24         self.MQTT_queue.put(msg)
25
26     def on_connect(self, client, userdata, flags, rc):
27         print("result code:", rc)
28         self.rc = rc
29
30 # 注意: 在 websocket 端口是 9001, 如使用 Python 代码, 端口需手动改成 1883。
31 hqyj_MQTT_clt = HQYJMQTTClient('127.0.0.1', 1883)
32 hqyj_MQTT_clt.client.loop_start()
33 time.sleep(1)
34 if hqyj_MQTT_clt.rc == 0:
35     print('MQTT Connected')
36     hqyj_MQTT_clt.client.subscribe('AIOTSIM2APP', qos=0)
37
38     while True:
39         MQTT_data = hqyj_MQTT_clt.MQTT_queue.get()
40
41         if ('RFID_24G' in MQTT_data) and ('id' in MQTT_data) and (MQTT_data['id'] == 0):
42             print((MQTT_data["RFID_24G"]))
43             if str((MQTT_data["RFID_24G"])).find(str('AED0F05C')) > -1:
44                 print('correct')
45                 hqyj_MQTT_clt.client.publish('APP2AIOTSIM', payload=JSON.dumps({"barrierGate
46 ":True, "id":0}),ensure_ascii=False))

```


15.3 实验结果

运行 Python 后，在中断中可显示下列信息，同时道闸会自动打开。

```
C:\Windows\SYSTEM32\cmd.exe

result code: 0
MQTT Connected
AED0F05C
correct
```

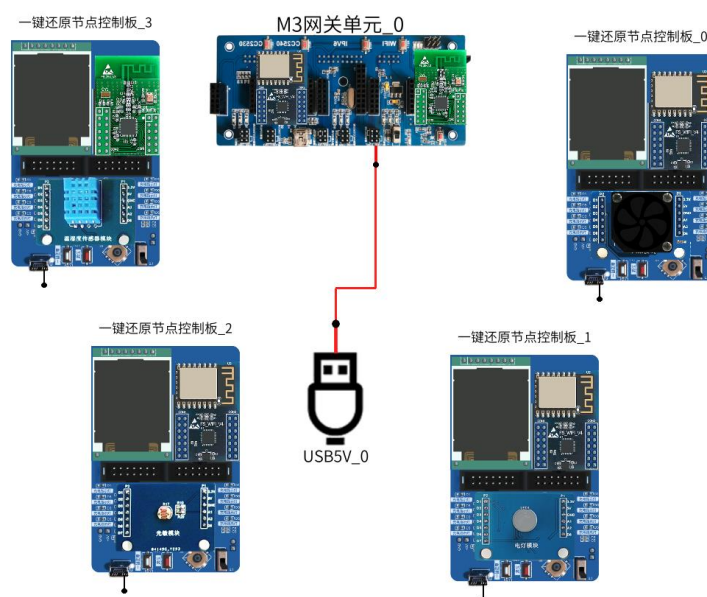
第 16 章 基于一键还原节点的多传感器控制实验

16.1 实验目的

利用虚拟仿真系统中的一键还原节点设置多个传感器，并连接 MQTT，利用 MQTT 收集和

16.2 实验步骤

打开 01-资料光盘镜像\01-程序源码\物联网虚拟仿真软件实验\MQTT 实验下的 11_基于一键还原节点的多传感器控制实验.AIOTSIM。



点击数据列表，打开数据。



左键点击网关单元，选择显示属性，设置 MQTT。

设备名称:M3网关单元_0

设备Id: 0

通过中间件连接网 ☒

作为协调器: ☒

工作电压: USB5v

ZigBee: ☒ PanID: 0x 0001

BLE: ☐ UUID: 0x Enter

IPv6: ☐ PanID: 0x Enter

LoRa: ☐ PanID: 0x Enter

Wi-Fi: ☒ SSID: mywifi 密码: 12345678

协议版本: 2.1

设备类型: 0x 设备地址: 0x

数据长度: 0x

MQTT配置: 设置

保存

MQTT配置

MQTT:

IP地址: 127.0.0.1 端口: 1883

ClientID: c1d7538a9f163cb5becabd2a253ca8b 生成

用户名: 请输入用户名

密码: 请输入密码

断开

与应用层、云端交互的Topic:

订阅: APP2AIOTSIM 复制

发布: AIOTSIM2APP 复制

生成Topic 取消订阅

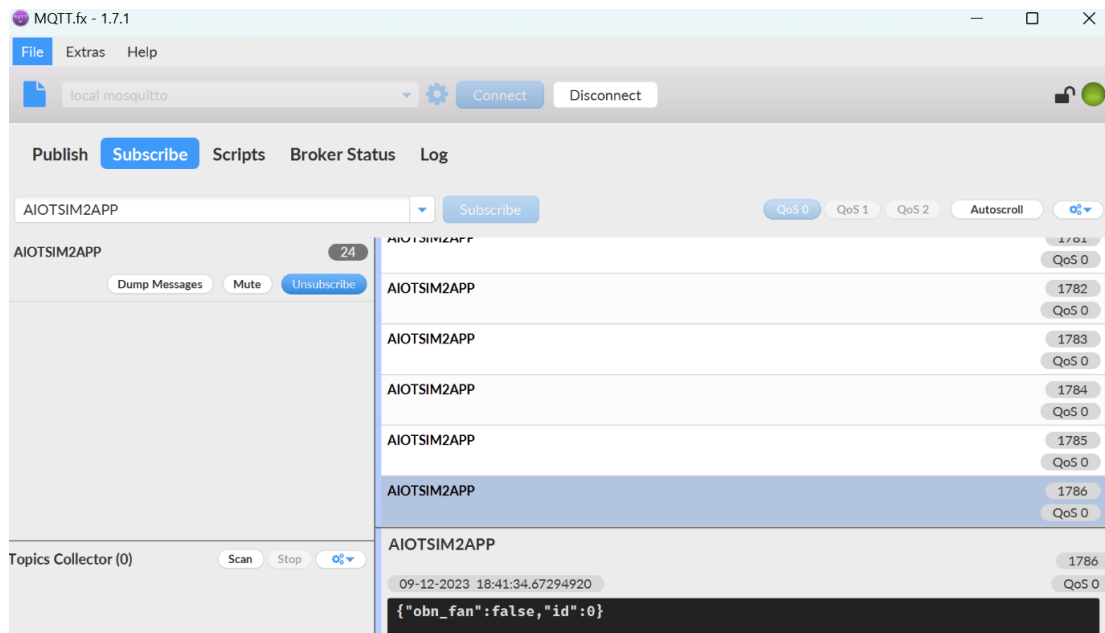
与硬件交互的Topic:

订阅: Device2AIOTSIM 复制

发布: AIOTSIM2Device 复制

生成Topic 订阅

运行虚拟仿真系统后，我们可以在 MQTT 的相关话题下看到网关发来的数据。



编写 Python 程序，实现传感器数据的自动收集和控制。

Python Code

```
# -*- coding:utf-8 -*-
2 import paho.MQTT.client as MQTT
3 from queue import Queue
4 import JSON
5 import base64
6 import requests
7 import time
8
9 class HQYJMqttClient:
10     def __init__(self, broker_ip: str, broker_port: int):
11         self.MQTT_queue = Queue(255)
12         self.is_connected = False
13         self.client = MQTT.Client()
14         self.client.on_message = self.on_message
15         self.client.on_connect = self.on_connect
16         self.rc = 100
17         try:
18             self.client.connect(broker_ip, broker_port, 3)
19         except Exception as e:
20             print(e)
21
22     def on_message(self, client, userdata, message):
23         msg = JSON.loads(message.payload.decode())
24         self.MQTT_queue.put(msg)
25
26     def on_connect(self, client, userdata, flags, rc):
```

```

27     print("result code:", rc)
28     self.rc = rc
29
30 # 注意: 在 websocket 端口是 9001, 如使用 Python 代码, 端口需手动改成 1883。
31 hqyj_MQTT_clt = HQYJMQTTClient('127.0.0.1', 1883)
32 hqyj_MQTT_clt.client.loop_start()
33 time.sleep(1)
34 if hqyj_MQTT_clt.rc == 0:
35     print('MQTT Connected')
36     hqyj_MQTT_clt.client.subscribe('AIOTSIM2APP', qos=0)
37
38     while True:
39         MQTT_data = hqyj_MQTT_clt.MQTT_queue.get()
40
41         if ('obn_tem' in MQTT_data) and ('id' in MQTT_data) and (MQTT_data['id'] == 0):
42             print((float(MQTT_data["obn_tem"])))
43             if (float(MQTT_data["obn_tem"])) > 30:
44                 hqyj_MQTT_clt.client.publish('APP2AIOTSIM', payload=JSON.dumps({"obn_fan":True, "id":0}),ensure_ascii=False))
45             if (float(MQTT_data["obn_tem"])) < 30:
46                 hqyj_MQTT_clt.client.publish('APP2AIOTSIM', payload=JSON.dumps({"obn_fan":False, "id":0}),ensure_ascii=False))
47             time.sleep(2)
48         if ('obn_photosensitive' in MQTT_data) and ('id' in MQTT_data) and (MQTT_data['id'] == 0):
49             print((float(MQTT_data["obn_photosensitive"])))
50             if (float(MQTT_data["obn_photosensitive"])) > 2000:
51                 hqyj_MQTT_clt.client.publish('APP2AIOTSIM', payload=JSON.dumps({"obn_lamp":True, "id":0}),ensure_ascii=False))
52             if (float(MQTT_data["obn_photosensitive"])) < 2000:
53                 hqyj_MQTT_clt.client.publish('APP2AIOTSIM', payload=JSON.dumps({"obn_lamp":False, "id":0}),ensure_ascii=False))
54             time.sleep(2)

```

16.3 实验结果

运行后, 终端会打印出相应的传感器数据, 并且会自动控制节点的执行器。



版本控制

版本号	修改内容	修改人	修改日期
V1.0	创建文档，设置 15 个实验。	AHN	2023.12.09
V1.1	增加 MQTT 协议 JSON 格式	AHN	2024.4.12